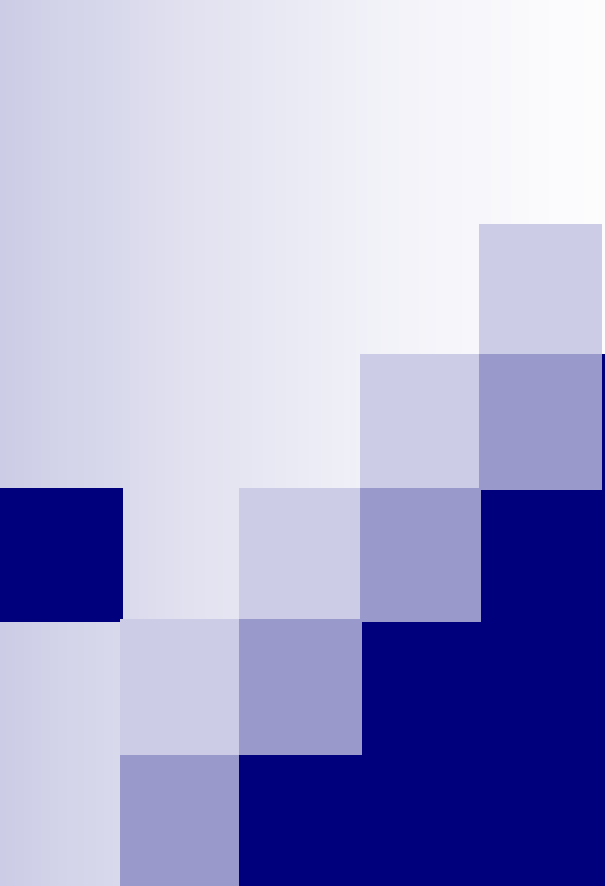




数据结构

第七章 图

主讲：陈锦秀

厦门大学信息学院计算机系

第七章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 图的连通性问题

7.5 有向无环图及其应用

7.6 最短路径

7.1 图的定义和术语

- 图的结构定义: 图是由一个顶点集 V 和一个弧集 R 构成的数据结构。

$$\text{Graph} = (V, R)$$

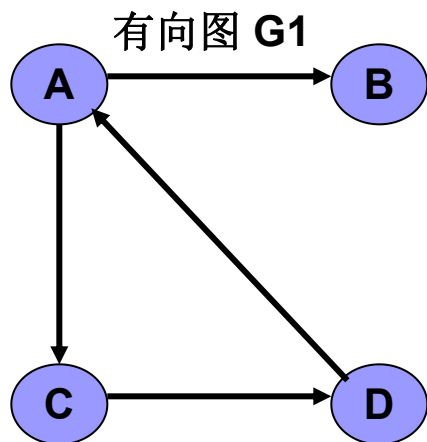
其中, $VR = \{ \langle v, w \rangle \mid v, w \in V \text{ 且 } P(v, w) \}$

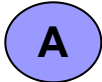
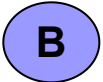
$\langle v, w \rangle$ 表示从 v 到 w 的一条弧, 并称 v 为弧头, w 为弧尾。

谓词 $P(v, w)$ 定义了弧 $\langle v, w \rangle$ 的意义或信息。

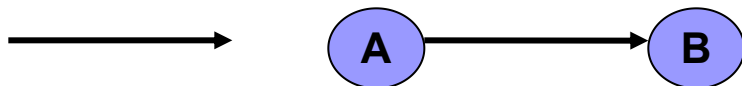
7.1 图的定义和术语

若 $\langle v, w \rangle \in VR$
必有 $\langle w, v \rangle \in VR$,

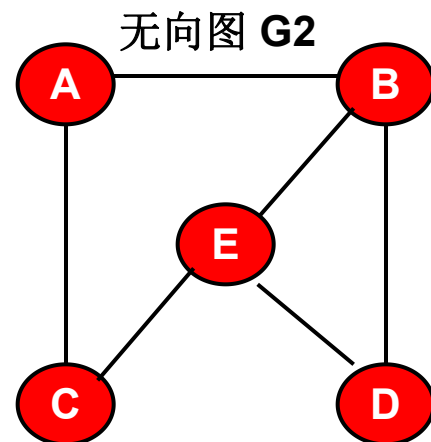


• 结点或 顶点:  

• 有向边（弧）、弧尾或初始结点、弧头或终止结点



• 有向图: $G_1 = (V_1, \{A_1\})$
 $V_1 = \{A, B, C, D\}$
 $A_1 = \{\langle A, B \rangle, \langle A, C \rangle,$
 $\langle C, D \rangle, \langle D, A \rangle\}$



• 结点或 顶点:  

• 无向边或边

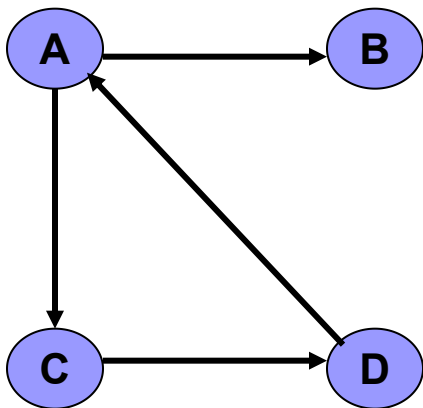


• 无向图: $G_2 = (V_2, \{A_2\})$
 $V_2 = \{A, B, C, D, E\}$
 $A_2 = \{(A, B), (A, C), (B, D),$
 $(B, E), (C, E), (D, E)\}$

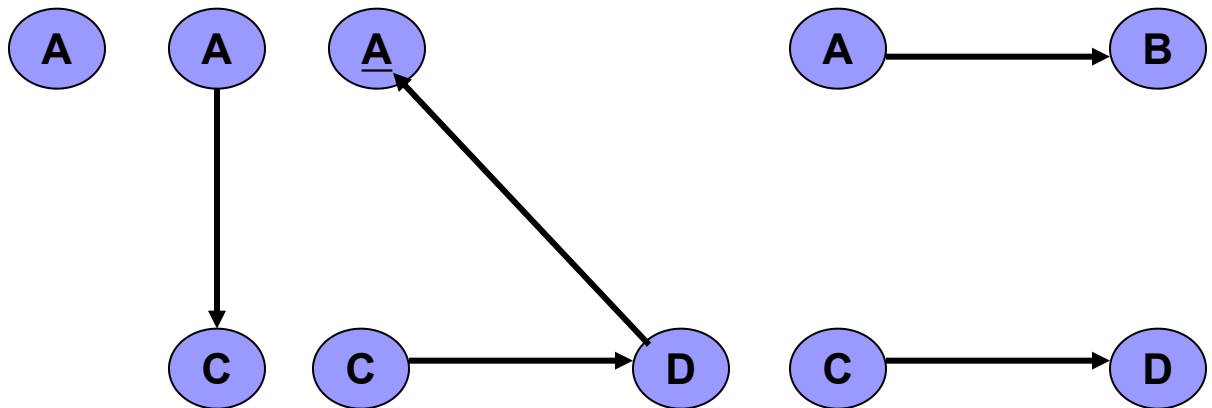
7.1 图的定义和术语——子图

设图 $G=(V,\{VR\})$ 和图 $G'=(V',\{VR'\})$,
且 $V'\subseteq V, VR'\subseteq VR$,
则称 G' 为 G 的子图。

有向图 G_1

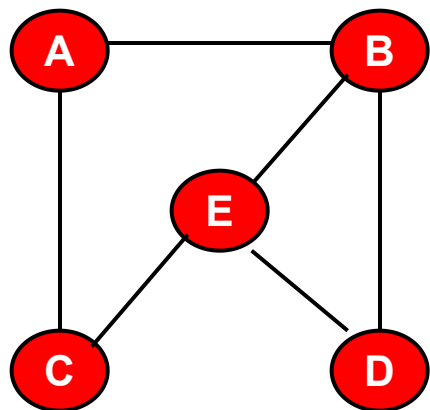


有向图 G_1 的子图

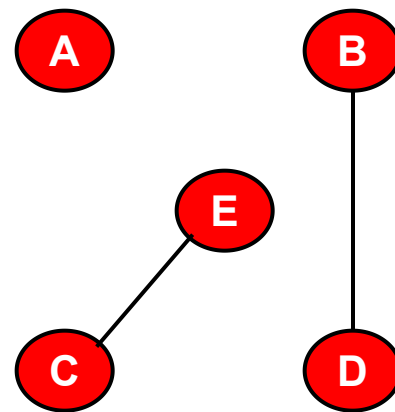
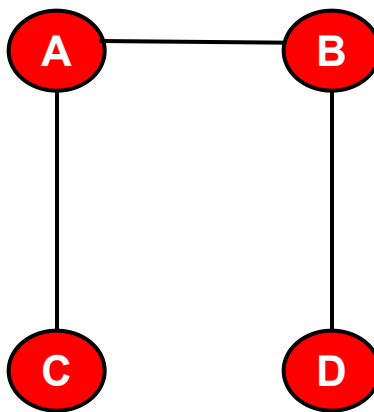
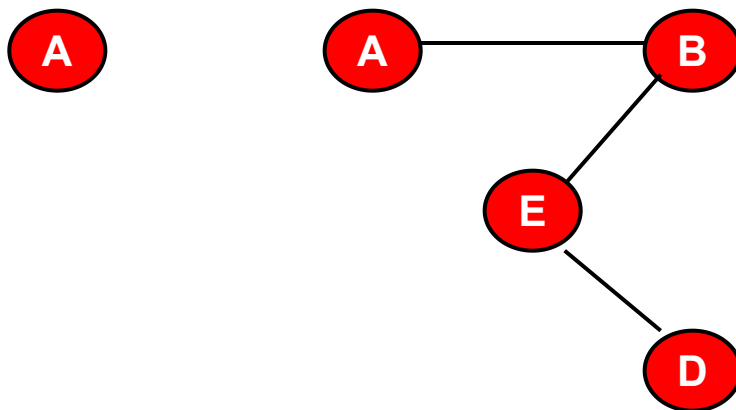


7.1 图的定义和术语——子图

无向图 G_2



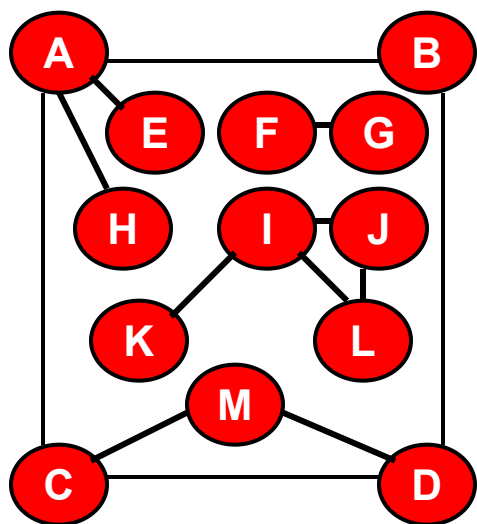
无向图 G_2 的子图



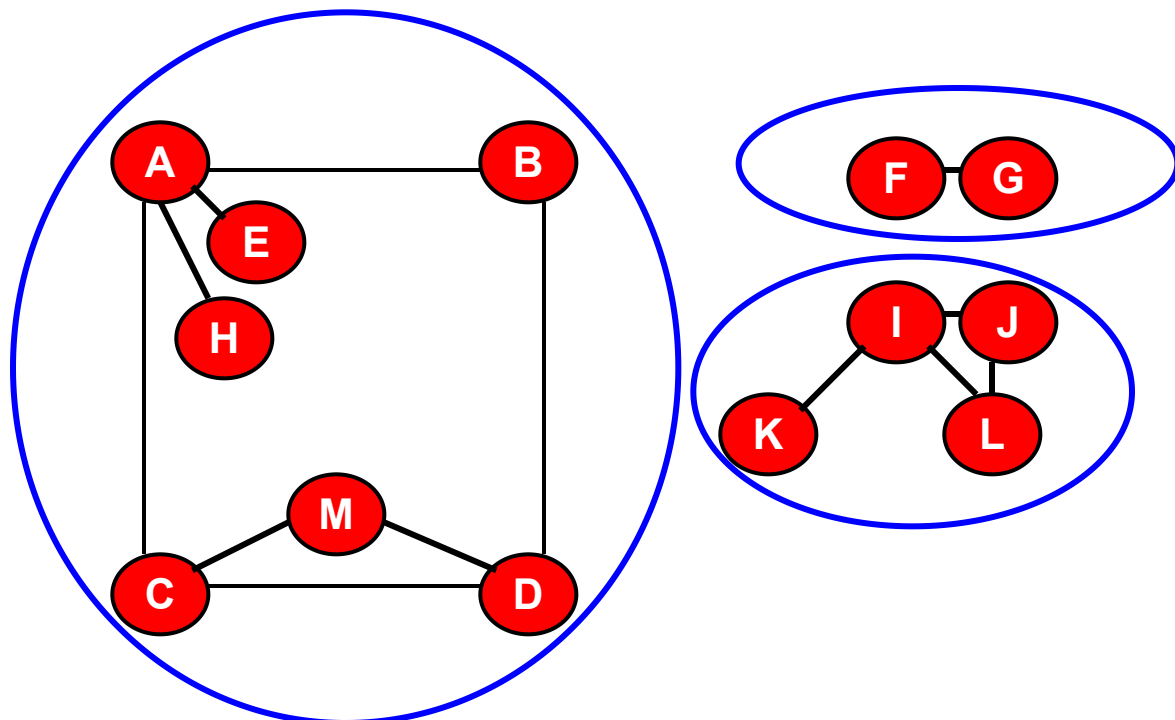
7.1 图的定义和术语——无向图的连通性

- **路径**：在无向图 $G=(V, \{E\})$ 中由顶点 v 至 v' 的顶点序列。
- **回路或环**：第一个顶点和最后一个顶点相同的路径。
- **简单回路或简单环**：除第一个顶点和最后一个顶点之外，其余顶点不重复出现的回路。
- **连通**：顶点 v 至 v' 之间有路径存在。
- **连通图**：无向图图 G 的任意两点之间都是连通的，则称 G 是连通图。
- **连通分量**：若无向图为非连通图，则图中各个极大连通子图称作此图的连通分量。

7.1 图的定义和术语——无向图的连通性



无向图G

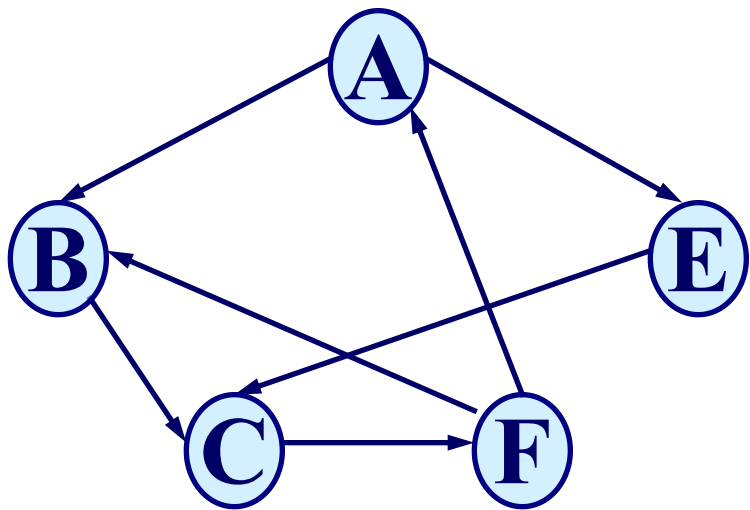


无向图G的三个连通分量

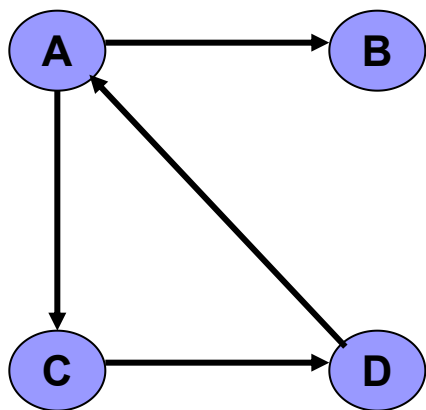
7.1 图的定义和术语——有向图的连通性

- **路径**：在有向图 $G=(V,\{E\})$ 中由顶点 v 经有向边至 v' 的顶点序列。路径上边的数目称作**路径长度**。
- **回路或环**：第一个顶点和最后一个顶点相同的路径。
- **简单回路或简单环**：除第一个顶点和最后一个顶点之外，其余顶点不重复出现的回路。
- **连通**：顶点 v 至 v' 之间有路径存在
- **强连通图**：有向图图 G 的任意两点之间都是连通的，则称 G 是强连通图。
- **强连通分量**：极大连通子图

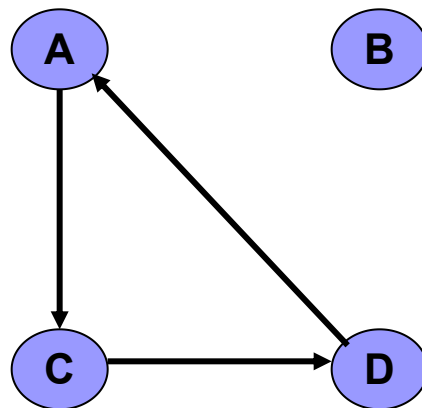
7.1 图的定义和术语——有向图的连通性



如: 长度为3的路径
 $\{A, B, C, F\}$



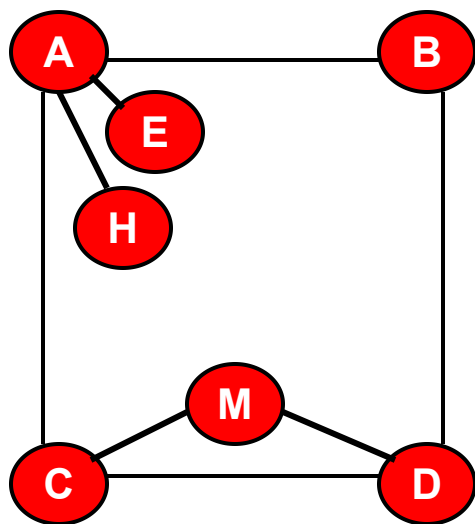
有向图**G**



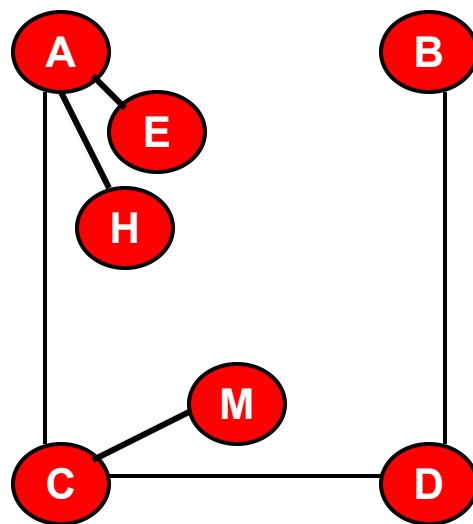
有向图**G**的两个强连通分量₁₀

7.1 图的定义和术语——生成树

■ **生成树**：极小连通子图，包含图的所有 n 个结点，但只含图的 $n-1$ 条边。在生成树中添加一条边之后，必定会形成回路或环。



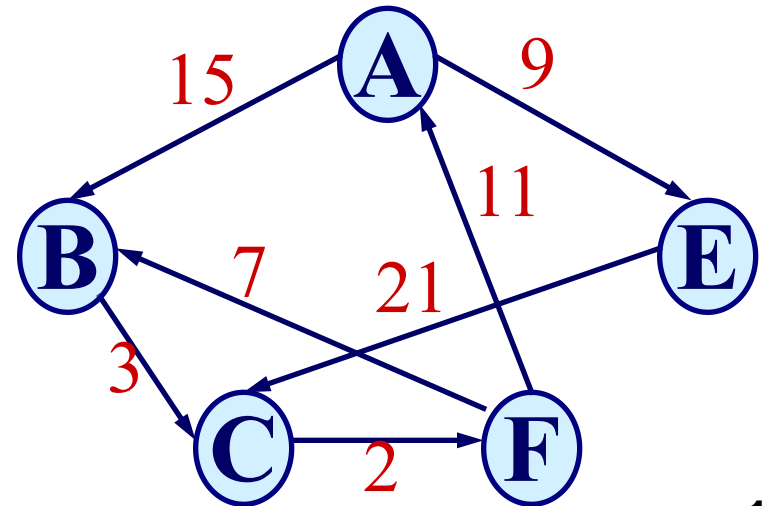
无向图G



无向图G的生成树

7.1 图的定义和术语——图的其它术语

- **完全图**：有 $n(n-1)/2$ 条边的无向图。其中 n 是结点个数。
- **有向完全图**：有 $n(n-1)$ 条边的有向图。
- 弧或边带权的图分别称作**有向网**或**无向网**。
- 若边或弧的个数 $e < n \log n$ ，则称作**稀疏图**，否则称作**稠密图**。



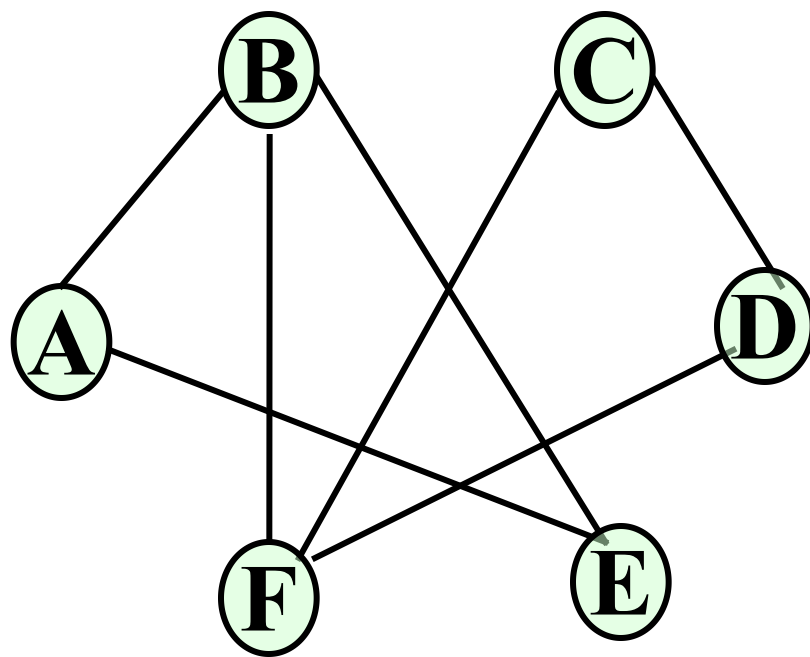
7.1 图的定义和术语——图的其它术语

- 假若顶点 v 和顶点 w 之间存在一条边，则称顶点 v 和 w 互为邻接点，边 (v, w) 和顶点 v 和 w 相关联。
- 和顶点 v 关联的边的数目定义为边的度。

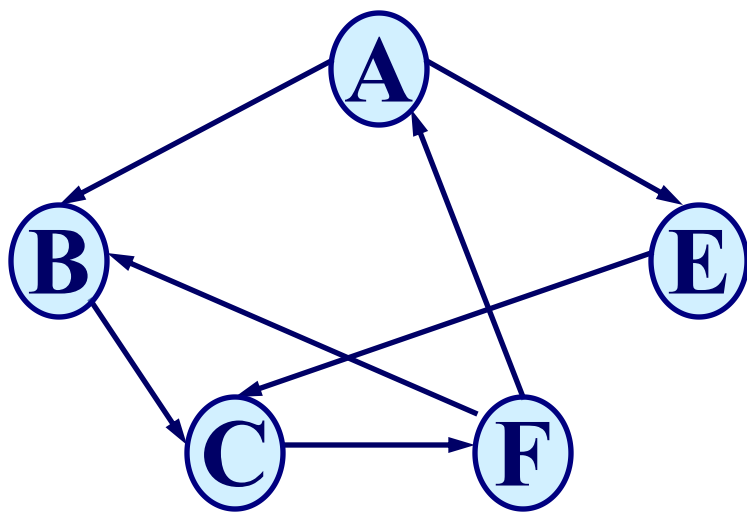
例如：

$$\text{ID}(\text{B}) = 3$$

$$\text{ID}(\text{A}) = 2$$



对有向图来说,



例如:

$$OD(B) = 1$$

$$ID(B) = 2$$

$$TD(B) = 3$$

- 顶点的**出度**: 以顶点 v 为弧尾的弧的数目;
- 顶点的**入度**: 以顶点 v 为弧头的弧的数目。

$$\text{顶点的度 (TD)} = \text{出度 (OD)} + \text{入度 (ID)}$$

7.1 图的定义和术语

- **有向树**：如果一个有向图恰有一个顶点的入度为0，其余顶点的入度均为1，则是一棵有向树。
- **有向图的生成森林**：由若干棵有向树组成，含有图中全部顶点，但只有足以构成若干棵不相交的有向树的弧。如图7.6。
- **图的抽象数据类型**见P156
- **注意**：关于“顶点的位置”和“邻接点的位置”只是一个相对的概念，但为了操作方便，我们一般会人为地为顶点排个序。这个顶点的顺序并不是图的本质特征。

基本操作

- 结构的建立和销毁
- 对顶点的访问操作
- 对邻接点的操作
- 插入或删除顶点
- 插入和删除弧
- 遍历

第七章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 图的连通性问题

7.5 有向无环图及其应用

7.6 最短路径

7.2 图的存储结构

图的四种常用的存储形式：

- 数组：邻接矩阵和加权邻接矩阵（**labeled adjacency matrix**）

- 邻接表

用于有向图，查询进入结点和离开结点的边容易

- 十字链表

- 邻接多重表

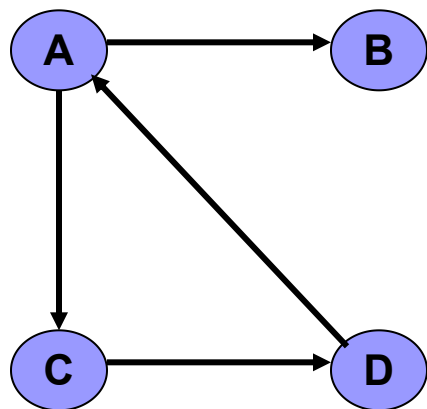
用于无向图，边表中的边结点只需存放一次，节约内存。

7.2.1 数组(邻接矩阵)存储表示

■ 无权值的有向图的邻接矩阵

■ 设有向图具有 n 个结点，则用 n 行 n 列的布尔矩阵 A 表示该有向图；并且 $A[i,j] = 1$ ，如果 i 至 j 有一条有向边； $A[i,j] = 0$ ，如果 i 至 j 没有一条有向边

■ 注意： $A[i,i] = 0$ 。出度： i 行之和。入度： j 列之和。



表示成右图矩阵

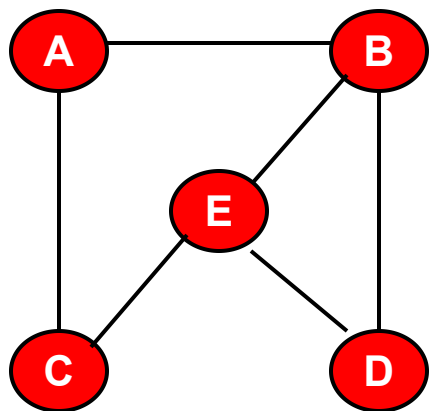
$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

7.2.1 数组(邻接矩阵)存储表示

■ 无权值的无向图的邻接矩阵

□ 设无向图具有 n 个结点，则用 n 行 n 列的布尔矩阵 A 表示该无向图；并且 $A[i,j] = 1$ ，如果 i 至 j 有一条无向边； $A[i,j] = 0$ ，如果 i 至 j 没有一条无向边。

□ 注意： $A[i,i] = 0$ 。 i 结点的度： i 行或 i 列之和。考虑无向图的邻接矩阵的对称性，则可采用压缩存储的方式只存入上三角矩阵或下三角矩阵。



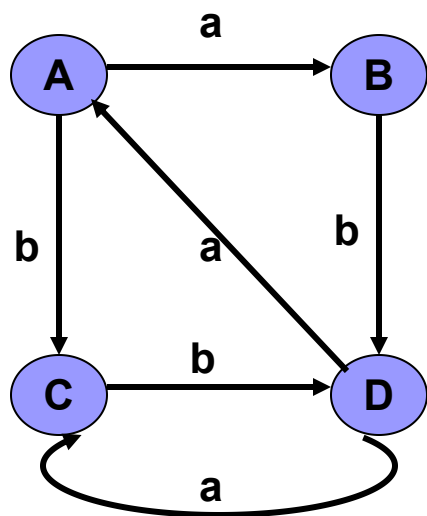
表示成右图矩阵

0	1	1	0	0
1	0	0	1	1
1	0	0	0	1
0	1	0	0	1
0	1	1	1	0

7.2.1 数组(邻接矩阵)存储表示

■ 有向图的加权邻接矩阵

- 设有向图具有 n 个结点，则用 n 行 n 列的矩阵 A 表示该有向图；并且 $A[i,j] = a$ ，如果 i 至 j 有一条有向边且它的权值为 a 。 $A[i,j] = \text{'空 或其它标志'}$ ，如果 i 至 j 没有一条有向边。
- 优点：判断任意两点之间是否有边方便，仅耗费 $O(1)$ 时间。
- 缺点：空间占用 $O(n^2)$ ，太多。



表示成右图矩阵

$$\begin{bmatrix} & a & b & \\ & & & b \\ a & & a & \\ & & & b \end{bmatrix}$$

7.2.1 数组(邻接矩阵)存储表示

- 图的邻接矩阵存储表示:

```
typedef enum{DG, DN, UDG, UDN} GraphKind;  
    // {有向图, 有向网, 无向图, 无向网}
```

```
typedef struct ArcCell{ // 弧的定义
```

```
    VRType adj; //对于无权图, 用1或0表示相邻否  
                //对于有权图, 则为权值
```

```
    InfoType *info; //该弧相关信息的指针
```

```
}ArcCell, AdjMatrix[MAX_VERTEX_NUM] [MAX_VERTEX_NUM]
```

7.2.1 数组(邻接矩阵)存储表示

■ 图的邻接矩阵存储表示

```
typedef struct{ //图的定义
```

```
    VertexType vexs[MAX_VERTEX_NUM]; //顶点向量
```

```
    AdjMatrix arcs; //邻接矩阵
```

```
    int vexnum, arcnum; // 图的当前顶点数和弧数
```

```
    GraphKind kind; // 图的种类标志
```

```
} MGraph;
```

- 算法7.2给出了无向网的邻接矩阵创建算法。构造一个具有 n 个顶点和 e 条边的无向网 G 的时间复杂度是 $O(n^2 + e \cdot n)$ ，其中对邻接矩阵 $G.arcs$ 的初始化耗费了 $O(n^2)$ 的时间。

7.2.2 邻接表

- 设有向图或无向图具有 n 个结点，则用**结点表**、**边表**表示该有向图或无向图。
- **结点表**：用数组或单链表的形式存放所有的结点。如结点数 n 已知，则采用数组形式，否则采用单链表的形式。
- **边表**（边结点表）：每条边用一个结点进行表示。同一个结点的所有的边形成它的边结点单链表。
- **优点**：内存 = 结点数 + 边数
- **缺点**：确定 $i \rightarrow j$ 是否有边，最坏需耗费 $O(n)$ 时间。无向图同一条边表示两次边表空间浪费一倍。有向图中寻找进入某结点的边，非常困难。

7.2.2 邻接表

结点表中的结点的表示:

- 用数组

data	firstarc
------	----------

data: 结点的数据域, 保存结点的数据值。

firstarc: 结点的指针域, 给出自该结点出发的第一条边的边结点的地址。

- 用单链表

data	firstarc	nextvex
------	----------	---------

data: 结点的数据域, 保存结点的数据值。

firstarc: 结点的指针域, 给出自该结点出发的第一条边的边结点的地址。

nextvex: 结点的指针域, 给出该结点的下一结点的地址。

7.2.2 邻接表

边结点表中的结点的表示:

info	adjvex	nextarc
------	--------	---------

info: 边结点的数据域, 保存边的权值等。

adjvex: 边结点的指针域, 给出本条边依附的另一结点 (非出发结点) 的地址。

nextarc: 结点的指针域, 给出自该结点出发的的下一条边的边结点的地址。

弧的结点结构

adjvex	nextarc	info
--------	---------	------

```
typedef struct ArcNode {  
    int      adjvex; // 该弧所指向的顶点的位置  
    struct ArcNode *nextarc;  
                // 指向下一条弧的指针  
    InfoType *info; // 该弧相关信息的指针  
} ArcNode;
```

顶点的结点结构

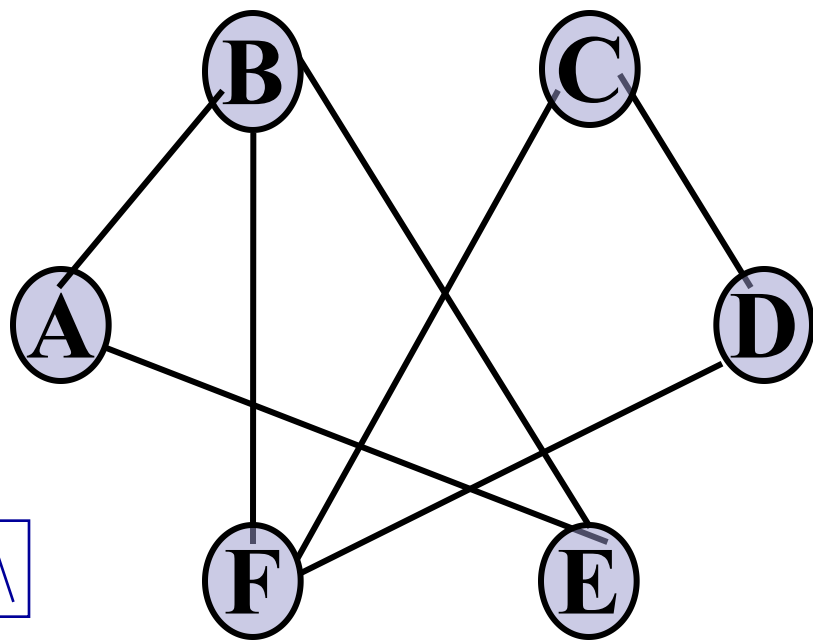
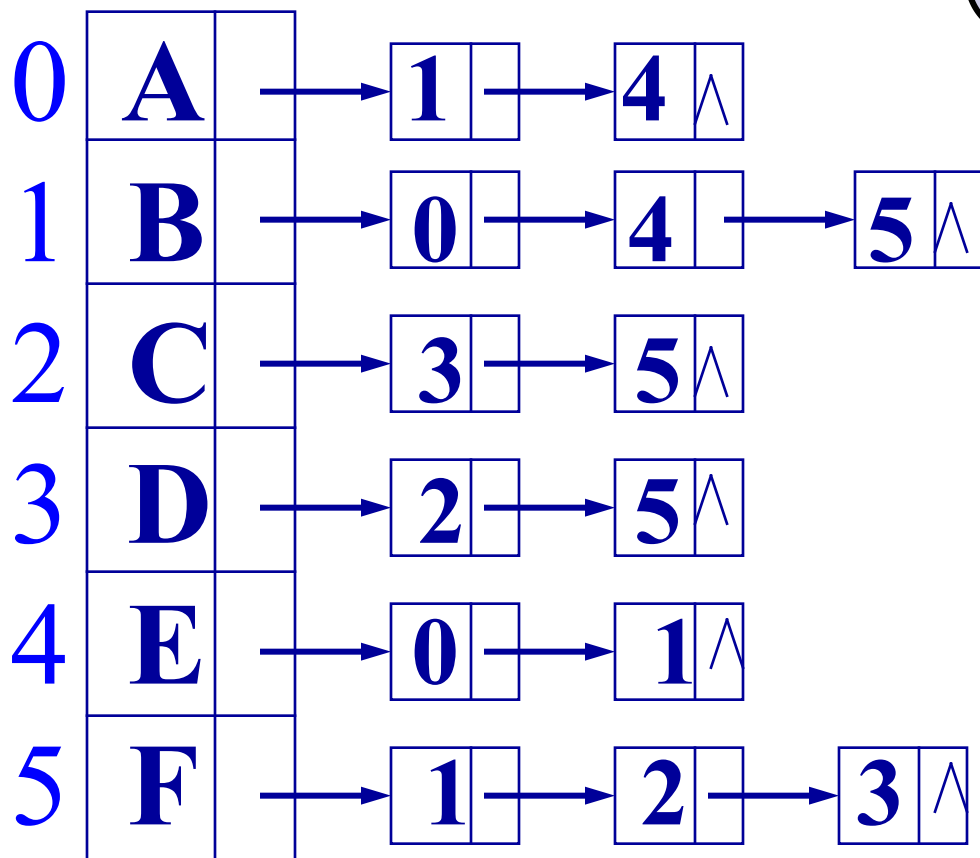
data	firstarc
------	----------

```
typedef struct VNode {  
    VertexType data; // 顶点信息  
    ArcNode *firstarc;  
    // 指向第一条依附该顶点的弧  
} VNode, AdjList[MAX_VERTEX_NUM];
```

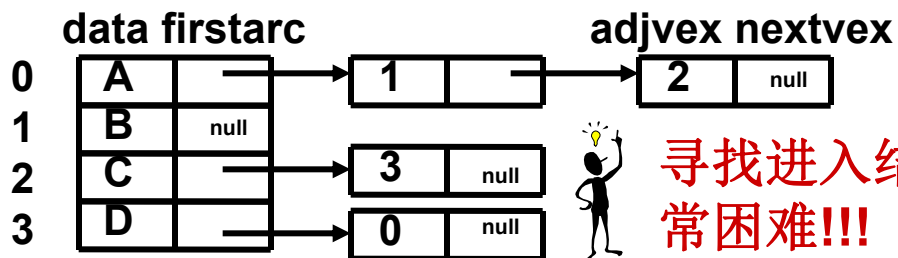
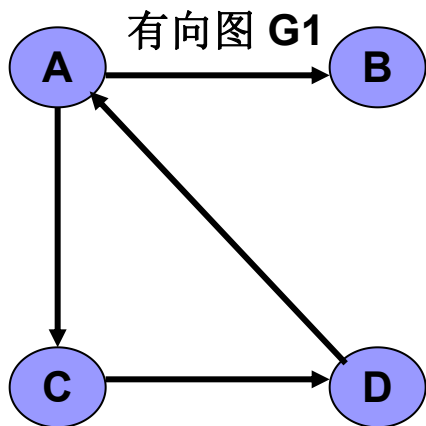
图的结构定义

```
typedef struct {  
    AdjList vertices;  
  
    int    vexnum, arcnum;  
  
    int    kind;           // 图的种类标志  
  
} ALGraph;
```

7.2.2 邻接表



7.2.2 邻接表

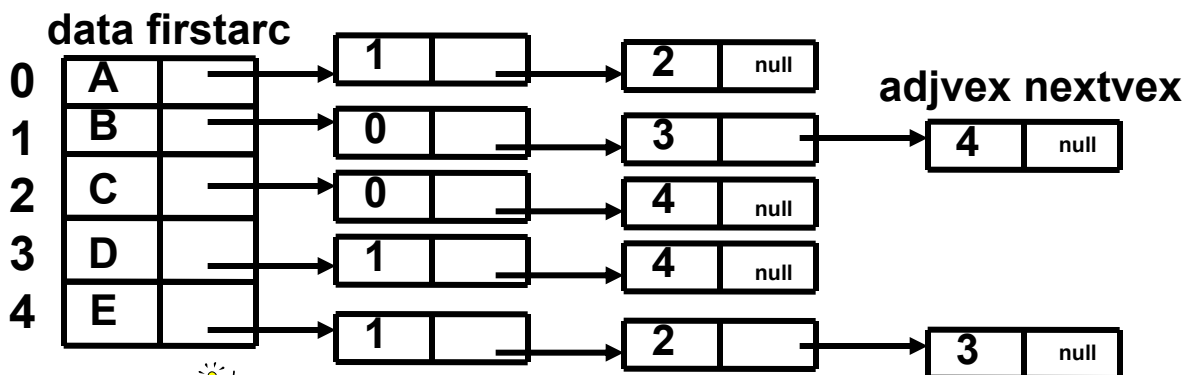
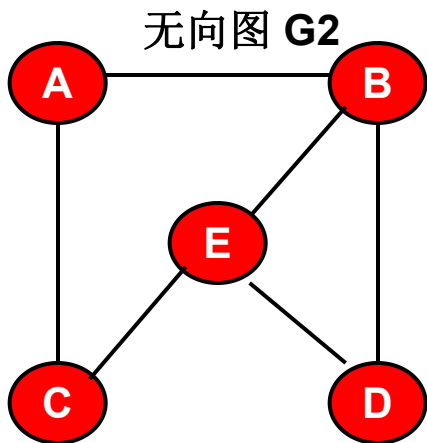


寻找进入结点的边非常困难!!!

改进：建立逆邻接表或十字链表



adjvex 指针域之值为相应结点的数组元素的下标!!!

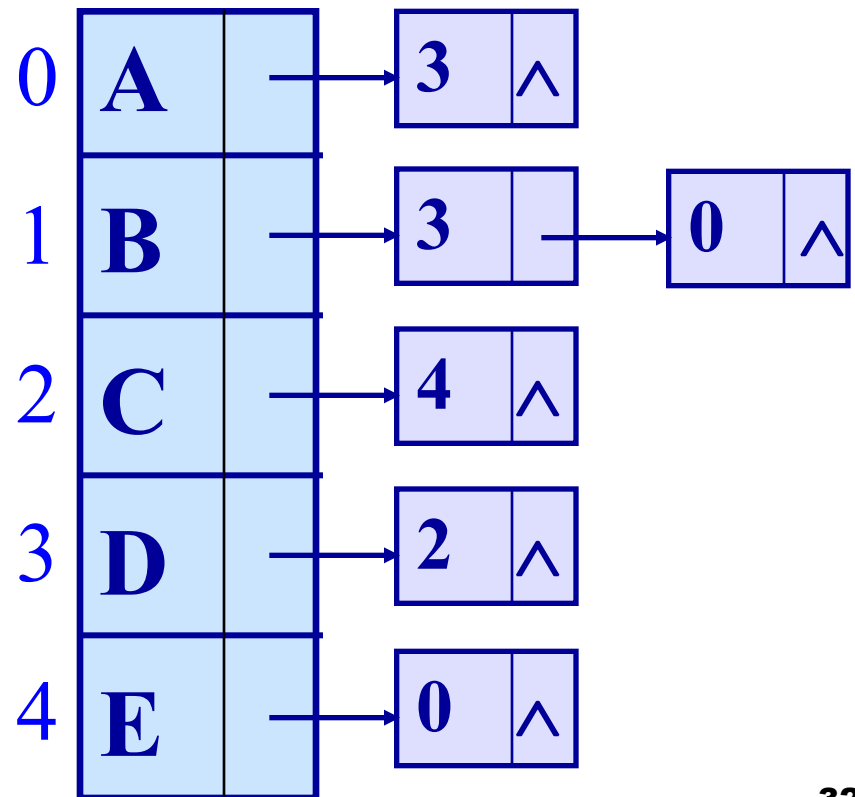
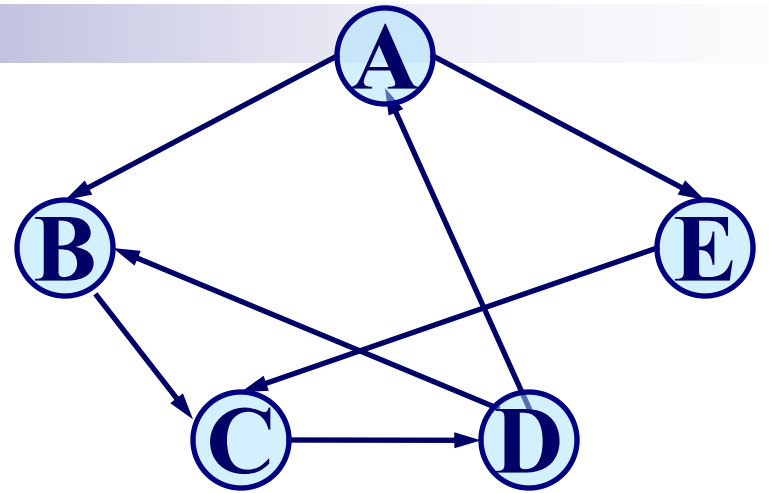


六条边却用了 12 个边结点!!!

改进：建立邻接多重表

有向图的逆邻接表:

在有向图的邻接表中，对每个顶点，链接的是指向该顶点的弧。



7.2.3 十字链表

- 结点表中的结点的表示:

data	firstin	firstout
------	---------	----------

data: 结点的数据域, 保存结点的数据值。

firstin: 结点的指针域, 给出进入该结点的第一条边的边结点的地址。

firstout: 结点的指针域, 给出自该结点出发的第一条边的边结点的地址。

- 边结点表中的结点的表示:

info	tailvex	headvex	hlink	tlink
------	---------	---------	-------	-------

info: 边结点的数据域, 保存边的权值等。

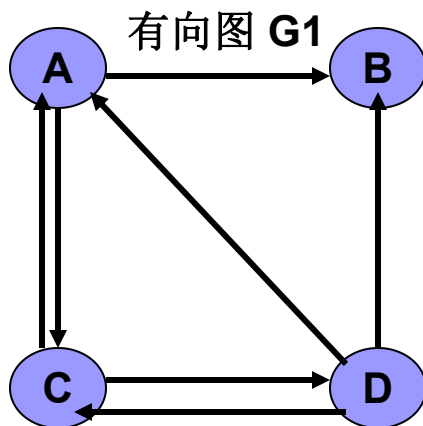
tailvex: 本条边的出发结点的地址。

headvex: 本条边的终止结点的地址。

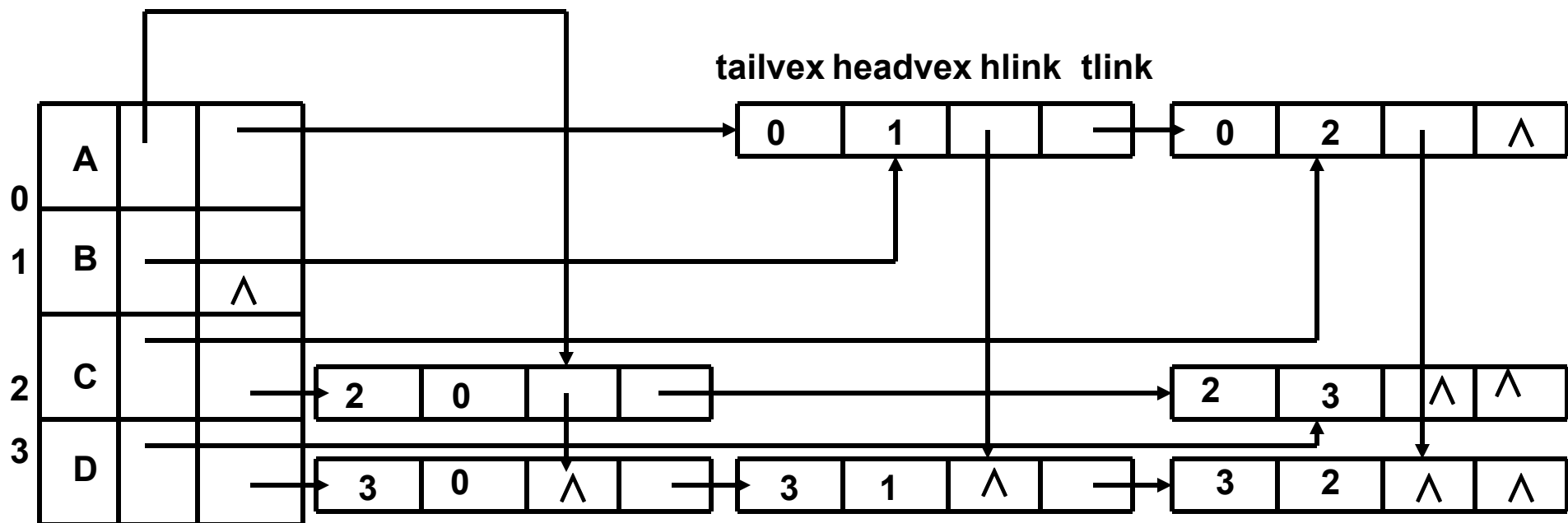
hlink: 出发结点相同的边中的下一条边的地址。

tlink: 终止结点相同的边中的下一条边的地址。

7.2.3 十字链表



- 用途：用于有向图，查询进入结点和离开结点的边容易。



data firstin firstout

有向图的十字链表存储表示:

弧的结点结构

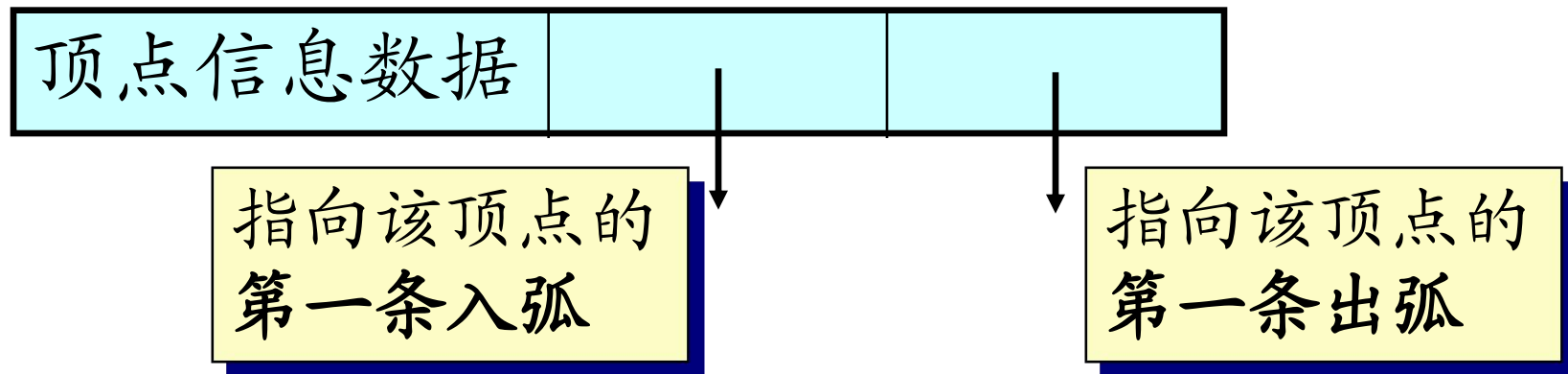
弧尾顶点位置	弧头顶点位置			弧的相关信息
--------	--------	--	--	--------

指向下一个
有相同弧尾
的结点

指向下一个
有相同弧头
的结点

```
typedef struct ArcBox { // 弧的结构表示
    int tailvex, headvex; InfoType *info;
    struct ArcBox *hlink, *tlink;
} VexNode;
```

顶点的结点结构



```
typedef struct VexNode { // 顶点的结构表示  
    VertexType data;  
    ArcBox *firstin, *firstout;  
} VexNode;
```

有向图的结构表示(十字链表)

```
typedef struct {  
    VexNode xlist[MAX_VERTEX_NUM];  
        // 顶点结点(表头向量)  
    int vexnum, arcnum;  
        // 有向图的当前顶点数和弧数  
} OLGraph;
```

7.2.4 邻接多重表

- 结点表中的结点的表示:

data	firstedge
------	-----------

data: 结点的数据域, 保存结点的数据值。

firstedge: 结点的指针域, 给出自该结点出发的第一条边的边结点的地址。

- 边结点表中的结点的表示:

mark	ivex	ilink	jvex	jlink	info
------	------	-------	------	-------	------

info: 边结点的数据域, 保存边的权值等。

mark: 边结点的标志域, 用于标识该条边是否被访问过。

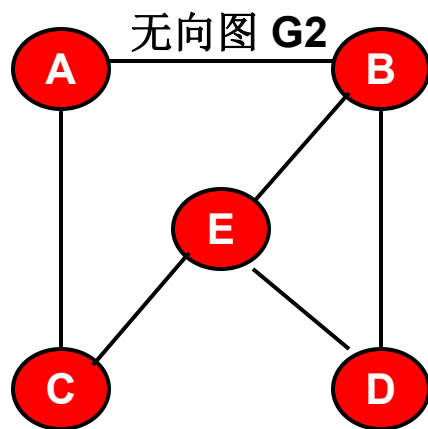
ivex: 本条边依附的一个结点的地址。

ilink: 依附于结点**ivex**的下一条边的地址。

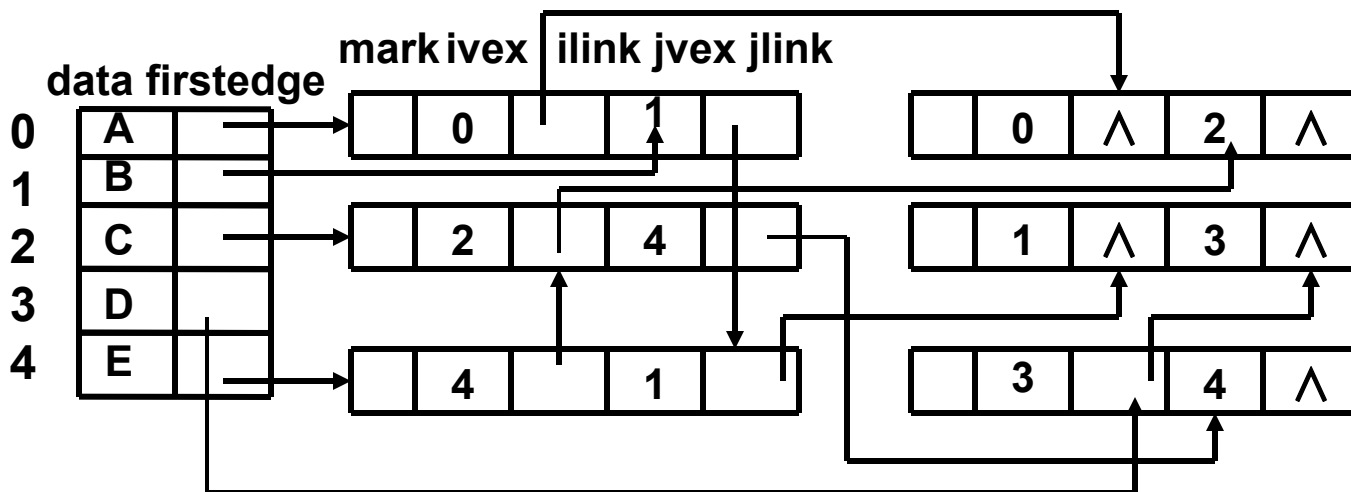
jvex: 本条边依附的另一个结点的地址。

jlink: 依附于结点**jvex**下一条边的地址

7.2.4 邻接多重表



- **用途**：用于**无向图**，边表中的边结点只需存放一次，节约内存。



无向图的邻接多重表存储表示

边的结构表示

```
typedef struct Ebox {  
    VisitIf    mark;    // 访问标记  
  
    int        ivex, jvex;  
                //该边依附的两个顶点的位置  
  
    struct EBox *ilink, *jlink;  
  
    InfoType    *info;    // 该边信息指针  
} EBox;
```


顶点的结构表示

```
typedef struct VexBox {  
    VertexType data;  
    EBox *firstedge; // 指向第一条依附该顶点的边  
} VexBox;
```

无向图的结构表示

```
typedef struct { // 邻接多重表  
    VexBox adjmulist[MAX_VERTEX_NUM];  
    int vexnum, edgenum;  
} AMLGraph;
```

第七章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 图的连通性问题

7.5 有向无环图及其应用

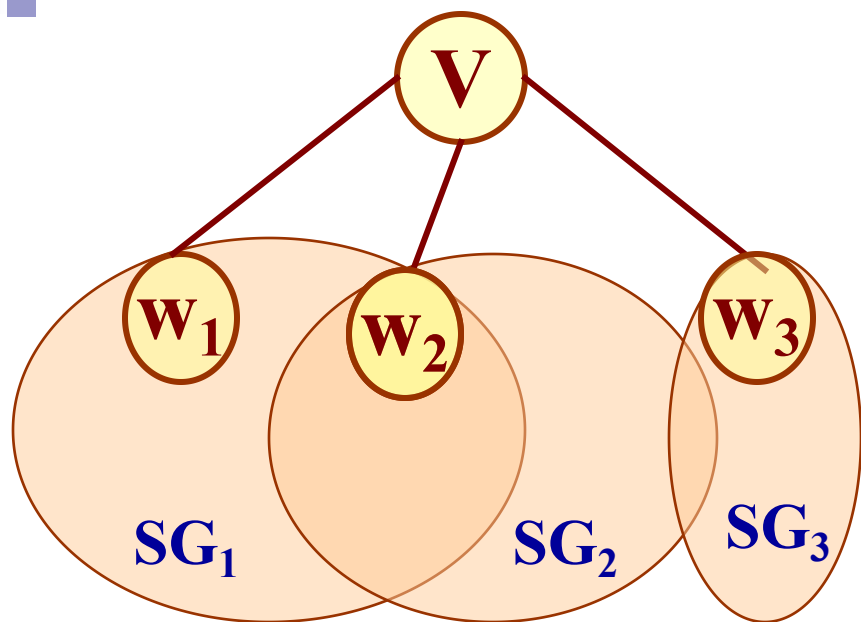
7.6 最短路径

7.3 图的遍历

- 从图中某个顶点出发游历图，访遍图中其余顶点，并且使图中的每个顶点仅被访问一次的过程。
- 图的常见的遍历形式：
 - 深度优先搜索
 - 广度优先搜索

7.3.1 深度优先搜索遍历图

- 从图中某个顶点 V_0 出发，访问此顶点，然后依次从 V_0 的各个未被访问的邻接点出发深度优先搜索遍历图，直至图中所有和 V_0 有路径相通的顶点都被访问到。若此时图中还有顶点未被访问，则另选图中一个未曾被访问的顶点作起始点，重复上述过程，直至图中所有顶点都被访问到为止。
- 结点的邻接结点的次序是任意的，因此深度优先搜索的序列可能有多种。



W_1 、 W_2 和 W_3 均为 V 的邻接点， SG_1 、 SG_2 和 SG_3 分别为含顶点 W_1 、 W_2 和 W_3 的子图。

访问顶点 V ：

for (W_1 、 W_2 、 W_3)

若该邻接点 W 未被访问，

则从它出发进行深度优先搜索遍历。

7.3.1 深度优先搜索遍历图

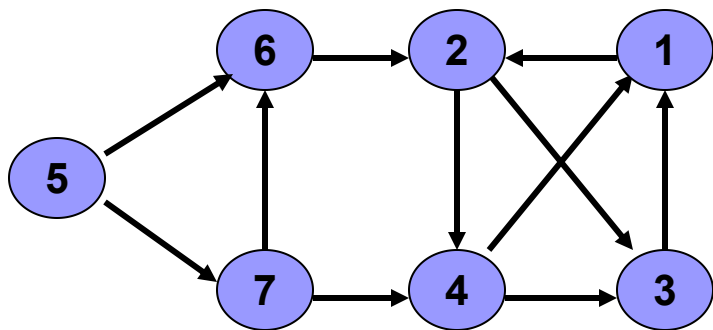
- 每个结点只能被访问一次，又因为一个结点可以和其它的任何结点相邻接，如何判别V的邻接点是否被访问？
- 解决的办法是：为每个顶点设立一个“访问标志 `visited[w]`”。

7.3.1 深度优先搜索遍历图

- 深度优先搜索类似于树的**先根遍历**，访问方式：
 - 1、选中第一个被访问的结点。
 - 2、对结点作已访问过的标志。
 - 3、依次从结点的未被访问过的第一个、第二个、第三个……邻接结点出发，进行深度优先搜索。转向2。
 - 4、如果还有顶点未被访问，则选中一个起始结点，转向2。
 - 5、所有的结点都被访问到，则结束。

7.3.1 深度优先搜索遍历图

例子：为了说明问题，邻接结点的访问次序以序号为准。序号小的先访问。
如：结点 5 的邻接结点有两个 6、7，则先访问结点 6，再访问结点 7。



从结点1出发的搜索序列：

1、2、3、4没有搜索

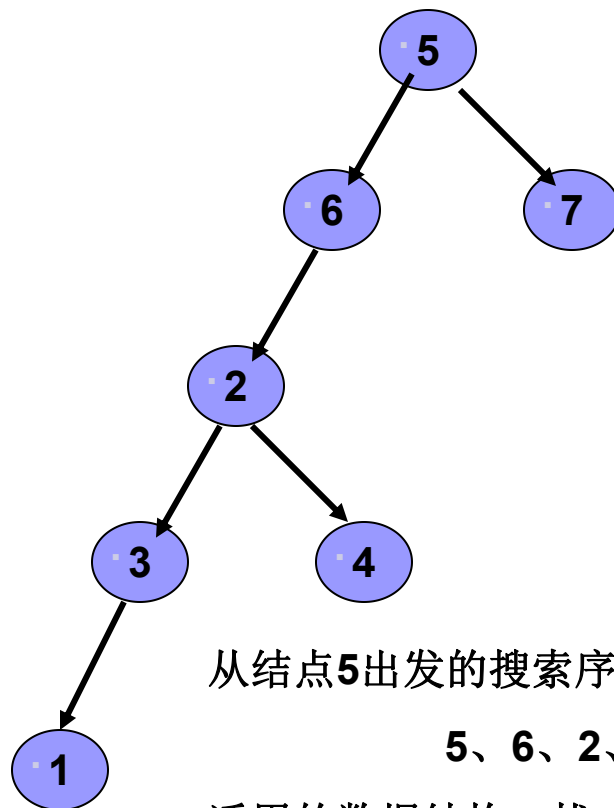
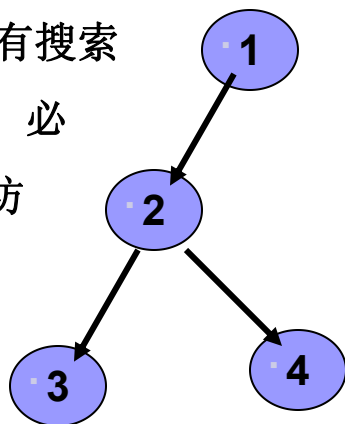
到所有的结点，必

须另选图中未访

问过的结点，

继续进行

搜索。



从结点5出发的搜索序列：

5、6、2、3、1、4、7

适用的数据结构：栈

深度优先搜索遍历图的实现:

使用的变量说明: **Boolean visited[MAX] ;** //用于标识结点是否已被访问过

Status (* VisitFunc) (int v); //函数变量

```
void DFS(Graph G, int v) {
```

```
    // 从顶点v出发, 深度优先搜索遍历连通图 G
```

```
    visited[v] = TRUE; VisitFunc(v);
```

```
    for(w=FirstAdjVex(G, v);
```

```
        w!=0; w=NextAdjVex(G,v,w))
```

```
        if (!visited[w]) DFS(G, w);
```

```
        // 对v的尚未访问的邻接顶点w递归调用DFS
```

```
} // DFS
```

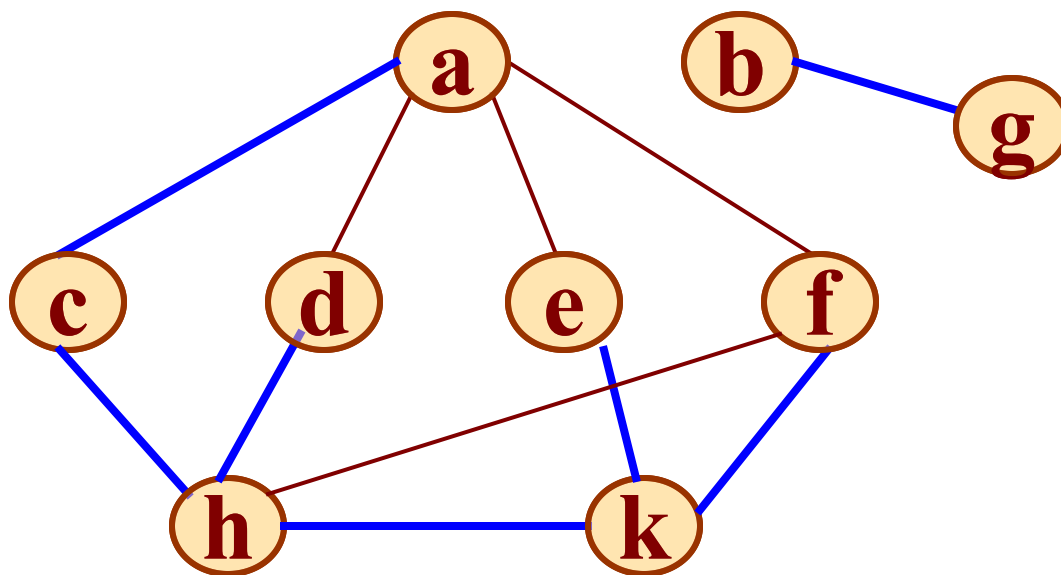
深度优先搜索遍历图的实现（续）：

```
void DFSTraverse(Graph G, Status (*Visit)(int v)) {  
    // 对图 G 作深度优先遍历。  
    VisitFunc = Visit;  
    for (v=0; v<G.vexnum; ++v)  
        visited[v] = FALSE; // 访问标志数组初始化  
    for (v=0; v<G.vexnum; ++v)  
        if (!visited[v]) DFS(G, v);  
        // 对尚未访问的顶点调用DFS  
}
```

时间复杂性（**n**：结点数；**e**：边数）：

邻接矩阵： **$O(n^2)$** 邻接表： **$O(n + e)$**

例如:



0 1 2 3 4 5 6 7 8

访问标志:

T	T	T	T	T	T	T	T	T
---	---	---	---	---	---	---	---	---

访问次序:

a	c	h	d	k	f	e	b	g
---	---	---	---	---	---	---	---	---

7.3.2 广度优先搜索遍历图

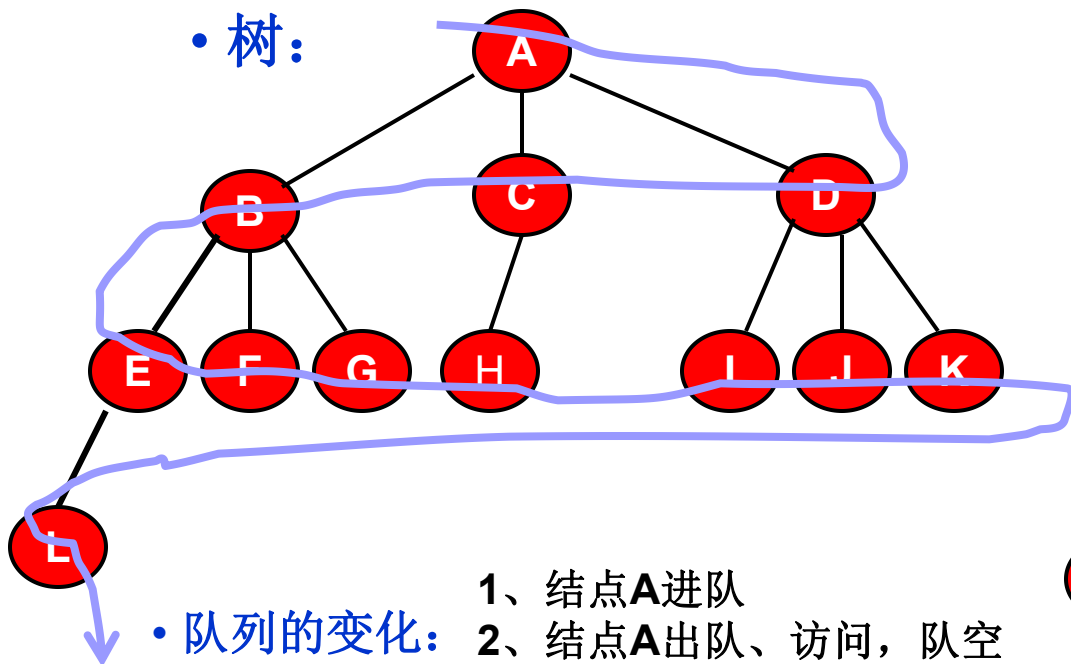
- 从图中的某个顶点 V_0 出发，并在访问此顶点之后依次访问 V_0 的所有未被访问过的邻接点，之后按这些顶点被访问的先后次序依次访问它们的邻接点，直至图中所有和 V_0 有路径相通的顶点都被访问到。
- 若此时图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作起始点，重复上述过程，直至图中所有顶点都被访问到为止。

7.3.2 广度优先搜索遍历图

- 结点的邻接结点的次序是任意的，因此广度优先搜索的序列可能有多种。
- 广度优先搜索类似于树的从根出发的按层次遍历。
- 访问方式：
 1. 选中第一个被访问的结点 V
 2. 对结点 V 作已访问过的标志。
 3. 依次访问结点 V 的未被访问过的第一个、第二个、第三个……第 M 个邻接结点 W_1 、 W_2 、 W_3 …… W_m ，且进行标记。
 4. 依次访问结点 W_1 、 W_2 、 W_3 …… W_m 的邻接结点，且进行标记。
 5. 如果还有结点未被访问，则选中一个起始结点，也标记为 V ，转向 2。
 6. 所有的结点都被访问到，则结束。

7.3.2 广度优先搜索遍历图

• 树:



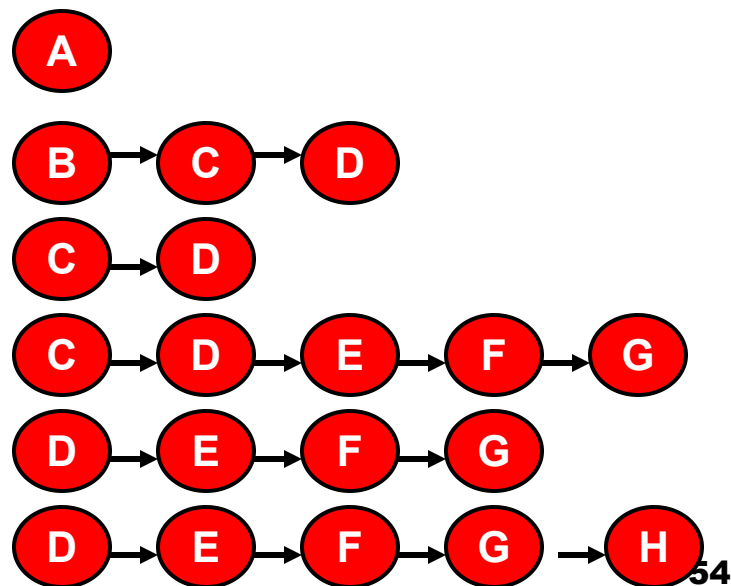
• 队列的变化:

- 1、结点A进队
- 2、结点A出队、访问，队空
- 3、结点A的儿子结点进队
- 4、结点B出队、访问
- 5、结点B的儿子结点进队
- 6、结点C出队、访问
- 7、结点C的儿子结点进队

树的按层次进行访问的次序:

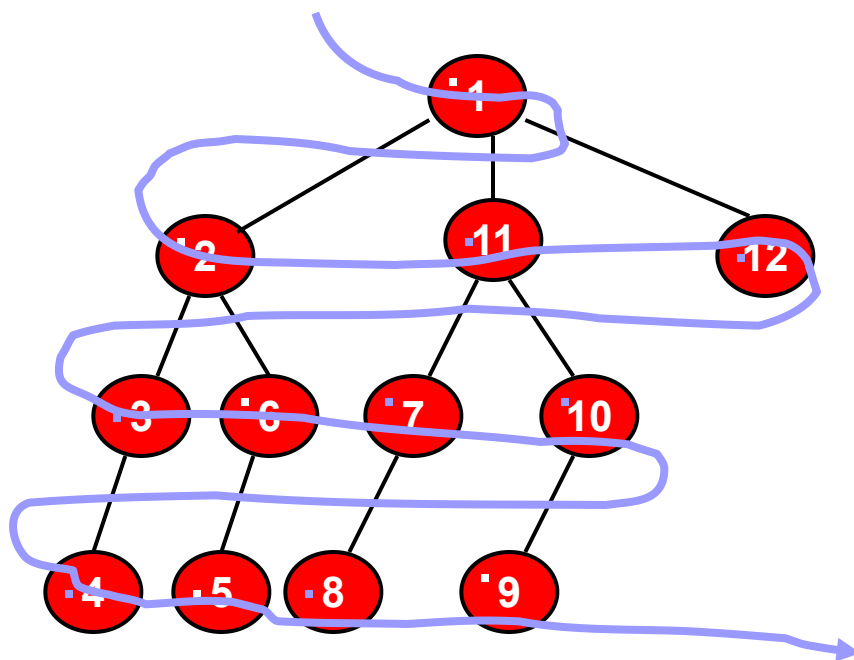
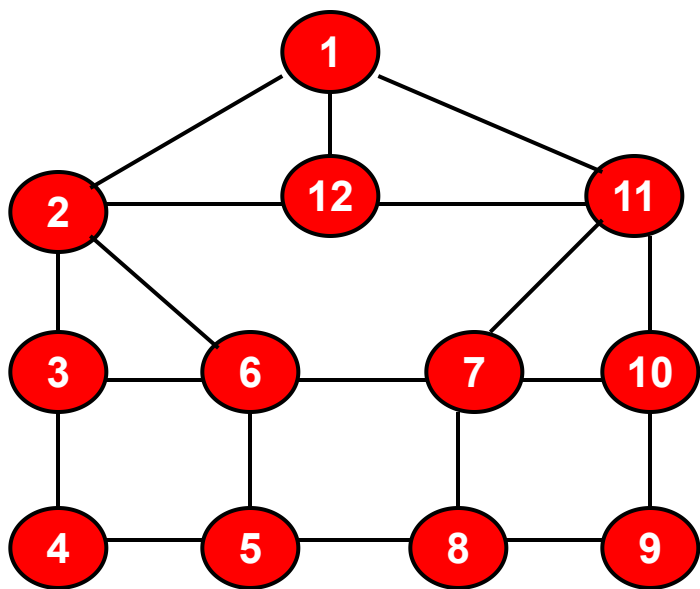
A、B、C、D、E、F、G、H、
I、J、K、L

适用的数据结构: 队列



7.3.2 广度优先搜索遍历图

- 例子：为了说明问题，邻接结点的访问次序以序号为准。序号小的先访问。
如：结点 1 的邻接结点有三个 2、12、11，则先访问结点 2、11，再访问12。



图的广度优先的访问次序：

1、2、11、12、3、6、7、10、4、5、8、9

适用的数据结构：队列

7.3.2 广度优先搜索遍历图

使用的变量说明: Boolean visited[MAX]; //用于标识结点是否已被访问过

Status (* VisitFunc) (int v); //函数变量

```
void BFSTraverse( Graph G, Status ( * Visit) (int v)); {
```

```
    for ( v=0; v <G.vexnum; ++v ) visited[v] = FALSE; //初始化访问标志
```

```
    InitQueue(Q); // 置空的辅助队列Q
```

```
    for ( v=0; v <G.vexnum; ++v )
```

```
        if ( ! Visited[ v ] ) // v 尚未访问,访问v, v入队列
```

```
        { Visited[ v ] = TRUE; Visit(v); EnQueue(Q,v);
```

```
            while (!EmptyQueue(Q))
```

```
            { DeQueue(Q,u); // 队头元素出队并置为u
```

```
                for ( w = FirstAdjVex(G, u) ; w ; w = NextAdjVex(G, u, w) )
```

```
                if ( ! Visited[ w ] ) Visited[ v ] = TRUE;
```

```
                Visit(v); EnQueue(Q, w); // 访问的顶点w入队列
```

```
            }
```

```
    } }
```

时间复杂性:

•邻接矩阵:

$O(n^2)$

•邻接表

$O(n + e)$

n: 结点数

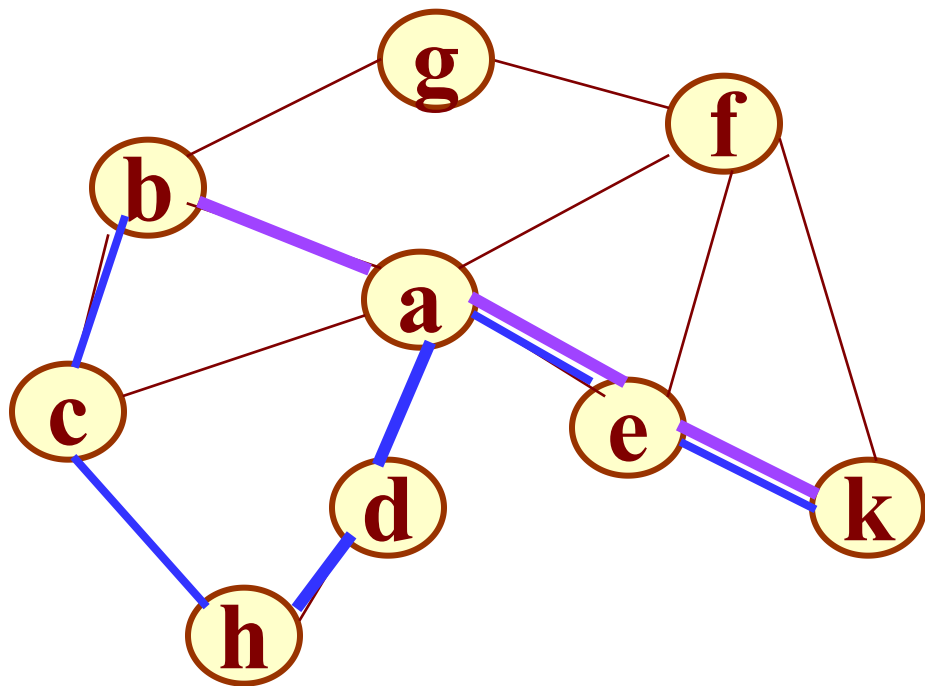
e: 边数

7.3.3 遍历应用实例

问题： 求两个顶点之间的一条路径长度最短的路径

若两个顶点之间存在多条路径，则其中必有一条路径长度最短的路径。如何求得这条路径？

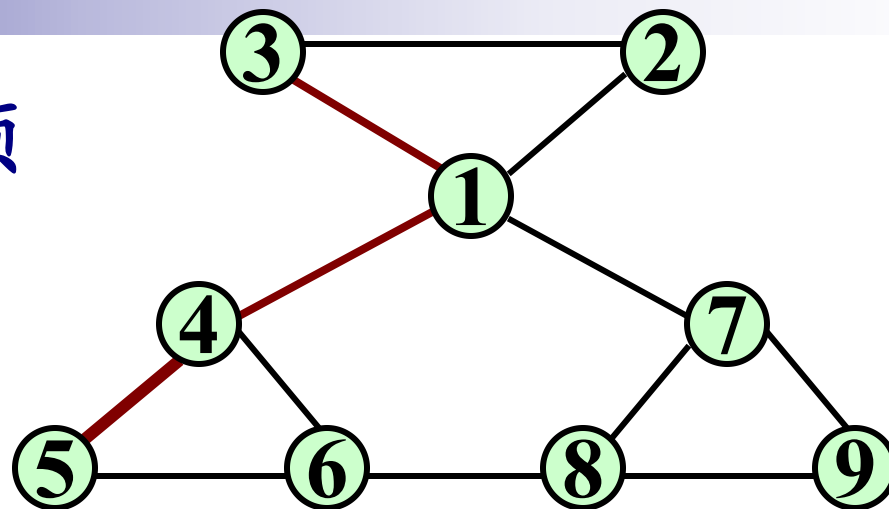
7.3.3 遍历应用实例



深度优先搜索访问顶点的次序取决于图的存储结构，而广度优先搜索访问顶点的次序是按“路径长度”渐增的次序。

因此，求路径长度最短的路径可以基于广度优先搜索遍历进行，但需要修改链队列的结点结构及其入队列和出队列的算法。

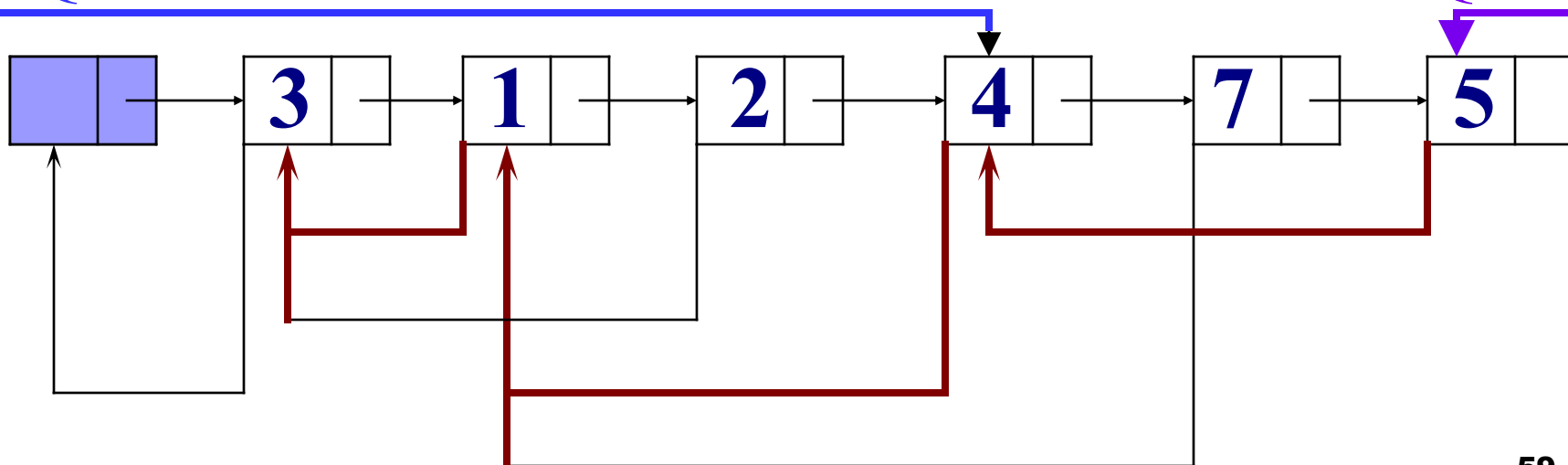
例如:求图中顶点 3 至顶点 5 的一条最短路径。



链队列的状态
如下所示:

Q.front

Q.rear



7.3.3 遍历应用实例

- 1) 将链队列的结点改为“双链”结点。即结点中包含next和prior两个指针;
- 2) 修改入队列的操作。插入新的队尾结点时, 令其prior域的指针指向刚刚出队列的结点, 即当前的队头指针所指结点;
- 3) 修改出队列的操作。出队列时, 仅移动队头指针, 而不将队头结点从链表中删除。

```
typedef DuLinkList QueuePtr;
void InitQueue(LinkQueue& Q) {
    Q.front=Q.rear=(QueuePtr)malloc(sizeof(QNode));
    Q.front->next = Q.rear->next = NULL
}
```

```
void EnQueue( LinkQueue& Q, QelemType e ) {
    p = (QueuePtr) malloc (sizeof(QNode));
    p->data = e; p->next = NULL;
    p->priou = Q.front
    Q.rear->next = p; Q.rear = p;
}
```

```
void DeQueue( LinkQueue& Q, QelemType& e ) {
    Q.front = Q.front->next; e = Q.front->data
}
```

第七章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 图的连通性问题

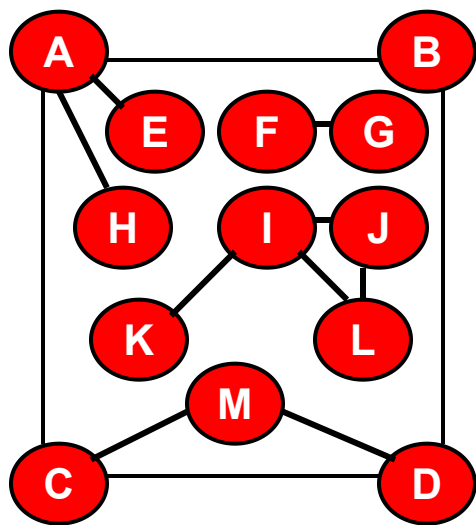
7.5 有向无环图及其应用

7.6 最短路径

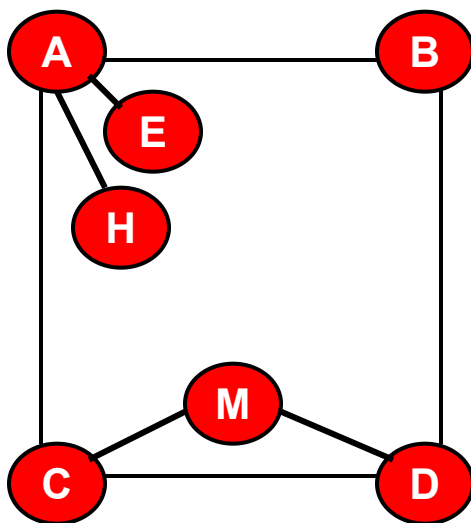
7.4 图的连通性问题

7.4.1 无向图的连通分量和生成树

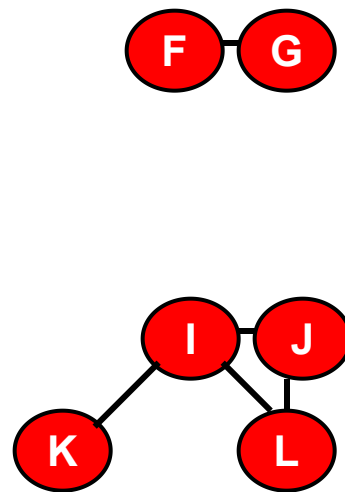
- **连通**：顶点 v 至 v' 之间有路径存在
- **连通图**：无向图 G 的任意两点之间都是连通的，则称 G 是连通图。
- **连通分量**：极大连通子图



无向图 G

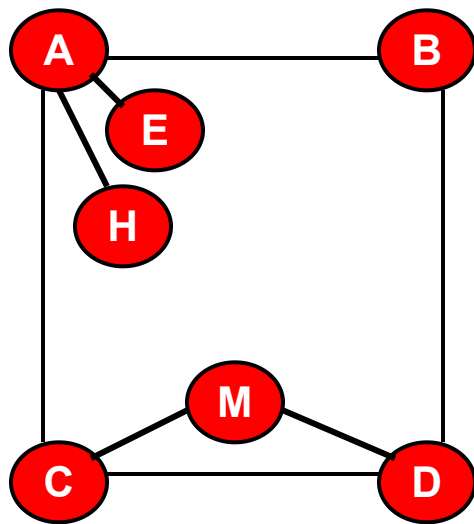


无向图 G 的三个连通分量

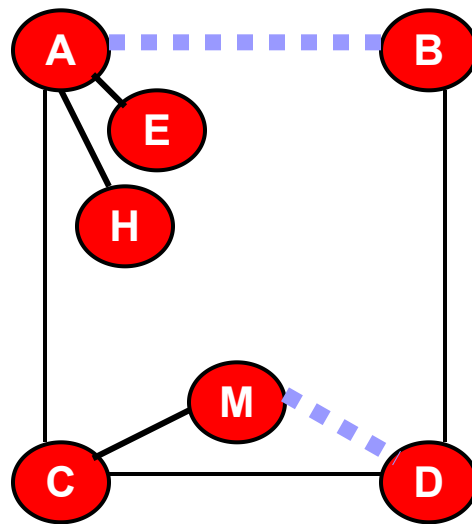


7.4.1 无向图的连通分量和生成树

- **生成树**：极小连通子图。包含图的所有 n 个结点，但只含图的 $n-1$ 条边
 - 在生成树中添加一条边之后，必定会形成**回路或环**。因为在生成树的任意两点之间，本来就是连通的，添加一条边之后，形成了这两点之间的第二条通路。
 - **生成方法**：进行深度为主的遍历或广度为主的遍历，得到深度优先生成树或广度优先生成树。



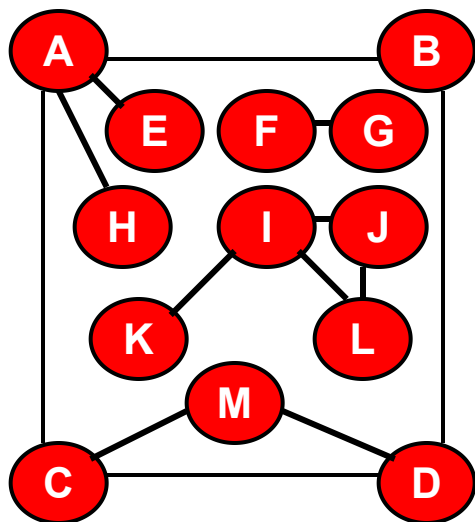
无向图G



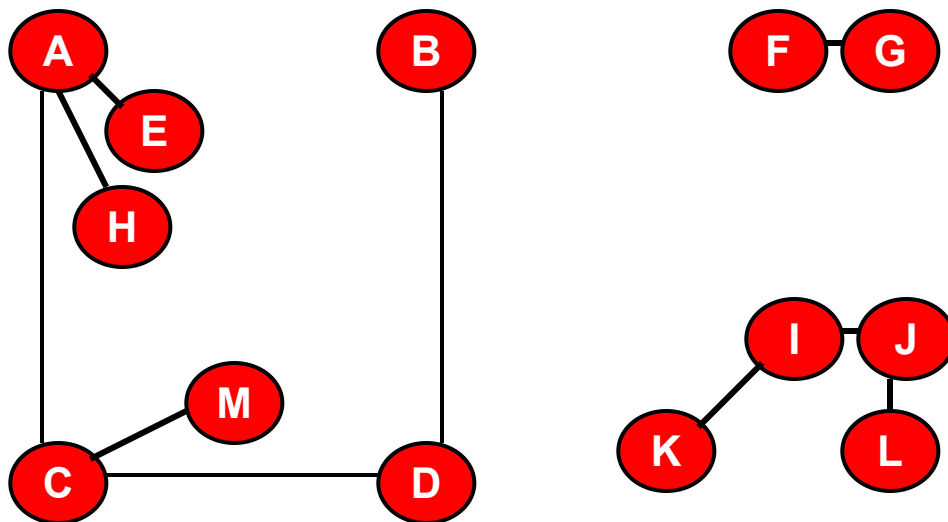
无向图G的生成树

7.4.1 无向图的连通分量和生成树

- **生成森林**：在进行深度为主的遍历或广度为主的遍历时，对于非连通图，将得到多棵深度优先生成树或广度优先生成树，称之为生成森林。
- 以**孩子兄弟链表**作生成森林的存储结构，则算法7.7生成非连通图的深度优先生成森林。算法的时间复杂度与遍历相同。



无向图G



无向图G的生成森林

7.4.1 无向图的连通分量和生成树

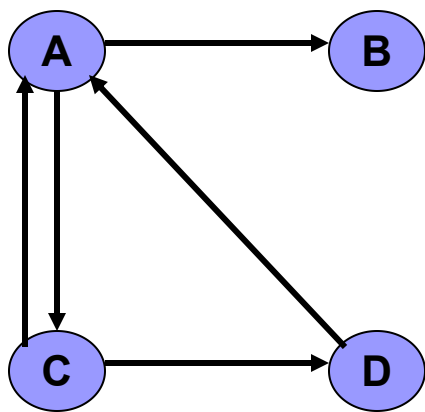
```
void DFSForest(Graph G, CSTree &T){  
    //建立无向图G的深度优先森林的孩子兄弟链表T  
    T=NULL;  
    for (v=0; v<G.vexnum; ++v)  
        visited[v]=FALSE;  
    for (v=0; v<G.vexnum; ++v)  
        if (!visited[v]) { //每一次生成一棵生成树  
            p=(CSTree) malloc(sizeof(CSNode));  
                                //按兄弟链表方式存储  
            *p= { GetVex(G,v), NULL, NULL};  
            if (!T) T=p;      //第一棵生成树的根是T  
            else q→nextsibling=p;  
                                //后面生成树的根是上一个哥哥的右儿子  
            q=p;  
            DFSTree(G, v, p);  
        }  
}
```

7.4.1 无向图的连通分量和生成树

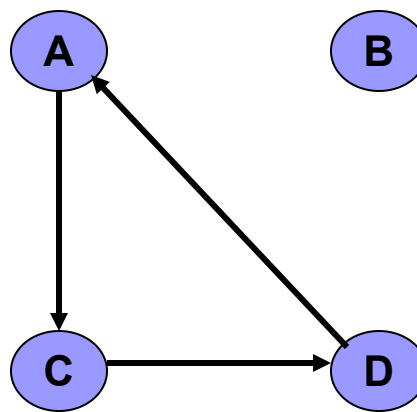
```
void DFSTree(Graph G, int v, CSTree &T){  
    //从第v个顶点出发深度优先遍历图G，建立以T为根的生成树。  
    visited[v]=TRUE; first=TRUE;  
    for (w=FirstAdjVex(G, v); w>=0; w=NextAdjVex(G, v, w))  
        if (!visited[w]){  
            p=(CSTree) malloc(sizeof(CSNode)); //生成结点  
            *p={GetVex(G, w), NULL, NULL};  
            if (first){ //如果是第一个孩子，则当根的左孩子  
                T→lchild=p; first=FALSE;  
            }  
            else{ //如果是后面的孩子，则是上一个哥哥的右孩子  
                q→nextsibling=p;  
            }  
            q=p;  
            DFSTree(G, w, q);  
        }  
    }  
}
```

7.4.2 有向图的强连通分量的求法

- **强连通图**：有向图图G的任意两点之间都是连通的，则称G是强连通图。
- **强连通分量**：极大连通子图



有向图G



有向图G的两个强连通分量

7.4.2 有向图的强连通分量的求法

■ 强连通分量的求法:

- 1、从有向图**G**的某个顶点出发进行**深度优先搜索遍历**，按照所有邻接点的搜索都完成的顺序（即退出**DFS**函数的顺序）给结点进行编号。最先退出**DFS**函数的结点的编号为**1**，其它结点的编号按次序逐次增大**1**。
- 2、将有向图**G**的所有的有向边进行**倒置**，构造新的有向图记为**G_r**。
- 3、根据步骤**1**对结点进行的编号，选取**最大编号的结点**。从该结点出发，在有向图**G_r**上进行**深度优先搜索遍历**。如果没有访问到所有的结点，则从剩余的那些结点中选取编号最大的结点再进行深度为主的遍历。直至所有的结点都被访问到为止。
- 4、在最后得到的生成森林中的每一棵生成树，都是有向图**G**的强连通分量顶点集。

利用遍历求强连通分量的
时间复杂度和遍历相同。

7.4.2 有向图的强连通分量的求法

在主程序中置： **count = 0**

DFS (VextexType v) ;

{ VextexType w;

mark [v] = visited ;

for (v 的邻接表中的每一个邻接结点 w)

{

if (mark [w] == unvisited)

DFS (w) ;

}

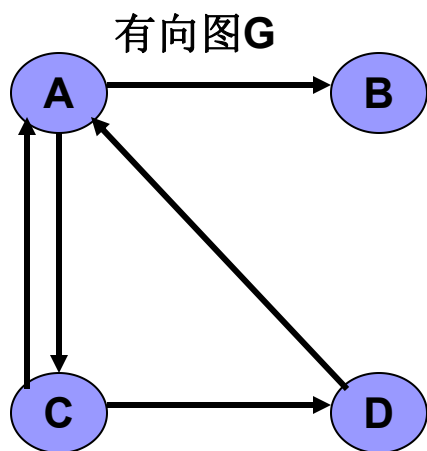
count ++ ;

finished [count] = v;

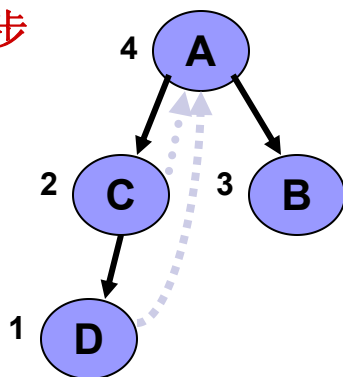
}

7.4.2 有向图的强连通分量的求法

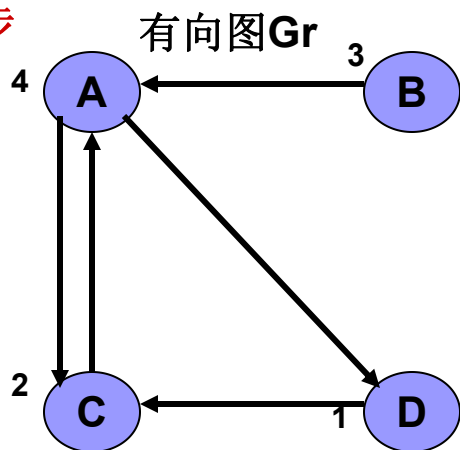
■ 求法实例



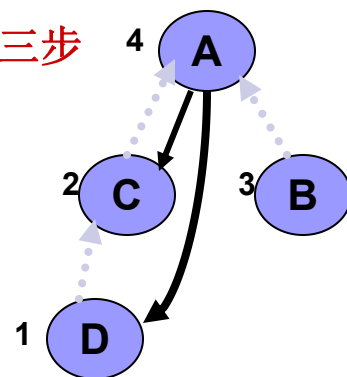
第一步



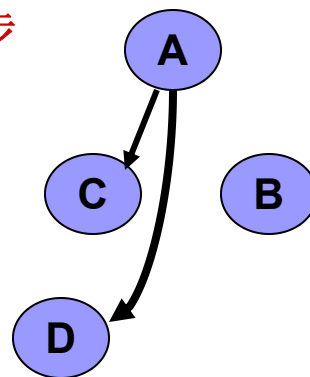
第二步



第三步



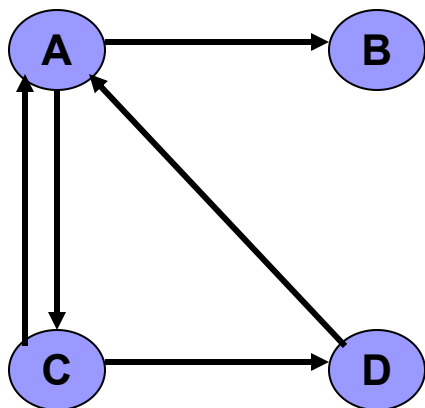
第四步



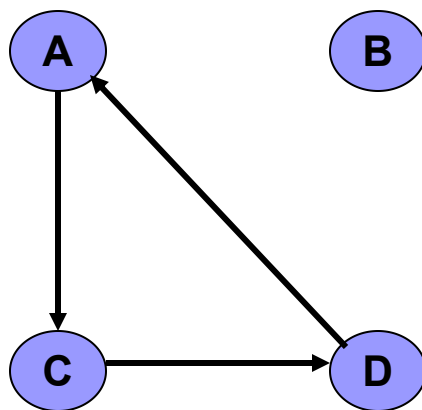
二棵生成树的结点是有向图G的两个强连通分量的顶点集： $\{A、C、D\}$ 及 $\{B\}$

7.4.2 有向图的强连通分量的求法

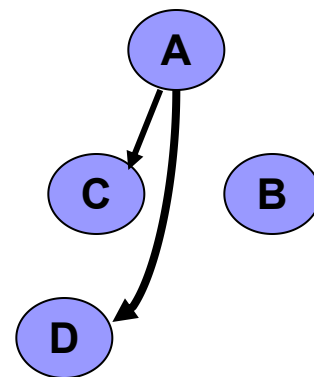
■ **断言**：有向图中的强连通分量中的顶点集和第二次在**Gr**图上深度优先搜索时得到的生成森林中的每一棵树的结点集精确对应。



有向图G



有向图G的两个强连通分量



二棵生成树的结点
是有向图G的两个
强连通分量的顶点
集：**{A、C、D}**及
{B}

7.4.2 有向图的强连通分量的求法

■ **断言：**有向图中的强连通分量中的顶点集和第二次在**Gr**图上深度优先搜索时得到的生成森林中的每一棵树的结点集精确对应。

■ **证明要点：**

- 1、一方面，相互直达的**v**、**w**，肯定在同一生成树之中。
- 2、另一方面，同一生成树中的任意二点 **v**、**w** 之间，相互可以直达。

假定在 **Gr** 中的根为**x**。

- 在 **Gr** 中，存在着由 **x** 至 **v** 的路径，那么在 **G** 中存在一条由 **v** 至 **x** 路径。在 **G** 中，由于 **x** 序号大，而 **v** 序号小；所有存在着一条 **x** 至 **v** 的路径。——**x**、**v**直达
- 同理：在 **Gr** 中，存在着由**x**至 **w** 的路径，那么在**G**中存在一条由 **w** 至 **x** 路径。 **G** 中，由于**x**序号大，而 **w** 序号小；所有存在着一条**x**至**w**的路径。——**x**、**w**直达
- 所以，**Gr** 各个生成树中的任意二点 **v**、**w** 之间，相互可以直达。

7.4.3 最小生成树

■ 问题:

假设要在 n 个城市之间建立通讯联络网，则连通 n 个城市只需要修建 $n-1$ 条线路，如何在最节省经费的前提下建立这个通讯网？

■ 该问题等价于:

构造网的一棵最小生成树，即：在 e 条带权的边中选取 $n-1$ 条边（不构成回路），使“权值之和”为最小。

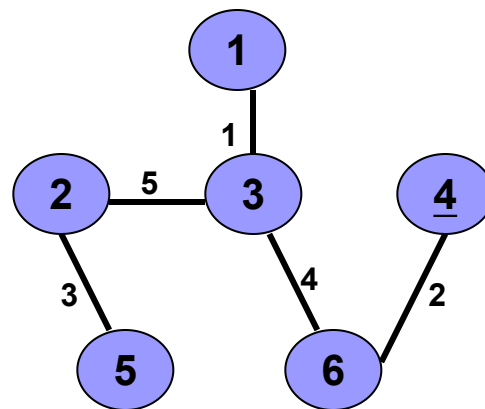
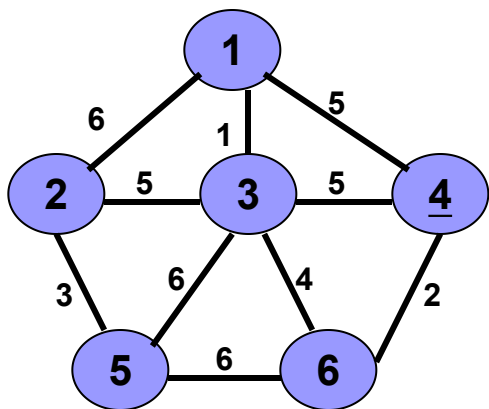
■ 解决方法:

算法一：普里姆（Prim）算法

算法二：克鲁斯卡尔（Kruscal）算法

7.4.3 最小生成树

- 定义：生成树中边的权值（代价）之和最小的树。
- 实例：



左图的最小代价生成树

7.4.3 最小生成树

■ MST 性质:

假设 $G = \{V, \{E\}\}$ 是一个连通图, U 是结点集合 V 的一个非空子集。若 (u, v) 是一条代价最小的边, 且 u 属于 U , v 属于 $V - U$, 则必存在一棵包括边 (u, v) 在内的最小代价生成树。

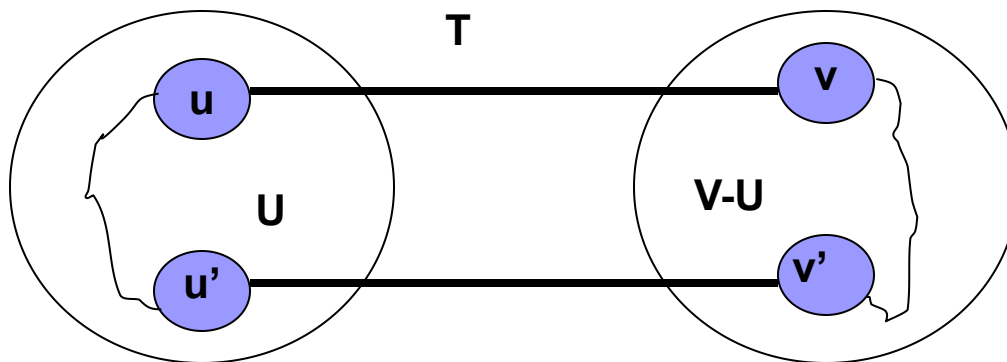
■ 证明:

- 假定存在一棵不包括边 (u, v) 在内的最小代价生成树, 设其为 T 。
- 将边 (u, v) 添加到树 T , 则形成一条包含 (u, v) 的回路。
- 因此, 必定存在另一条边 (u', v') , 且 u' 属于 U , v' 属于 $V - U$ 。删去边 (u', v') , 得到另一棵生成树 T' 。
- 因为边 (u, v) 的代价小于边 (u', v') 的代价, 所以新的生成树 T' 将是代价最小的树。和原假设矛盾。
- 新的生成树 T' 是树的理由: 连通且无回路。

7.4.3 最小生成树

■ MST 性质:

假设 $G = \{V, \{E\}\}$ 是一个连通图, U 是结点集合 V 的一个非空子集。若 (u, v) 是一条代价最小的边, 且 u 属于 U , v 属于 $V-U$, 则必存在一棵包括边 (u, v) 在内的最小代价生成树。



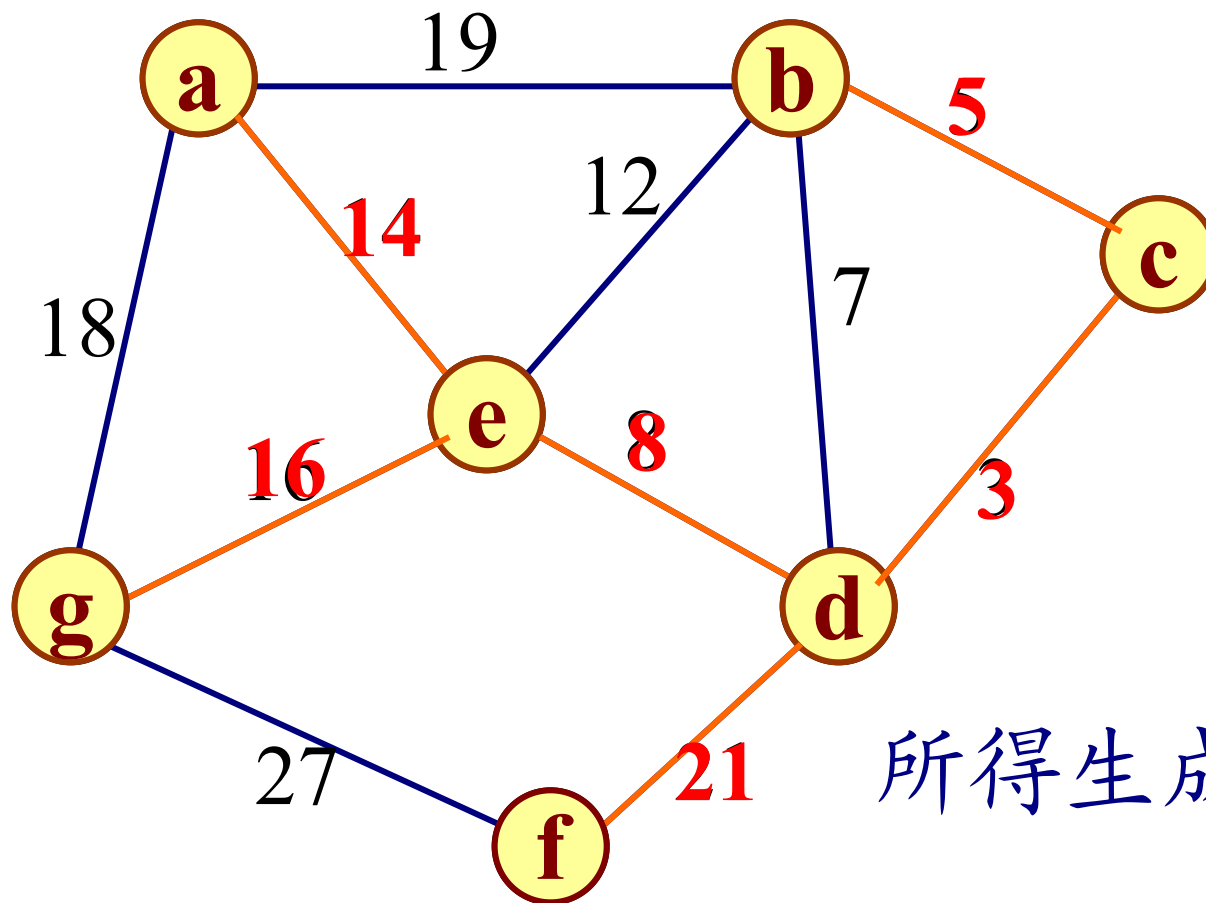
7.4.3 最小生成树——普里姆（Prim）算法

普里姆算法的基本思想：

取图中任意一个顶点 v 作为生成树的根，之后往生成树上添加新的顶点 w 。在添加的顶点 w 和已经在生成树上的顶点 v 之间必定存在一条边，并且该边的权值在所有连通顶点 v 和 w 之间的边中取值最小。之后继续往生成树上添加顶点，直至生成树上含有 $n-1$ 个顶点为止。

7.4.3 最小生成树——普里姆 (Prim) 算法

例如:

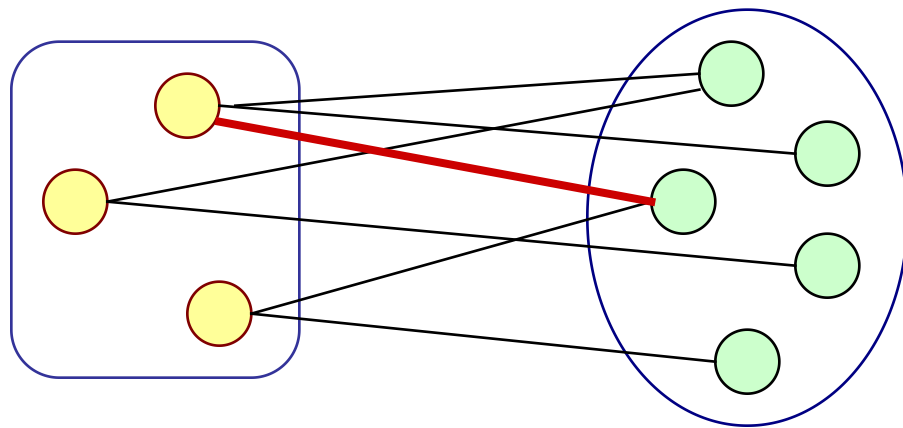


所得生成树权值和

$$= 14 + 8 + 3 + 5 + 16 + 21 = 67$$

7.4.3 最小生成树——普里姆 (Prim) 算法

- 一般情况下所添加的顶点应满足下列条件：
在生成树的构造过程中，图中 n 个顶点分属两个集合：已落在生成树上的顶点集 U 和尚未落在生成树上的顶点集 $V-U$ ，则应在所有连通 U 中顶点和 $V-U$ 中顶点的边中选取权值最小的边。



7.4.3 最小生成树——普里姆 (Prim) 算法

■ Prim 算法的基本描述

设 G : Graph, TE : 最小生成树的边集

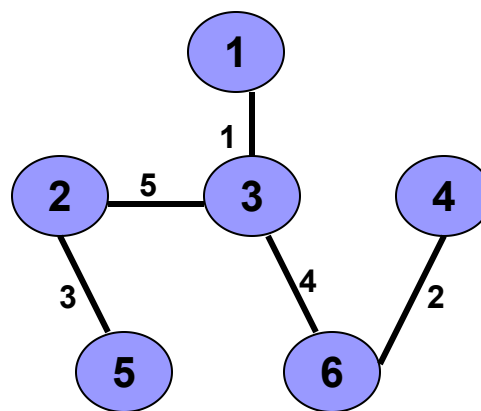
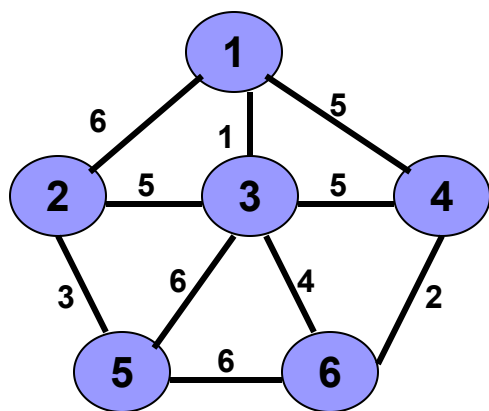
U, V : 顶点集; //初始时, V 包含所有顶点

u, v : 顶点

```
{  
     $TE = \varnothing; \quad U = \{u_0\};$   
    while (  $U \neq V$  ) {  
        设 $(u,v)$ 是一条代价最小的边, 并且 $u$ 在 $U$ 中,  $v$ 在 $V-U$ 中。  
         $TE = TE \cup (u,v);$   
         $U = U \cup \{v\};$   
    }  
}
```

时间复杂性: $O(n^2)$

7.4.3 最小生成树——普里姆（Prim）算法



最小代价生成树

7.4.3 最小生成树——普里姆（Prim）算法

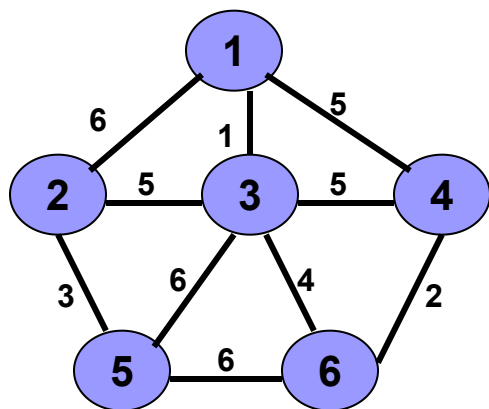
- 设置一个辅助数组 `closedge`，对当前 $V-U$ 集中的每个顶点，记录和顶点集 U 中顶点相连接的代价最小的边：

```
struct {  
    VertexType  adjvex; // 该边所依附的U中的顶点序号  
    VRType      lowcost; // 边的权值  
} closedge[MAX_VERTEX_NUM];
```

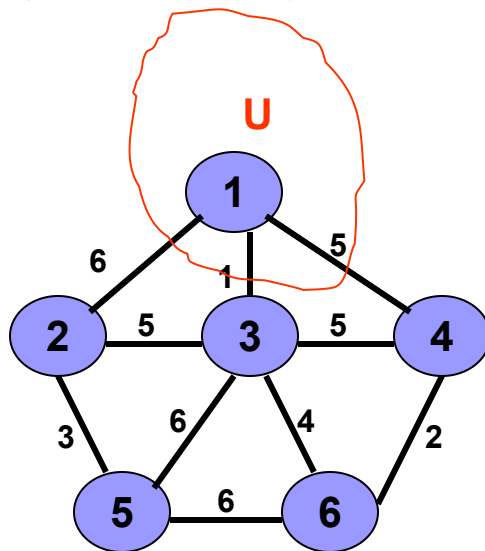
7.4.3 最小生成树——普里姆 (Prim) 算法

- 对每个顶点 v_i 属于 $V-U$ ，在辅助数组中存在一个相应分量 **closedge[i-1]**，它包括两个域：
 - **lowcost** 存储该边上的权。显然，
$$\text{closedge}[i-1].\text{lowcost} = \text{Min}\{\text{cost}(u, v_i) | u \text{ 属于 } U\}$$
 - **adjvex** 域存储该边依附在 U 中的顶点。

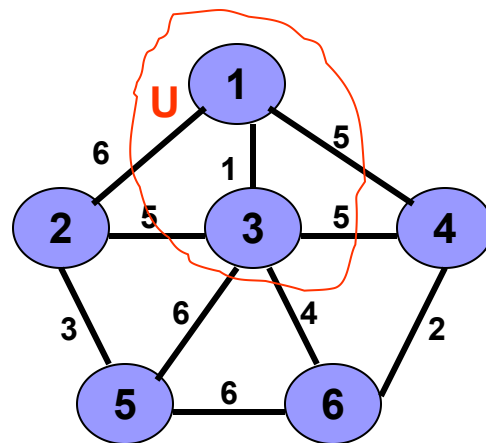
7.4.3 最小生成树——普里姆 (Prim) 算法



图G



图G



图G

注意:

closedge[0]. lowcost = 0且

closedge[2]. lowcost = 0 表示结点1和3 已在U中

数组: closedge[6]

0		0
1	0	6
2	0	1
3	0	5
4	0	∞
5	0	∞

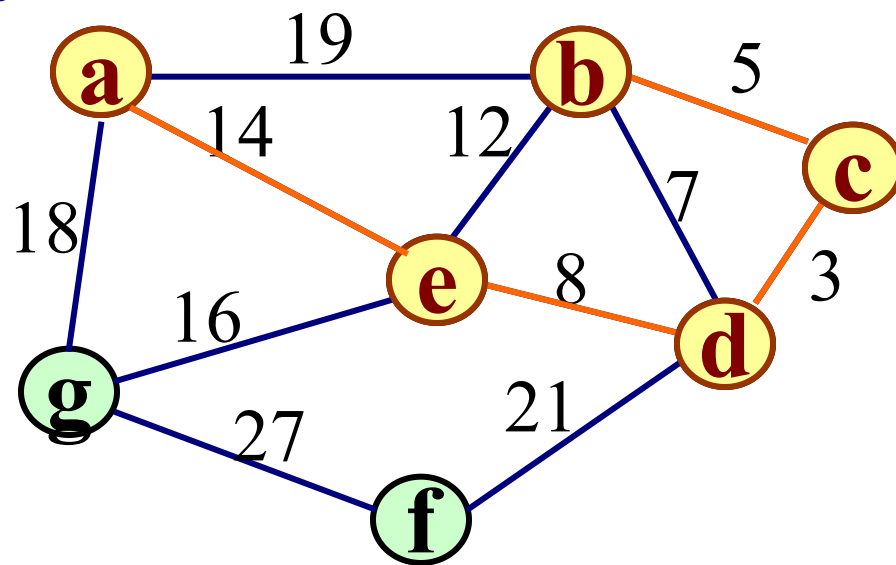
adjvex lowcost

数组: closedge[6]

0		0
1	2	5
2		0
3	0	5
4	2	6
5	2	4

adjvex lowcost

例如:



closededge \ Adjvex	0	1	2	3	4	5	6
Lowcost		c	d	e	a	d	e
		5	3	8	14	21	16

```
void MiniSpanTree_P(MGraph G, VertexType u) {  
    //用普里姆算法从顶点u出发构造网G的最小生成树  
    k = LocateVex ( G, u );  
    for ( j=0; j<G.vexnum; ++j ) // 辅助数组初始化  
        if (j!=k)  
            closedge[j] = { u, G.arcs[k][j].adj };  
    closedge[k].lowcost = 0;    // 初始, U = {u}  
    for (i=0; i<G.vexnum; ++i) {  
        继续向生成树上添加顶点;  
    }  
}
```

(继续向生成树上添加顶点)

k = minimum(closedge);

// 求出加入生成树的下一个顶点(k)

printf(closedge[k].adjvex, G.vexs[k]);

// 输出生成树上一条边

closedge[k].lowcost = 0; // 第k顶点并入U集

for (j=0; j<G.vexnum; ++j)

//修改其它顶点的最小边

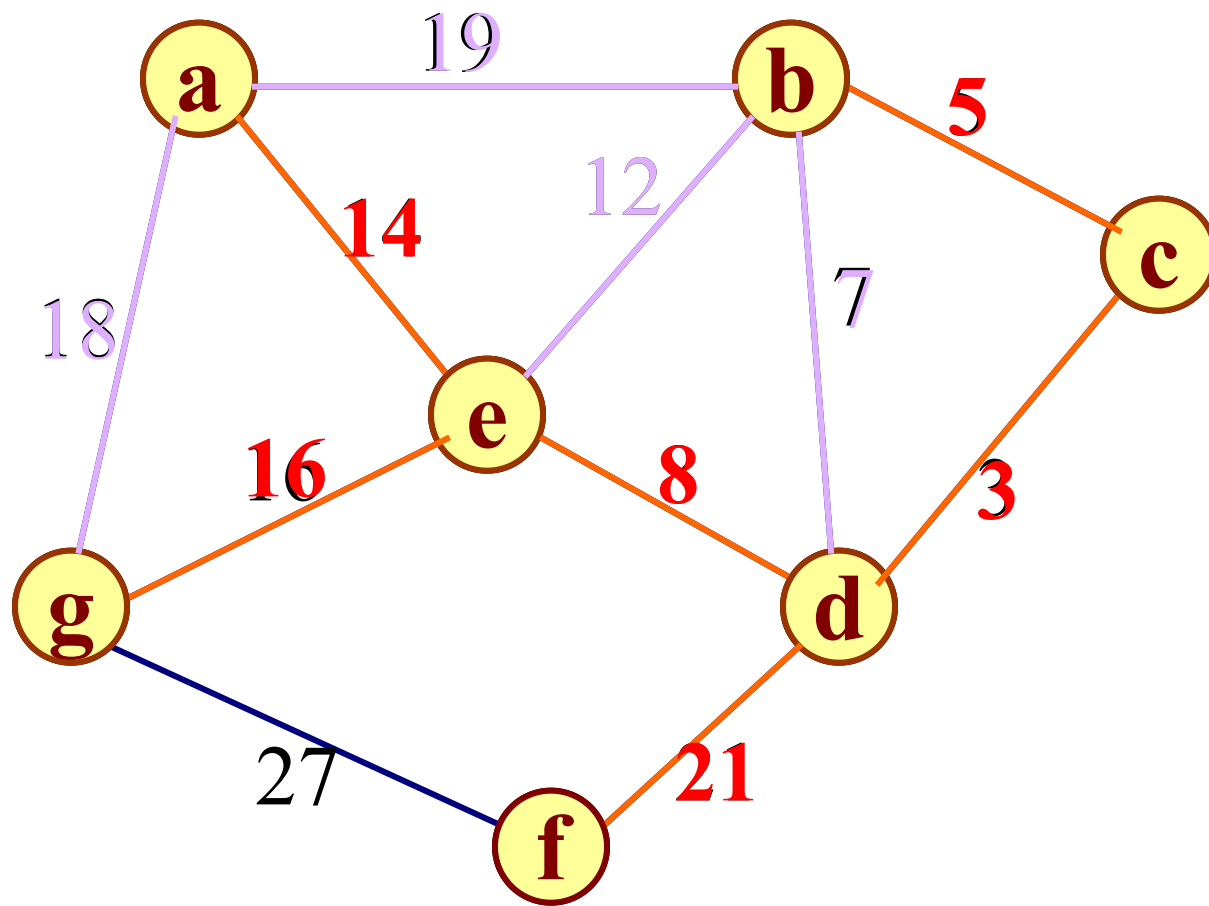
if (G.arcs[k][j].adj < closedge[j].lowcost)

closedge[j] = { G.vexs[k], G.arcs[k][j].adj };

7.4.3 最小生成树——Kruscal算法

- **克鲁斯卡尔算法考虑问题的出发点**：为使生成树上边的权值之和达到最小，则应使生成树中每一条边的权值尽可能地小。
- **具体做法**：先构造一个只含 n 个顶点的子图SG，然后从权值最小的边开始，若它的添加不使SG中产生回路，则在SG上加上这条边，如此重复，直至加上 $n-1$ 条边为止。

例如:



7.4.3 最小生成树——Kruscal算法

算法描述:

构造非连通图 $ST=(V,\{\})$;

$k = i = 0$; // k 计选中的边数

while ($k < n-1$) {

++ i ;

检查边集 E 中第 i 条权值最小的边 (u,v) ;

若 (u,v) 加入 ST 后不使 ST 中产生回路,

则 输出边 (u,v) ; 且 $k++$;

}

- 数据结构: 用堆实现, 则每次选择最小代价的边 仅需 $O(\log e)$ 的时间。

- 时间复杂性: $O(e \log e)$

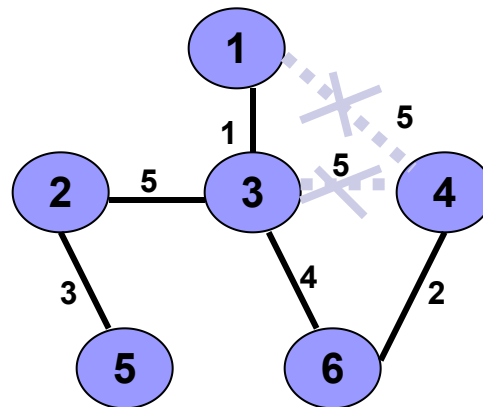
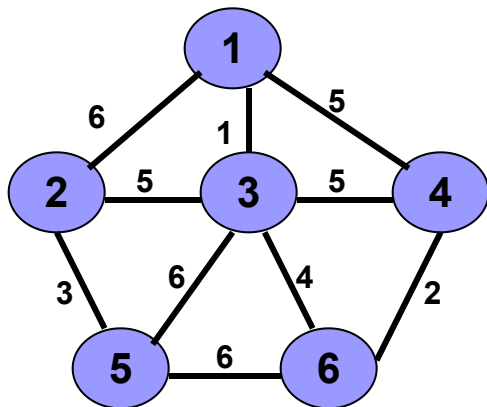
比较两种算法

算法	普里姆算法	克鲁斯卡尔算法
----	-------	---------

时间复杂度	$O(n^2)$	$O(e \log e)$
-------	----------	---------------

适应范围	稠密图	稀疏图
------	-----	-----

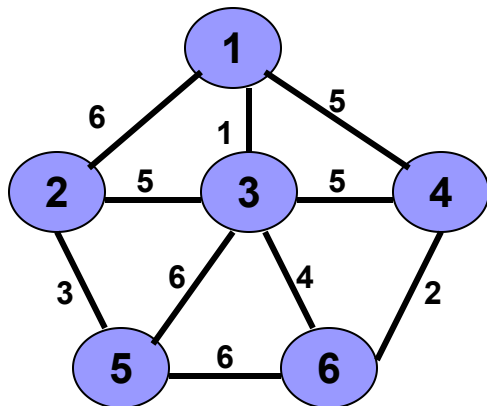
7.4.3 最小生成树——Kruscal算法



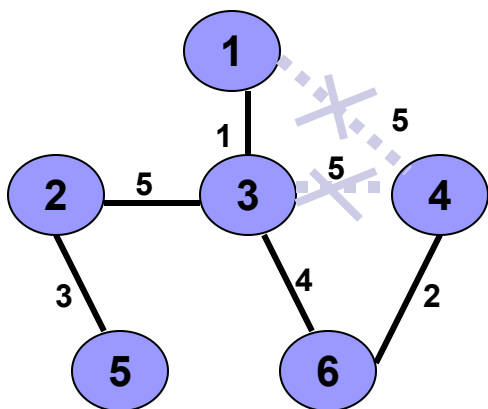
最小代价生成树

7.4.3 最小生成树——Kruscal算法

• 实例的执行过程



图G



最小代价生成树

1、初始连通分量: $\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}$

2、反复执行添加、放弃动作。条件: 边数不等于 $n-1$ 时

边	动作	连通分量
(1,3)	添加	$\{1,3\}, \{4\}, \{5\}, \{6\}, \{2\}$
(4,6)	添加	$\{1,3\}, \{4,6\}, \{2\}, \{5\}$
(2,5)	添加	$\{1,3\}, \{4,6\}, \{2,5\}$
(3,6)	添加	$\{1,3,4,6\}, \{2,5\}$
(1,4)	放弃	因构成回路
(3,4)	放弃	因构成回路
(2,3)	添加	$\{1,3,4,5,6,2\}$

第七章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

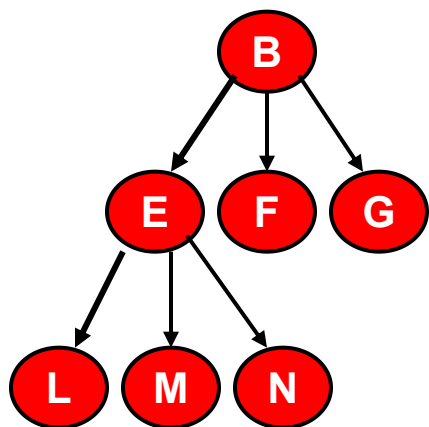
7.4 图的连通性问题

7.5 有向无环图及其应用

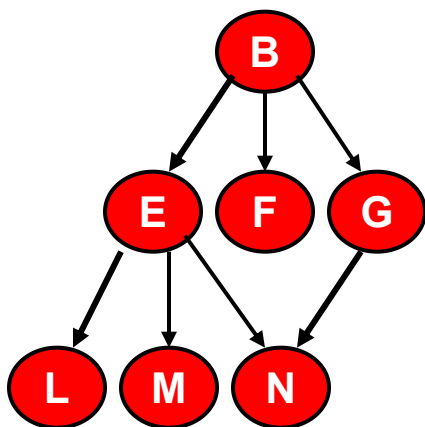
7.6 最短路径

7.5 有向无环图DAG及其应用

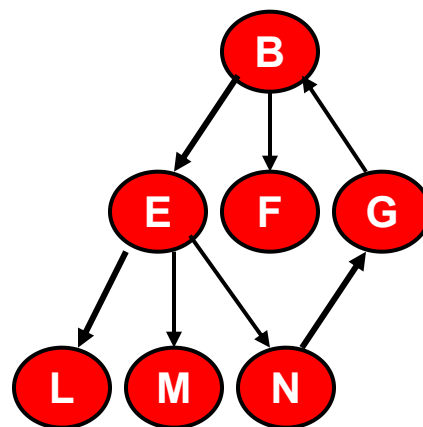
- 一个无环的有向图称为有向无环图（**DAG** 图）



有向树



DAG图



有向图（含环）

7.5.1 拓扑排序 (Topological Sort)

■ 问题:

假设以有向图表示一个工程的施工图或程序的数据流图，则图中不允许出现回路。

检查有向图中是否存在回路的方法之一，是对有向图进行**拓扑排序**。

7.5.1 拓扑排序 (Topological Sort)

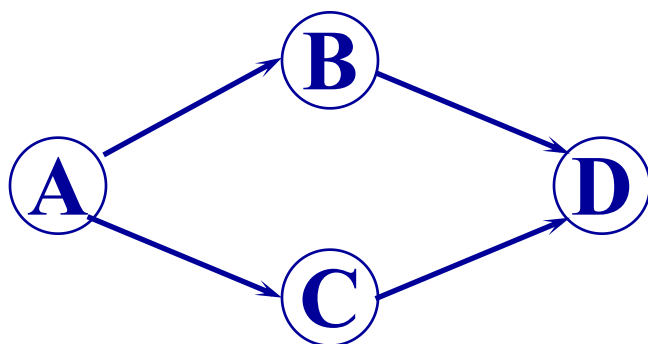
“拓扑排序”对有向图进行如下操作：

按照有向图给出的次序关系，将图中顶点排成一个线性序列，对于有向图中没有限定次序关系的顶点，则可以人为加上任意的次序关系。

由此所得顶点的线性序列称之为
拓扑有序序列

7.5.1 拓扑排序 (Topological Sort)

例如：对于下列有向图

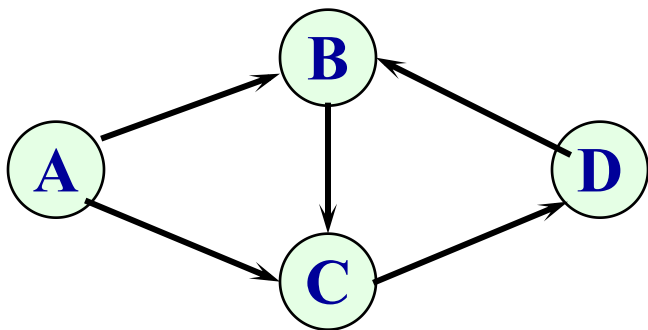


可求得拓扑有序序列：

A B C D 或 **A C B D**

7.5.1 拓扑排序 (Topological Sort)

反之，对于下列有向图



不能求得它的拓扑有序序列。

因为图中存在一个回路 {B, C, D}

7.5.1 拓扑排序 (Topological Sort)

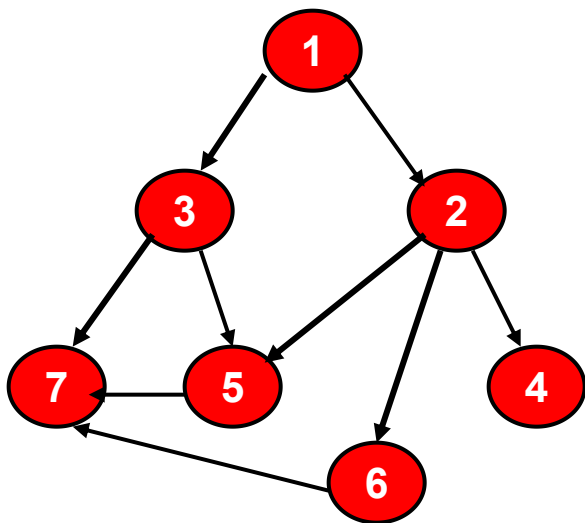
- **拓扑排序**: 由某个集合上的一个偏序得到该集合上的一个全序。
- **偏序**: 指集合中仅有部分成员之间可比较
- **全序**: 集合中全体成员之间均可比较
- **形式化描述**:

如果有一个序列 $A = a_1, a_2, a_3, \dots, a_n$ 是集合 X 上的元素的一个序列, 且当 (a_i, a_j) 属于 R 时, 则 $i < j$, 则称 A 是相对于 R 拓扑排序的。

注意:

- i 和 j 是元素 a_i 和 a_j 在序列 A 中的序号。
- 用 $a_i \rightarrow a_j$ 表示 (a_i, a_j) 属于 R , $i < j$ 。令 a_i 为前件, 而 a_j 为后件。**前件的序号一定要小于后件**。否则将不满足拓扑排序的定义。

实例：下述集合 **M** 代表课程的集合，其中，**1**代表数学，**2**代表程序设计，**3**代表离散数学，**4**代表汇编程序设计，**5**代表数据结构，**6**代表结构程序设计，**7**代表编译原理。**关系 R** 是课程学习的先后关系，如数学必须在离散数学之前学习。**要求排一张学习的先后次序表。**



序列：**1、3、2、4、6、5、7** 合乎拓扑排序的要求
序号 **1、2、3、4、5、6、7** 前件的序号 < 后件的序号

序列：**3、1、2、4、6、5、7** 不合乎拓扑排序的要求
序号 **1、2、3、4、5、6、7**
元素 **3** 的序号 **1**（后件）< 元素 **1**（前件）的序号 **2**

那么，如何得到拓扑排序？

- 第一个输出的结点（序列中的第一个元素）：必须无前件。
- 后件：必须等到它的前件输出之后，因此时序号才会大于前件的序号。
- 无前件及后件的结点：任何时候都可输出。

7.5.1 拓扑排序 (Topological Sort)

- 用顶点表示活动，用弧表示活动间的优先关系的有向图称为顶点表示活动的网 (Activity On Vertex network)，简称AOV-网。

7.5.1 拓扑排序 (Topological Sort)

■ 如何进行拓扑排序？

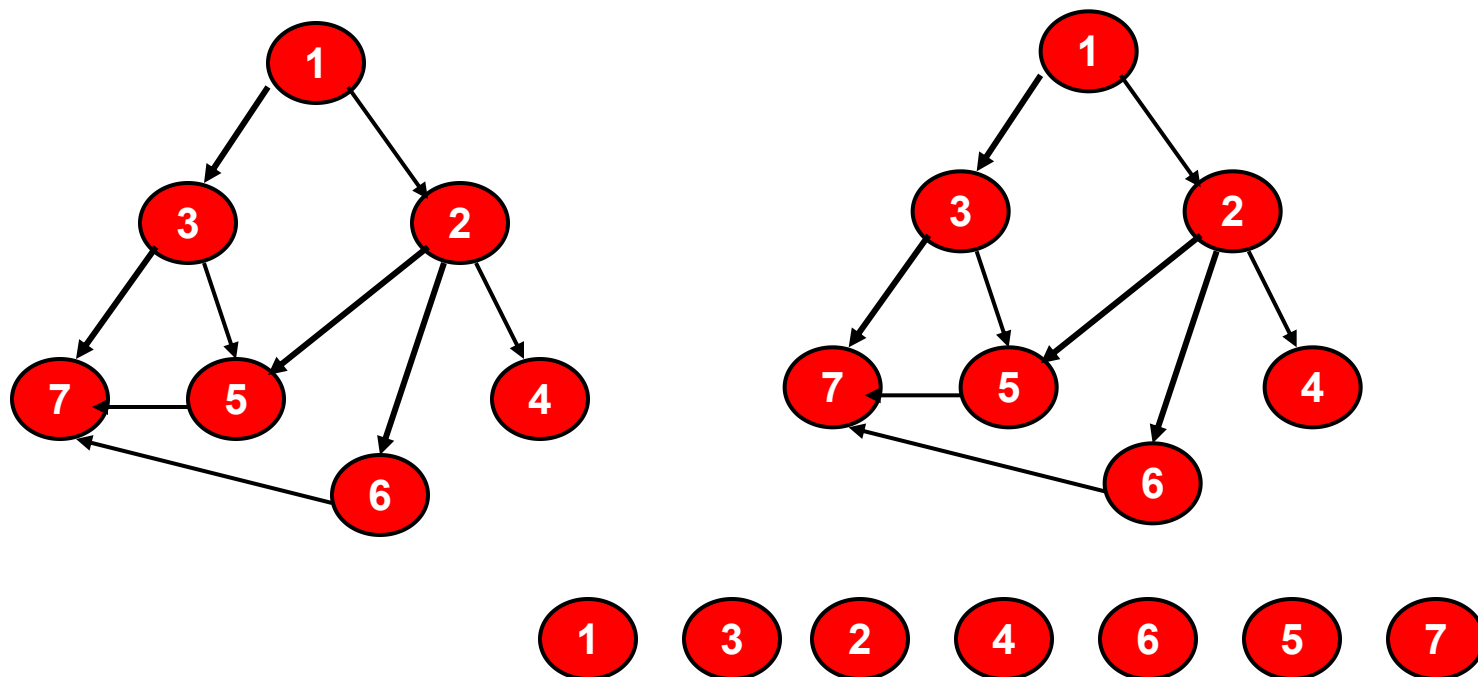
一、从有向图中选取一个没有前驱的顶点，并输出之；

二、从有向图中删去此顶点以及所有以它为尾的弧；

■ 重复上述两步，直至图空，或者图不空但找不到无前驱的顶点为止。

7.5.1 拓扑排序 (Topological Sort)

实例：下述集合 M 代表课程的集合，其中，1代表数学，2代表程序设计，3代表离散数学，4代表汇编程序设计，5代表数据结构，6代表结构程序设计，7代表编译原理。**关系 R** 是课程学习的先后关系，如数学必须在离散数学之前学习。**要求排一张学习的先后次序表。**



a b h c d g f e

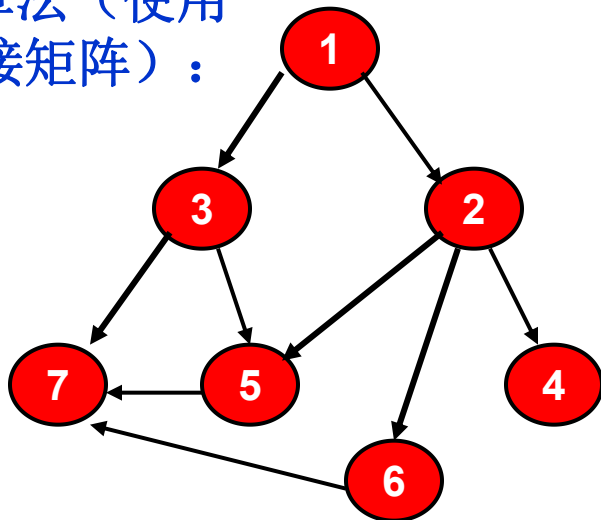
在算法中需要用定量的描述替代定性的概念

没有前驱的顶点 $\equiv$ 入度为零的顶点

删除顶点及以它为尾的弧 $\equiv$ 弧头顶点的入度减1

7.5.1 拓扑排序 (Topological Sort)

■ 算法 (使用邻接矩阵):



算法的执行步骤:

- 1、找到全为零的第 j 列, 输出 j
- 2、将第 j 行的全部元素置为零
- 3、找到全为零的第 k 列, 输出 k
- 4、将第 k 行的全部元素置为零

.....

反复执行 3、4; 直至所有元素输出完毕。

	1	2	3	4	5	6	7
1	0	1	1	0	0	0	0
2	0	0	0	0	1	1	1
3	0	0	0	0	1	0	1
4	0	0	0	0	0	0	0
5	0	0	0	0	0	0	1
6	0	0	0	0	0	0	1
7	0	0	0	0	0	0	0

	1	2	3	4	5	6	7
1	0	0	0	0	0	0	0
2	0	0	0	0	1	1	1
3	0	0	0	0	1	0	1
4	0	0	0	0	0	0	0
5	0	0	0	0	0	0	1
6	0	0	0	0	0	0	1
7	0	0	0	0	0	0	0

7.5.1 拓扑排序 (Topological Sort)

■ 算法 (使用邻接表):

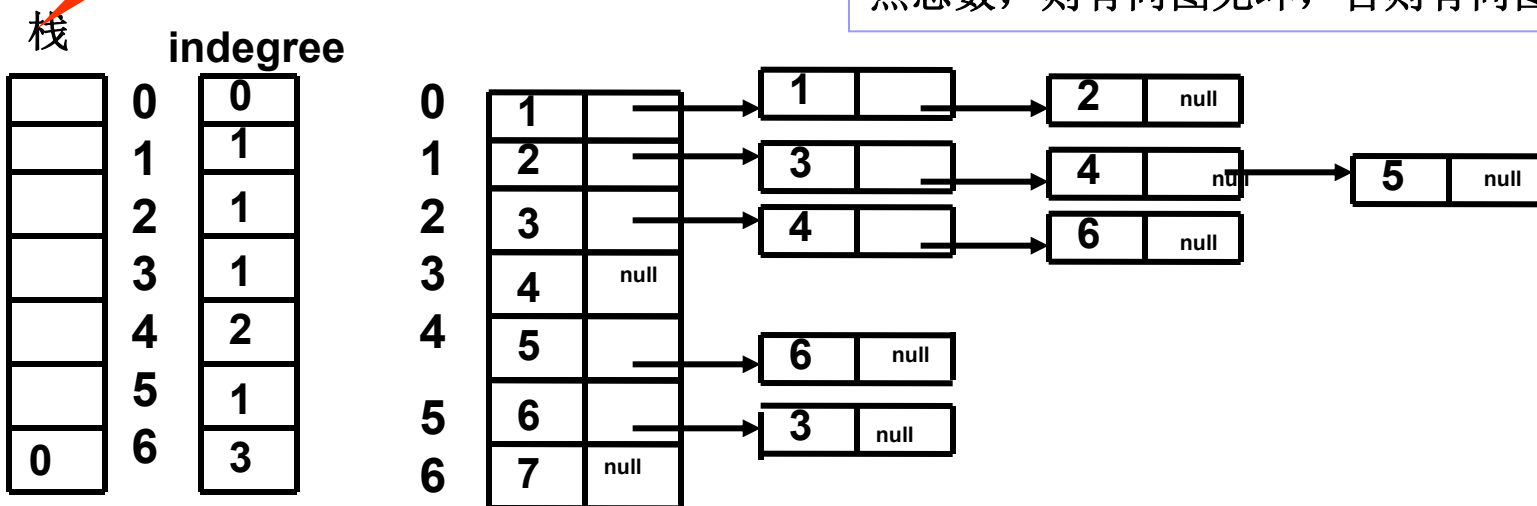
设置栈保存“入度为0”的顶点，避免每次都要搜索入度为0的顶点。

算法的执行步骤:

- 1、用一个数组记录每个结点的入度。将入度为零的结点进栈。
- 2、将栈中入度为零的结点V输出。
- 3、根据邻接表找到结点V的所有的邻接结点，并将这些邻接结点的入度减一。如果某一结点的入度变为零，则进栈。
- 4、反复执行 2、3；直至栈空为止。

.....

次序执行结束，如果输出结点数等于图的结点总数，则有向图无环，否则有向图有环。



■使用邻接表的算法

算法的时间复杂度
为: $O(n+e)$

Status Topologicalsort (ALGraph G)

```
{ FindinDegree ( G, indegree ) ; //对各顶点求入度 indegree[0..vernum-1]
  Initstack ( S ) ;
  for ( i = 0; i < G.vexnum; ++i ) //建零入度顶点栈
    if ( ! indegree [ i ] ) Push ( S, i ) ; //入度为0者进栈
  count = 0; //对输出顶点计数
  while ( ! StackEmpty ( S ) )
  { Pop(S, i); printf ( i, vertices[ i ].data ) ; ++count; //输出i号顶点并计数
    for ( p=G.vertices[i]. firstarc; p; p=p->nextarc )
    { k = p->adjvex; //对i号顶点的每个邻接点的入度减1
      if ( ! ( -- indegree [ k ] ) ) Push ( S, k ) ; }
    }
  if ( count < G.vexnum ) return ERROR; //该有向图有回路
  else return OK;
} // end
```

7.5.2 关键路径

与上述AOV-网对应的是**AOE-网**(Activity On Edge)即边表示活动的网。AOE-网是一个带权的有向无环图，其中，**顶点**表示事件，**弧**表示活动，**权**表示活动持续的时间。通常，AOE-网可用来估算工程的完成时间。

7.5.2 关键路径

问题：——估算工程项目完成时间

假设以有向网表示一个施工流图，弧上的权值表示完成该项子工程所需时间。

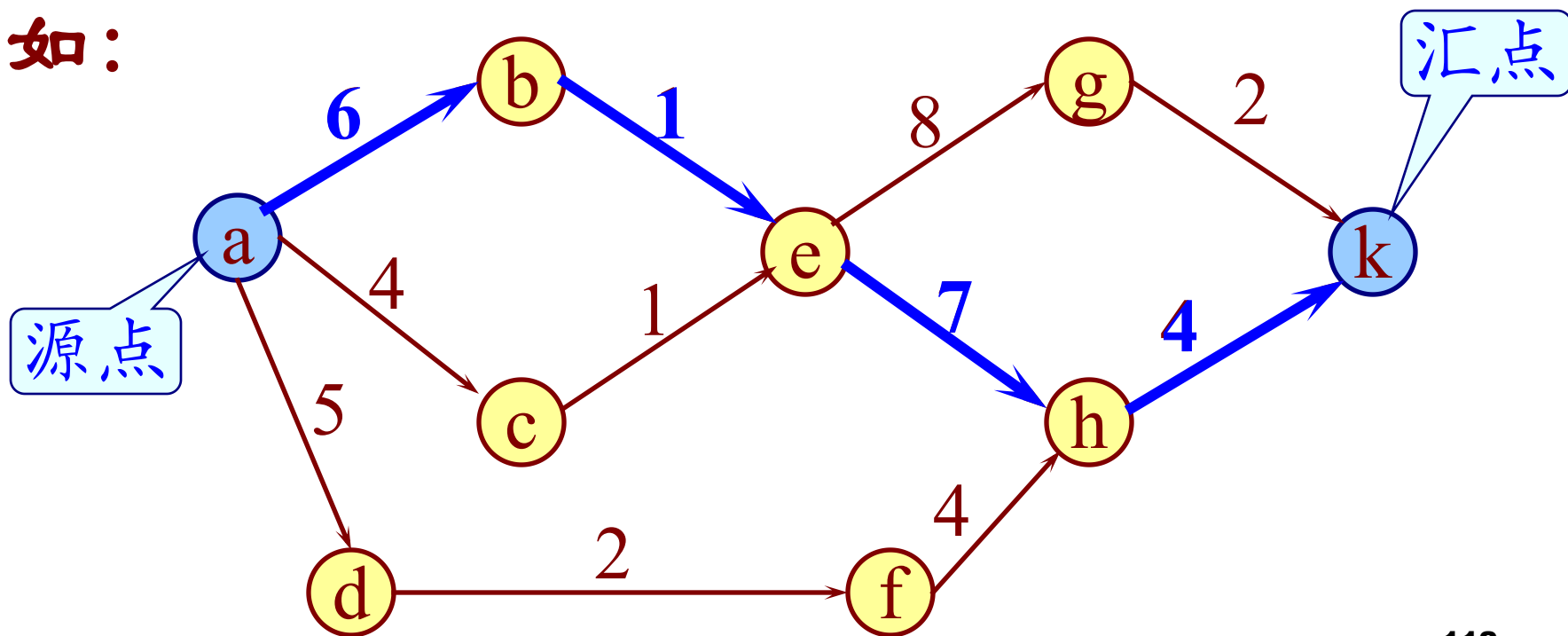
问：（1）完成整项工程至少需要多少时间？

（2）哪些子工程项是“**关键工程**”？即：哪些子工程项将影响整个工程的完成期限的。

7.5.2 关键路径

- 整个工程完成的时间为：从有向图的源点到汇点的最长路径。
- 路径长度是指路径上各活动持续时间之和，不是路径上弧的数目。
- 路径长度最长的路径叫做关键路径（**Critical Path**）。
- “关键活动”指的是：该弧上的权值增加将使有向图上的最长路径的长度增加。

例如：



7.5.2 关键路径

意味着事件最早能够发生的时刻。

如何求关键活动？

“事件(顶点)” 的最早发生时间 $ve(j)$

$ve(j)$ = 从源点到顶点j的最长路径长度；

“事件(顶点)” 的最迟发生时间 $vl(k)$

$vl(k)$ = 从顶点k到汇点的最短路径长度。

不影响工程的如期完工，本结点事件必须发生的时刻。

7.5.2 关键路径

- 假设第*i*条弧为<*j*, *k*>, 其持续时间为 $dut(<j,k>)$, 则对第*i*项活动:

“活动(弧)”的最早开始时间:

$$e(i) = ve(j)$$

“活动(弧)”的最迟开始时间:

$$l(i) = vl(k) - dut(<j,k>)$$

- 关键活动: 最早开始时间=最迟开始时间的活动
- 关键路径: 从源点到收点的最长的一条路径, 或者全部由关键活动构成的路径。

7.5.2 关键路径

- 事件发生时间 $ve(j)$ 和 $vl(j)$ 的递推计算步骤:

(1) $ve(\text{源点}) = 0;$

$$ve(j) = \text{Max}\{ve(i) + dut(<i, j>)\}$$

$<i, j>$ 属于 T , 其中 T 是所有以第 j 个顶点为头的弧的集合, $j=1, 2, \dots, n-1$ 。

(2) $vl(\text{汇点}) = ve(\text{汇点});$

$$vl(i) = \text{Min}\{vl(j) - dut(<i, j>)\}$$

$<i, j>$ 属于 S , 其中 S 是所有以第 i 个顶点为尾的弧的集合, $i=n-2, \dots, 0$ 。

。

7.5.2 关键路径

算法的实现要点:

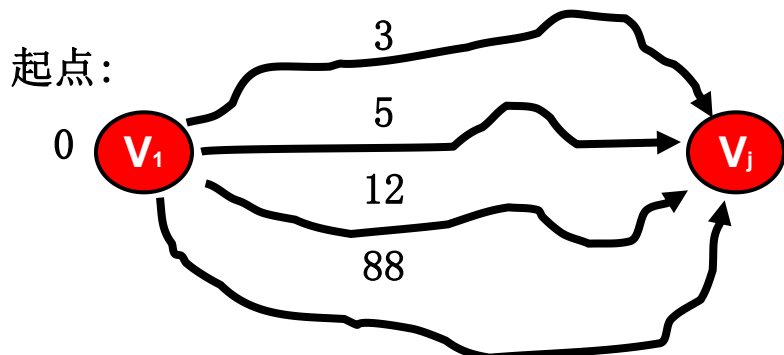
- 求 v_e 的顺序应该是按拓扑有序的次序;
- 求 v_l 的顺序应该是按逆拓扑有序的次序;

因为逆拓扑有序序列即为拓扑有序序列的逆序列,

因此应该在拓扑排序的过程中, 另设一个“栈”记下拓扑有序序列。

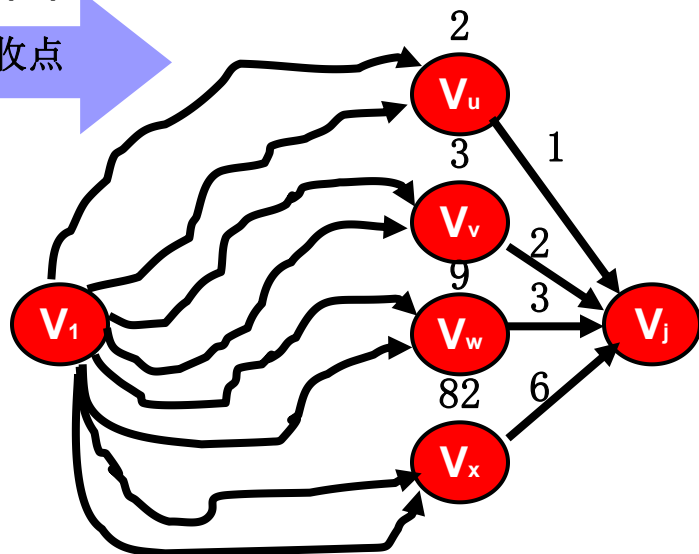
7.5.2 关键路径

■ $Ve(j)$ 及 $VI(j)$ 的求法:

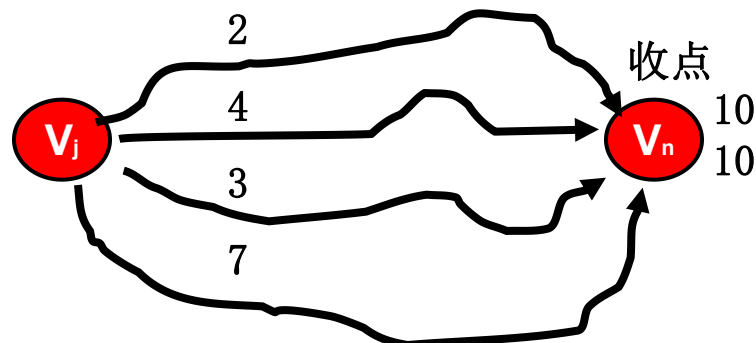


$Ve(V_j) = 3, 5, 12, 88$ 的最大值 88

由源点至收点

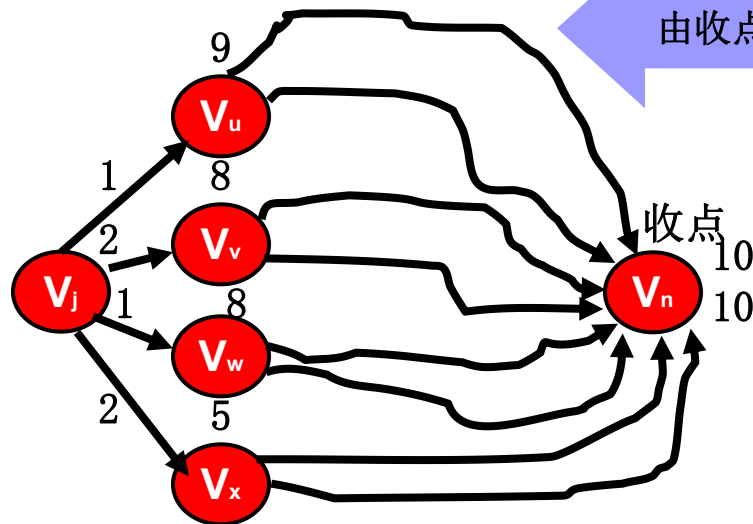


$Ve(V_j) = V_i$ 的起始结点的最早发生时间 + 各自的边的权值中的和的最大值 = 88



$VI(V_j) =$ 取 $10-2, 10-4, 10-3, 10-7$ 的最小值 3

由收点至源点



$VI(V_j) =$ 取终止结点的最迟发生时间 - 各自的边的权值的差的最小值 = 3

7.5.2 关键路径

实例：求事件结点的最早发生时间

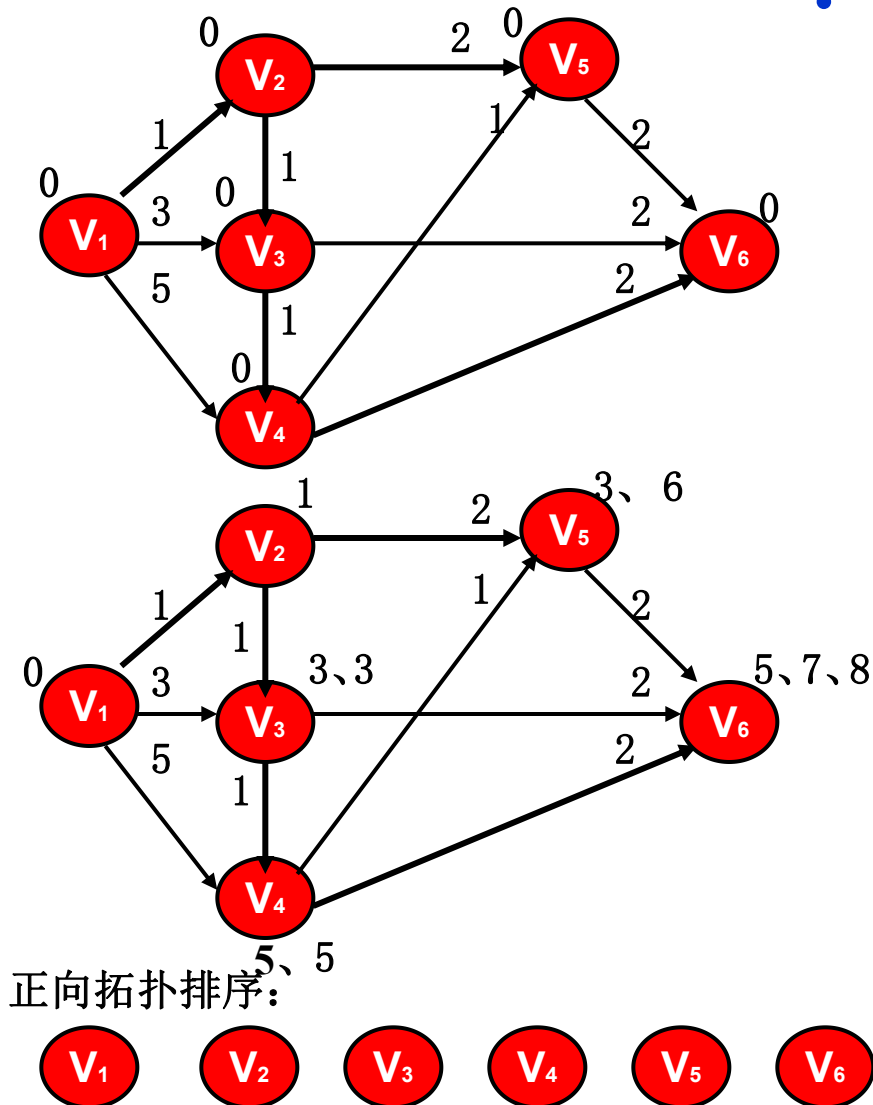
利用拓扑排序算法求事件结点的最早发生时间的执行步骤：

1、设每个结点的最早发生时间为0，将入度为零的结点**进栈**。

2、将栈中入度为零的结点**V**取出，并压入另一栈，用于形成逆向拓扑排序的序列。。

3、根据**邻接表**找到结点**V**的所有的邻接结点，将结点**V**的最早发生时间 + 活动的权值 得到的和同邻接结点的原最早发生时间进行比较；**如果该值大，则用该值取代原最早发生时间**。另外，将这些邻接结点的入度减一。如果某一结点的入度变为零，则进栈。

4、反复执行 2、3；直至栈空为止。

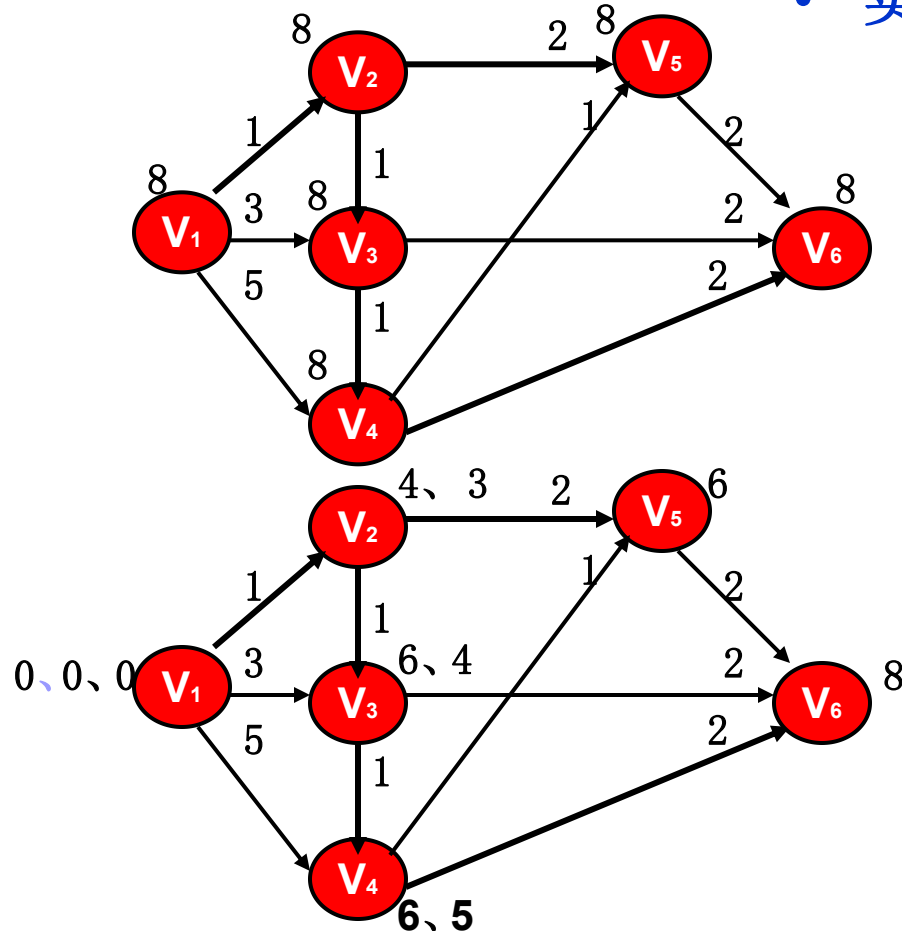


Status Topologicalsort (ALGraph G, Stack &T)

```
{ //有向网G采用邻接表存储结构，求各顶点事件的最早发生时间ve（全局变量）。  
//T为拓扑序列顶点栈。  
// 对各顶点求入度，建立入度为零的栈 S，  
FindinDegree (G, indegree) ;  
Initstack (T) ; count = 0;   ve [ 0 .. G.vexnum - 1 ] = 0;  
while ( ! StackEmpty (S) )  
    { Pop (S, j) ;Push (T, j) ; ++count; // j号顶点入栈并计数  
      for (p=G.vertices[i]. firstarc; p; p=p->nextarc)  
          { k = p->adjnextr; //对j号顶点的每个邻接点的入度减1  
            if ( ! ( - - indegree [ k ] ) ) Push (S, k) ; //若入度减为0，则入栈  
            if (ve[j] + * ( p->info ) > ve[ k ] )  
                ve[ k ] = ve[ j ] + * ( p->info ) ; }  
    }  
if ( count < G.vexnum ) return ERROR; //有回路  
else return OK; //G无回路，用栈T返回一个拓扑序列  
} // 栈 T 为求事件的最迟发生时间的时候用。
```

7.5.2 关键路径

- 实例：求事件结点的最迟发生时间



利用**逆向**拓扑排序算法求事件结点的最迟发生时间的执行步骤：

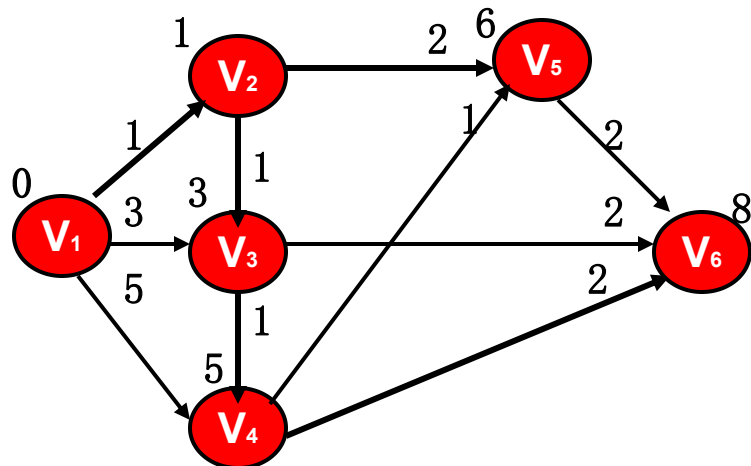
- 1、设每个结点的最迟发生时间为收点的最早发生时间。
- 2、将栈中的结点**V**取出。
- 3、根据**逆邻接表**找到结点**V**的所有的起始结点，将结点**V**的最迟发生时间 - 活动的权值得到的差同起始结点的原最迟发生时间进行比较；如果该值小，则用该值取代原最迟发生时间。
- 4、反复执行 2、3；直至栈空为止。

逆向拓扑排序：

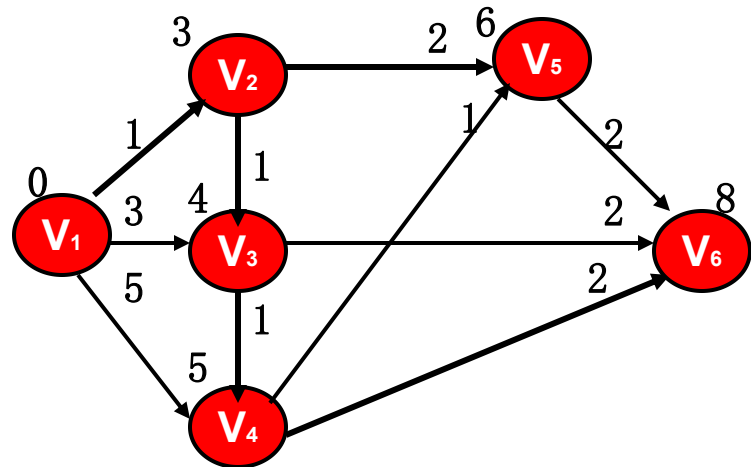


7.5.2 关键路径

- 实例的事件结点的最早发生时间



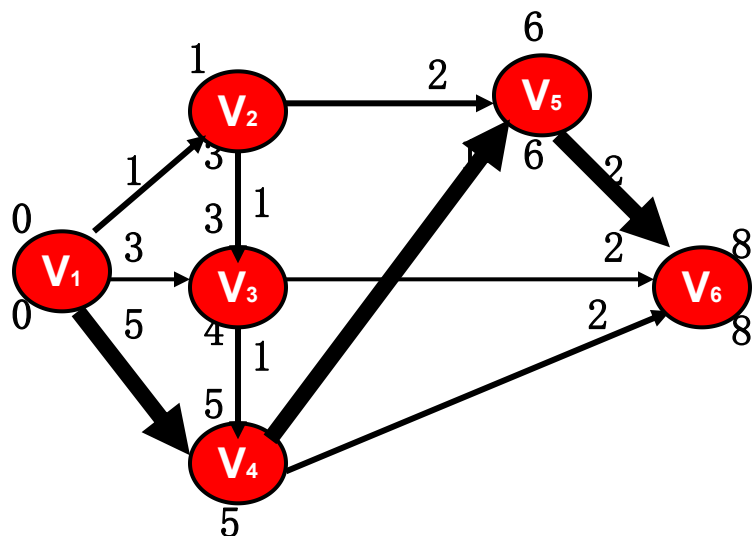
- 实例的事件结点的最迟发生时间



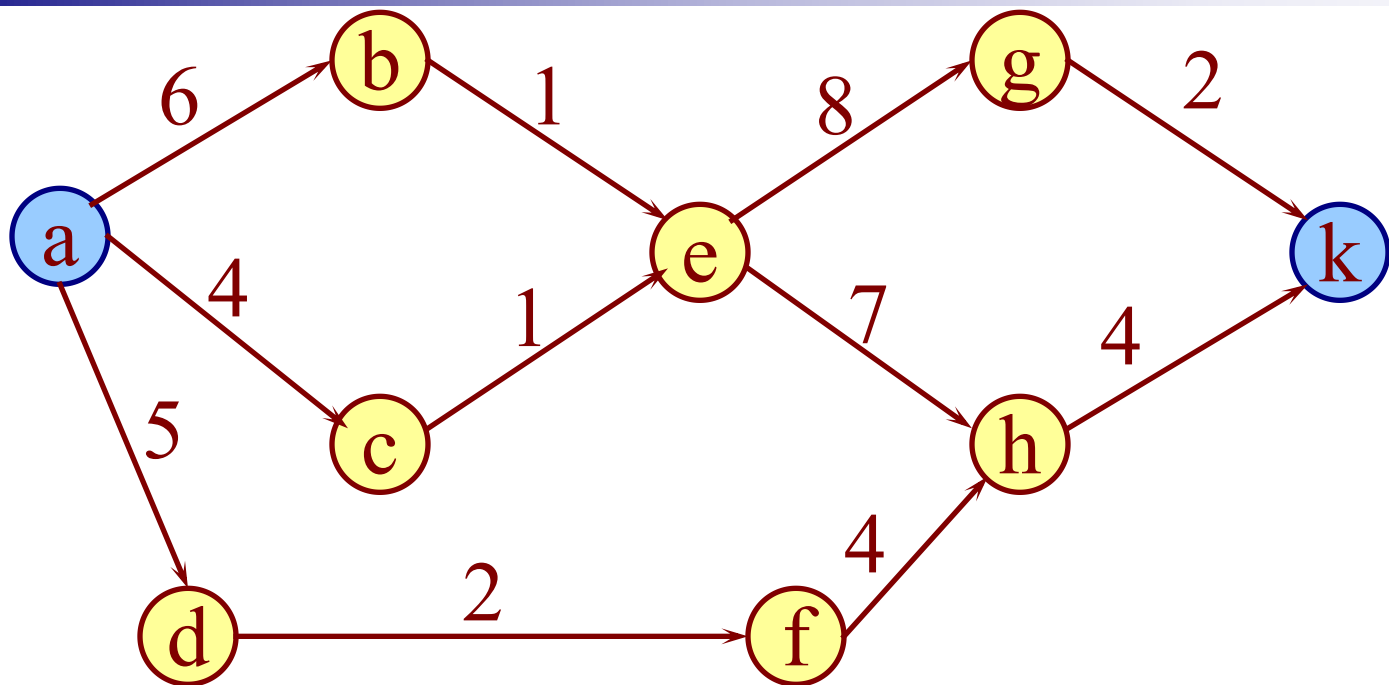
边	最早发生时间	最迟发生时间	
	0	2	
	0	1	
	0	0	关键活动
	1	3	
	1	4	
	3	4	
	3	6	
	5	5	关键活动
	5	6	
	6	6	关键活动

7.5.2 关键路径

- 实例的关键路径（粗大的黑色箭头所示）



- 算法7.14的时间复杂度为 $O(n+e)$ ，其中计算活动最早开始时间和最迟开始时间的复杂度均为 $O(e)$ 。
- 注意：关键路径可有多条。
- 缩短工期必须缩短关键活动所需的时间。



	a	b	c	d	e	f	g	h	k
ve	0	6	4	5	7	7	15	14	18
vl	0	6	6	8	7	10	16	14	18

拓扑有序序列: **a - d - f - c - b - e - h - g - k**

	a	b	c	d	e	f	g	h	k
ve	0	6	4	5	7	7	15	14	18
vl	0	6	6	8	7	10	16	14	18

	ab	ac	ad	be	ce	df	eg	eh	fh	gk	hk
权	6	4	5	1	1	2	8	7	4	2	4
e	0	0	0	6	4	5	7	7	7	15	14
l	0	2	3	6	6	8	8	7	10	16	14
	☒			☒				☒			☒

第七章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 图的连通性问题

7.5 有向无环图及其应用

7.6 最短路径

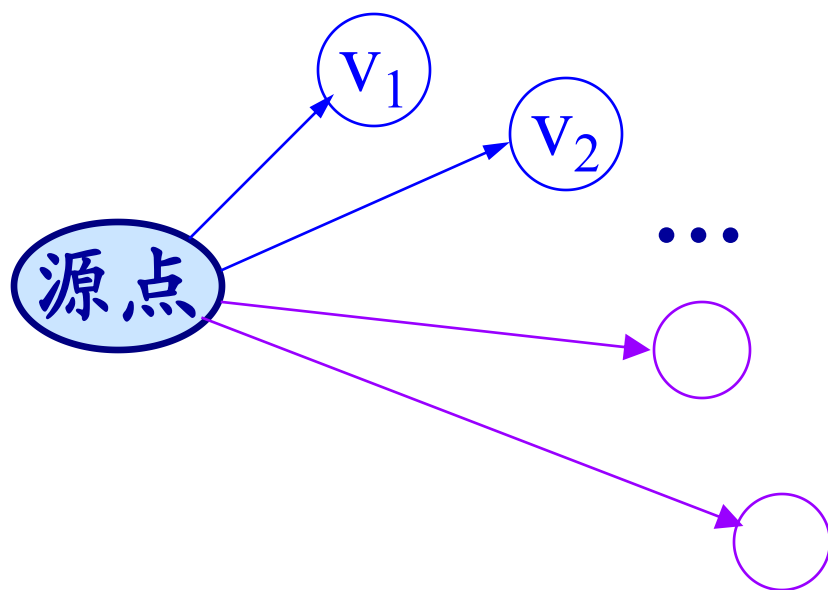
7.6 最短路径

- 求从某个源点到其余各点的最短路径
- 每一对顶点之间的最短路径

7.6.1 求从某个源点到其余各顶点的最短路径

算法的基本思想:

依最短路径的长度递增的次序求得各条路径



其中，从源点到
顶点 v 的最短路径
是所有最短路径
中长度最短者。

7.6.1 求从某个源点到其余各顶点的最短路径

■ 路径长度最短的最短路径的特点：

在这条路径上，必定只含一条弧，并且这条弧的权值最小。

■ 下一条路径长度次短的最短路径的特点：

它只可能有两种情况：或者是直接从源点到该点（只含一条弧）；或者是从源点经过顶点 v_1 ，再到达该顶点（由两条弧组成）。

■ 再下一条路径长度次短的最短路径的特点：

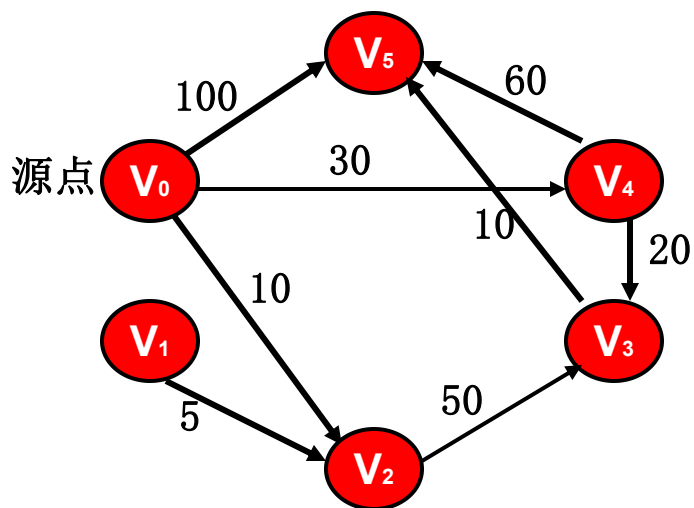
它可能有三种情况：或者是直接从源点到该点（只含一条弧）；或者是从源点经过顶点 v_1 ，再到达该顶点（由两条弧组成）；或者是从源点经过顶点 v_2 ，再到达该顶点。

■ 其余最短路径的特点：

它或者是直接从源点到该点（只含一条弧）；或者是从源点经过已求得最短路径的顶点，再到达该顶点。

7.6.1 求从某个源点到其余各顶点的最短路径

■ 实例:加权有向图



加权邻接矩阵:

	0	1	2	3	4	5
0	∞	∞	10	∞	30	100
1	∞	∞	5	∞	∞	∞
2	∞	∞	∞	50	∞	∞
3	∞	∞	∞	∞	∞	10
4	∞	∞	∞	20	∞	60
5	∞	∞	∞	∞	∞	∞

源点	终点	最短路径	路径长度
V ₀	V ₁	无	
	V ₂	(V ₀ , V ₂)	10
	V ₃	(V ₀ , V ₄ , V ₃)	50
	V ₄	(V ₀ , V ₄)	30
	V ₅	(V ₀ , V ₄ , V ₃ , V ₅)	60

7.6.1 求从某个源点到其余各顶点的最短路径

■ 求最短路径的迪杰斯特拉(Dijkstra)算法:

设置辅助数组**Dist**，其中每个分量**Dist[k]** 表示当前所求得的从源点到其余各顶点 **k** 的最短路径。

■ 一般情况下，

Dist[k] = <源点到顶点 k 的弧上的权值>

或者 = **<源点到其它顶点的路径长度>**

+ <其它顶点到顶点 k 的弧上的权值>。

7.6.1 求从某个源点到其余各顶点的最短路径

1) 在所有从源点出发的弧中选取一条权值最小的弧, 即为第一条最短路径。

$$Dist[k] = \begin{cases} arcs[v_0][k] & V_0 \text{ 和 } k \text{ 之间存在弧} \\ \infty & V_0 \text{ 和 } k \text{ 之间不存在弧} \end{cases}$$

其中的最小值即为最短路径的长度。

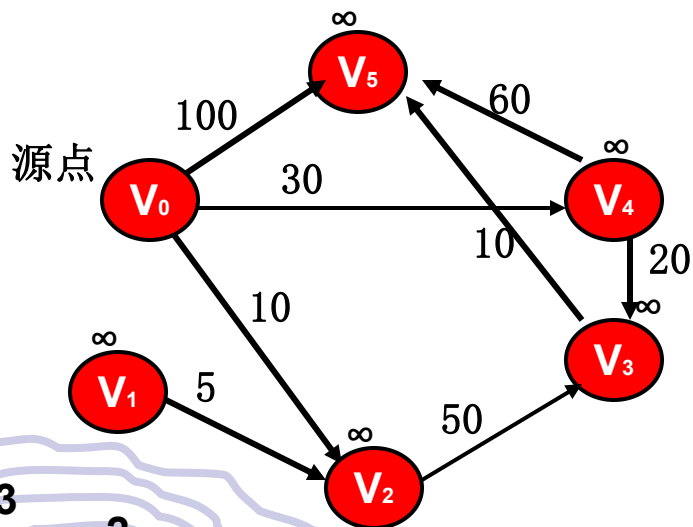
2) 修改其它各顶点的 $Dist[k]$ 值。

假设求得最短路径的顶点为 u , 若

$$Dist[u] + arcs[u][k] < Dist[k]$$

则将 $Dist[k]$ 改为 $Dist[u] + arcs[u][k]$ 。

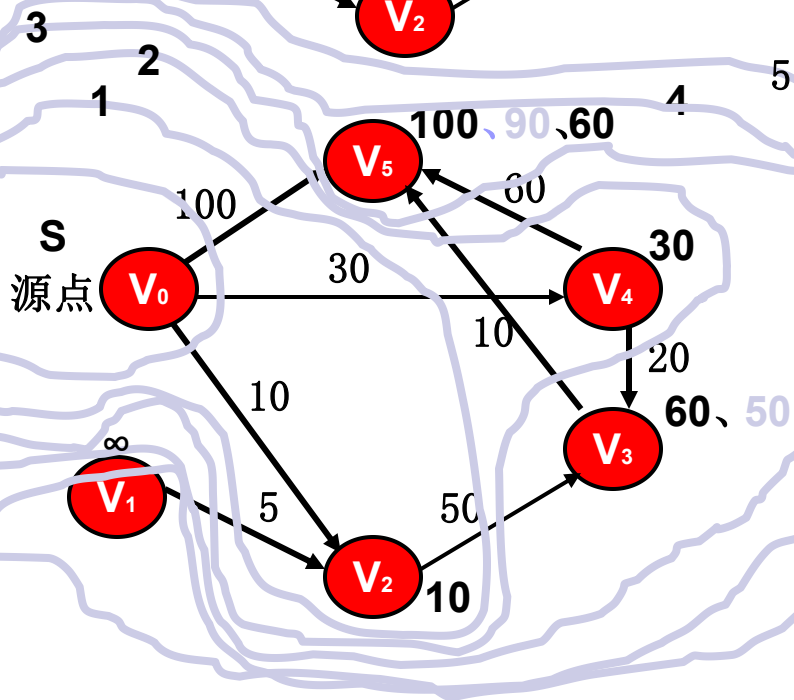
7.6.1 求从某个源点到其余各顶点的最短路径



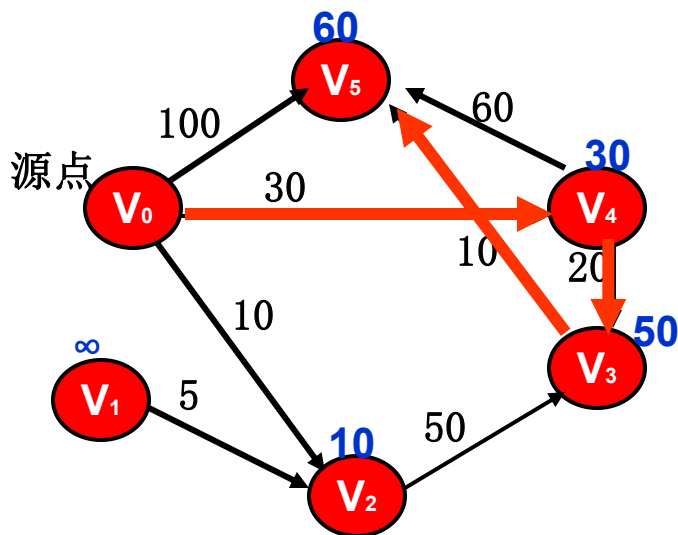
Dijkstra 算法求单源最短路径:

设 V 是该有向图的结点的集合、集合 S 是已求得最短路径的结点的集合, 求 V_0 至其余各结点的最短距离。

- 1、 $S = \{0\}$; // 结点 V_0 最短路径已求得
- 2、for ($i=1$; $i < n$; $i++$)
- 3、 $D[i] = c[0, i]$; // V_0 至 V_i 的边长
- 4、for ($i=1$; $i < n$; $i++$)
- 5、{ 在 $V - S$ 中选择一个结点 V_w ; 使得 $D[w]$ 最小。将 w 加入集合 S
- 6、for (每一个在 $V - S$ 中的结点 V)
- 7、{ $D[v] = \min(D[v], D[w] + C[w, v])$;
- 8、}
- 9、}

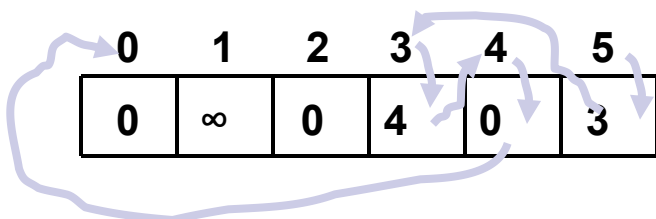


7.6.1 求从某个源点到其余各顶点的最短路径



• Dijkstra 算法:

路径的求法: 在 8、处增加 $P[v] = w$;



• Dijkstra 算法的时间复杂性:

邻接矩阵: $O(n*n)$

Dijkstra 算法求单源最短路径:

设 V 是该有向图的结点的集合、集合 S 是已求得最短路径的结点的集合, 求 V_0 至其余各结点的最短距离。

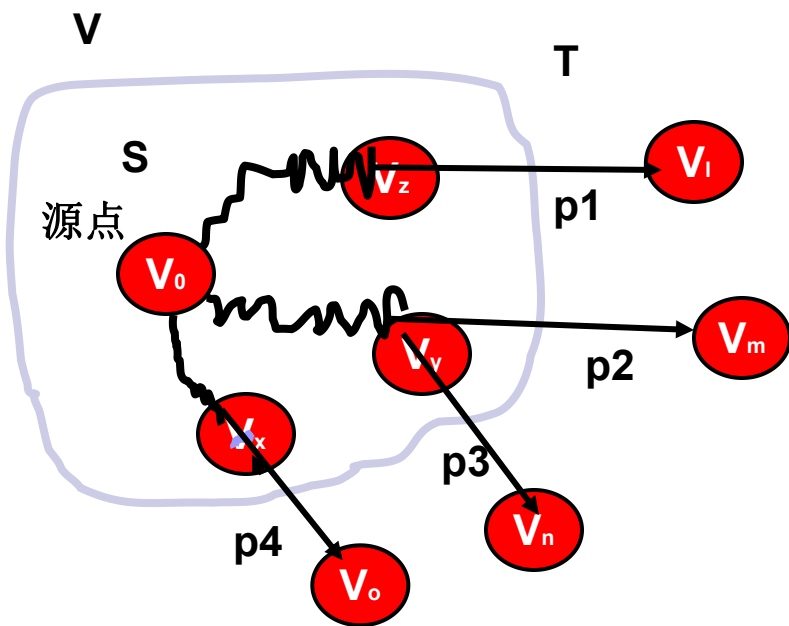
- 1、 $S = \{0\}$; // 结点 V_0 最短路径已求得
- 2、for ($i=1; i<n; i++$)
- 3、 $D[i]=c[0, i]$; // V_0 至 V_i 的边长
- 4、for ($i=1; i<n; i++$)
- 5、{ 在 $V-S$ 中选择一个结点 V_w ; 使得 $D[w]$ 最小。将 W 加入集合 S
- 6、for (每一个在 $V-S$ 中的结点 V)
- 7、 $\{ D[v]=MIN(D[v], D[w]+C[w,v])$;
- 8、如果 $D[V]$ 改变, 则 $P[v] = w$;
- 9、}

7.6.1 求从某个源点到其余各顶点的最短路径

• Dijkstra 算法: 正确性证明

设 V 是结点的集合, S 是已经得到最短路径 (由源点至本结点) 的结点的集合。

$$T = V - S$$

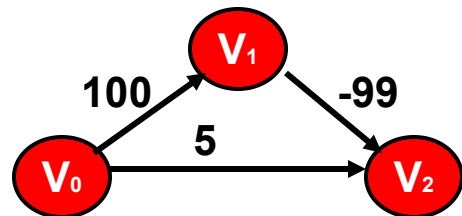


证明:

设 V_0 经 S 中的结点, 到 V_i 后直接经有向边到达结点 V_i 的路径, 是所有这些路径中最短的。如图所示, 由 V_0 出发的路径中, $p_1 < p_2; p_1 < p_3; p_1 < p_4$ 。则 p_1 就是源点 V_0 至 V_i 的最短路径。

假定存在一条更短的路径, 那么肯定要经过 $T - \{ V_i \}$ 中的一个顶点, 设为 V_n ; 但 $p_3 > p_1$, 同假设矛盾, 不可能。

Dijkstra 算法: 边的权值不可以为负值。



7.6.2 每一对顶点之间的最短路径

弗洛伊德（Floyd）算法的基本思想是：

从 V_i 到 V_j 的所有可能存在的路径中，选出一条长度最短的路径。

若 $\langle v_i, v_j \rangle$ 存在, 则存在路径 $\{v_i, v_j\}$

// 路径中不含其它顶点

若 $\langle v_i, v_1 \rangle, \langle v_1, v_j \rangle$ 存在, 则存在路径 $\{v_i, v_1, v_j\}$

// 路径中所含顶点序号不大于1

若 $\{v_i, \dots, v_2\}, \{v_2, \dots, v_j\}$ 存在,

则存在一条路径 $\{v_i, \dots, v_2, \dots, v_j\}$

// 路径中所含顶点序号不大于2

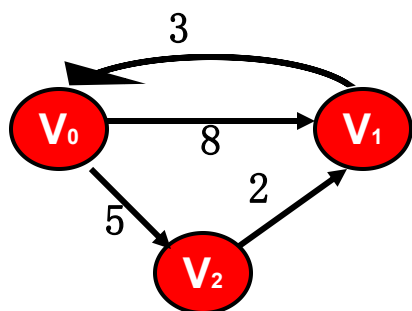
...

依次类推, 则 v_i 至 v_j 的最短路径应是上述这些路径中, 路径长度最小者。

7.6.2 每一对顶点之间的最短路径:

弗洛伊德 (Floyd) 算法

- 加权有向图: 求各结点之间的最短路径



	0	1	2
0	0	8	5
1	3	0	∞
2	∞	2	0

右图的代价矩阵 C

Floyd 算法的求解过程

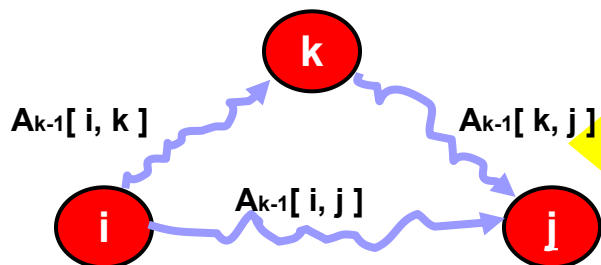
设 C 为 n 行 n 列的代价矩阵, $c[i, j]$ 为 $i \rightarrow j$ 的权值。如果 $i=j$; 那么 $c[i, j] = 0$ 。如果 i 和 j 之间无有向边; 则 $c[i, j] = \infty$

- 1、使用 n 行 n 列的矩阵 A 用于计算最短路径。

初始时, $A[i, j] = c[i, j]$

- 2、进行 n 次迭代

在进行第 k 次迭代时, 我们将使用如下的公式:



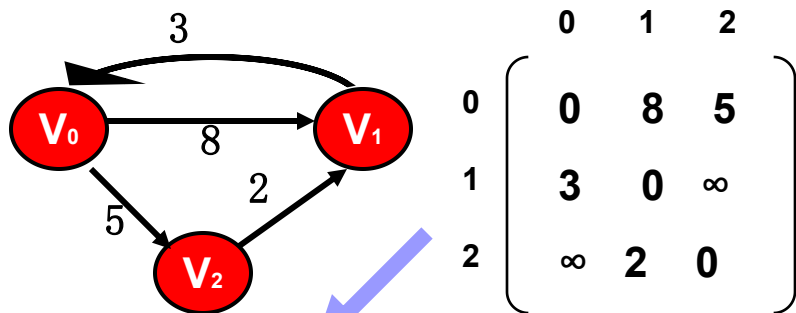
$$A_k[i, j] = \min \begin{cases} A_{k-1}[i, j] \\ A_{k-1}[i, k] + A_{k-1}[k, j] \end{cases}$$

注意: 第 k 次迭代时, 针对结点 k 进行。原 A_{k-1} 矩阵的第 k 行, 第 k 列保持不变。左上至右下的对角线元素也不变。

k 从 0 取到 $n-1$, 表示路径前 k 个顶点是所能找到的最短路径。

7.6.2 每一对顶点之间的最短路径： 弗洛伊德（Floyd）算法

实例及求解过程：



$$\begin{bmatrix} 0 & 8 & 5 \\ 3 & 0 & \infty \\ \infty & 2 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 8 & 5 \\ 3 & 0 & 8 \\ \infty & 2 & 0 \end{bmatrix}$$

$A_{\text{初值}}$

A_0

$$\begin{bmatrix} 0 & 8 & 5 \\ 3 & 0 & 8 \\ 5 & 2 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 7 & 5 \\ 3 & 0 & 8 \\ 5 & 2 & 0 \end{bmatrix}$$

A_1

A_2

Floyd 算法的求解过程

设 C 为 n 行 n 列的代价矩阵， $c[i, j]$ 为 $i \rightarrow j$ 的权值。如果 $i=j$ ；那么 $c[i, j]=0$ 。如果 i 和 j 之间无有向边；则 $c[i, j]=\infty$

1、使用 n 行 n 列的矩阵 A 用于计算最短路径。

初始时， $A[i, j] = c[i, j]$

2、进行 n 次迭代

在进行第 k 次迭代时，我们将使用如下的公式：

$$A_k[i, j] = \min \begin{cases} A_{k-1}[i, j] \\ A_{k-1}[i, k] + A_{k-1}[k, j] \end{cases}$$

注意：第 k 次迭代时，针对结点 k 进行。原 A_{k-1} 矩阵的第 k 行，第 k 列保持不变。左上至右下的对角线元素也不变。

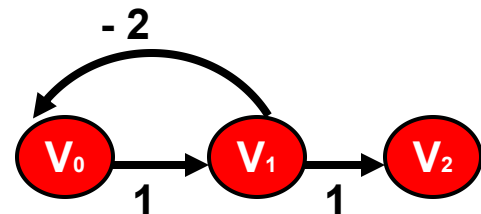
7.6.2 每一对顶点之间的最短路径: 弗洛伊德 (Floyd) 算法

- 算法的程序实现:

Floyd 算法的程序的简单描述:

```
int i, j, k; // 代表结点
for ( i = 0; i < n; i++ )
    for ( j = 0; j < n; j++ ) A[ i, j ] = C[ i, j ];
for ( i = 0; i < n; i++ ) A[ i, i ] = 0;
for ( k = 0; k < n; k++ )
    for ( i = 0; i < n; i++ )
        for ( j = 0; j < n; j++ )
            if ( A[ i, k ] + A[ k, j ] < A[ i, j ] )
                A[ i, j ] = A[ i, k ] + A[ k, j ];
```

- 应用范围: 无负长度的圈



$$\begin{bmatrix} 0 & 1 & \infty \\ -2 & 0 & 1 \\ \infty & \infty & 0 \end{bmatrix}$$

A 初值

$$\begin{bmatrix} 0 & 1 & \infty \\ -2 & -1 & 1 \\ \infty & \infty & 0 \end{bmatrix}$$

A_0

$A_1[0, 2] \neq$
 $\text{MIN} \{ A_0[0, 2],$
 $A_0[0, 1]$
 $+ A_0[1, 2]$
 $\}$

因为 $V_0 \rightarrow V_1 \rightarrow V_0$
可以有許多圈。

- 算法的时间代价: $O(n^3)$

本章要点

1. 熟悉图的各种存储结构及其构造算法，了解实际问题的求解效率与采用何种存储结构和算法有密切联系。
2. 熟练掌握图的两搜索路径的遍历：遍历的逻辑定义、深度优先搜索和广度优先搜索的算法。

在学习中应注意图的遍历算法与树的遍历算法之间的类似和差异。

3. 应用图的遍历算法求解各种简单路径问题。
4. 理解教科书中讨论的各种图的算法。
5. 熟练掌握两种最小生成树算法。
6. 熟悉有向无环图的两个重要应用：拓扑排序和关键路径。
7. 了解最短路径算法。

- 
- 
- 推荐作业:
 - 7.1, 7.15, 7.16
 - 7.3, 7.4, 7.25, 7.27
 - 7.7, 7.9, 7.33
 - 7.10, 7.11, 7.13, 7.36

本章重点复习

1、图的存储结构

图的四种常用的存储形式：

- 数组：邻接矩阵和加权邻接矩阵（**labeled adjacency matrix**）

- 邻接表

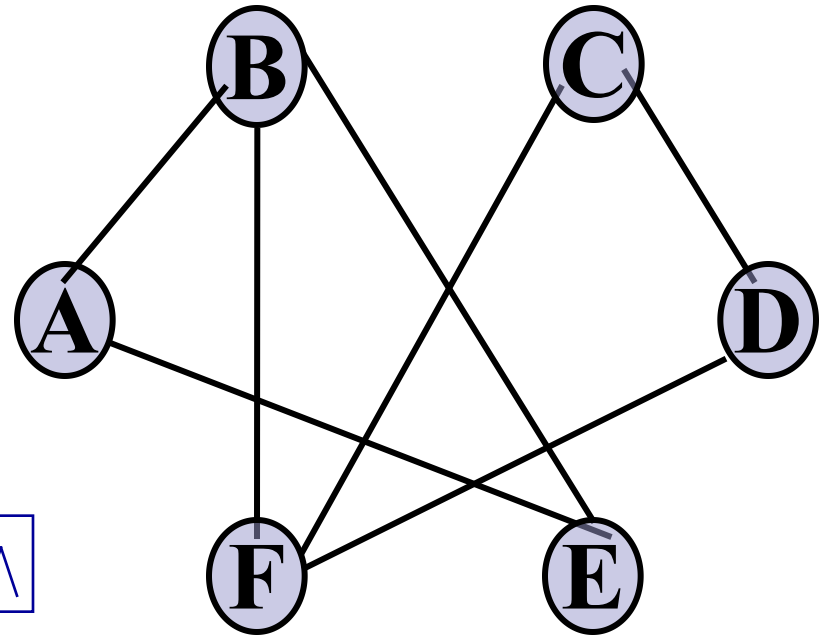
用于有向图，查询进入结点和离开结点的边容易

- 十字链表

- 邻接多重表

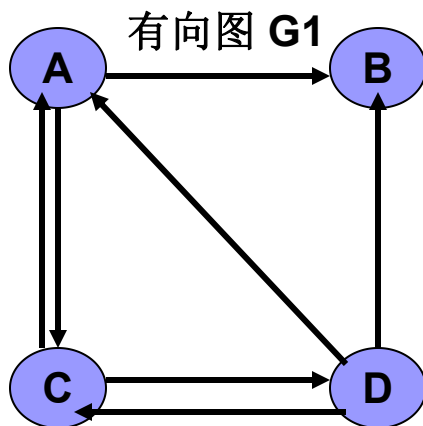
用于无向图，边表中的边结点只需存放一次，节约内存。

邻接矩阵&邻接表

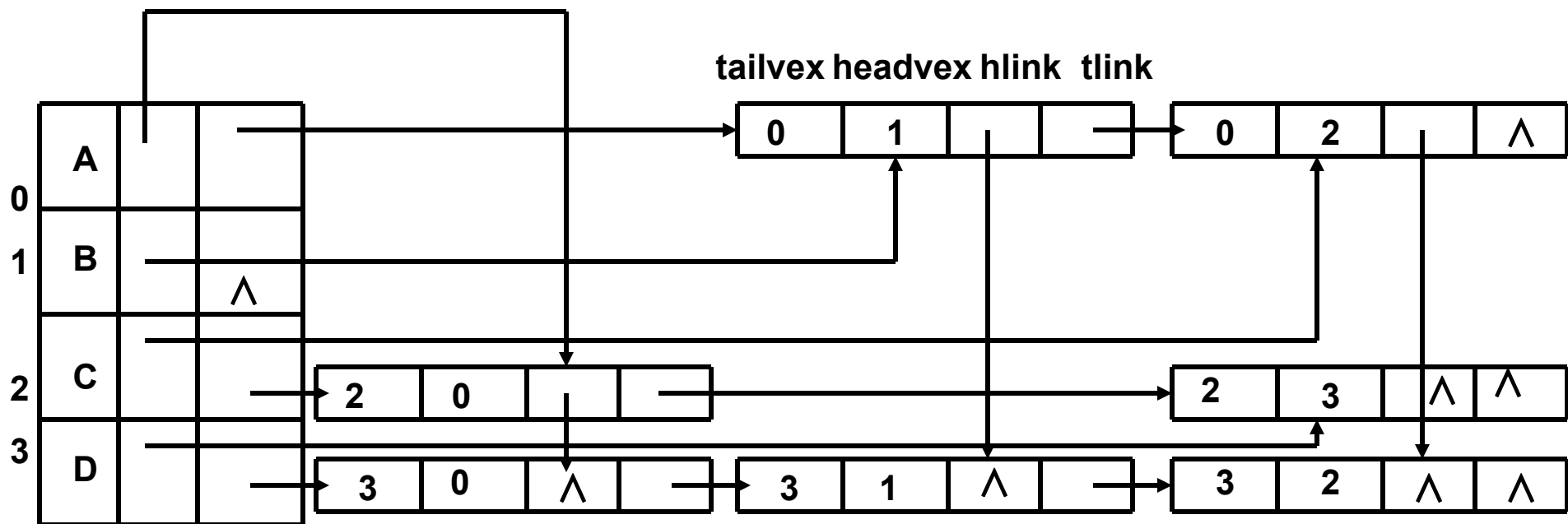


0	A	→	1	→	4	∧
1	B	→	0	→	4	→ 5 ∧
2	C	→	3	→	5	∧
3	D	→	2	→	5	∧
4	E	→	0	→	1	∧
5	F	→	1	→	2	→ 3 ∧

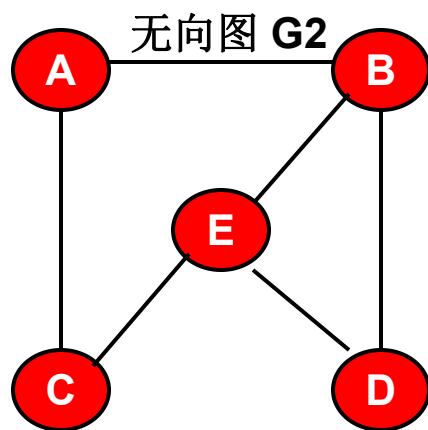
十字链表



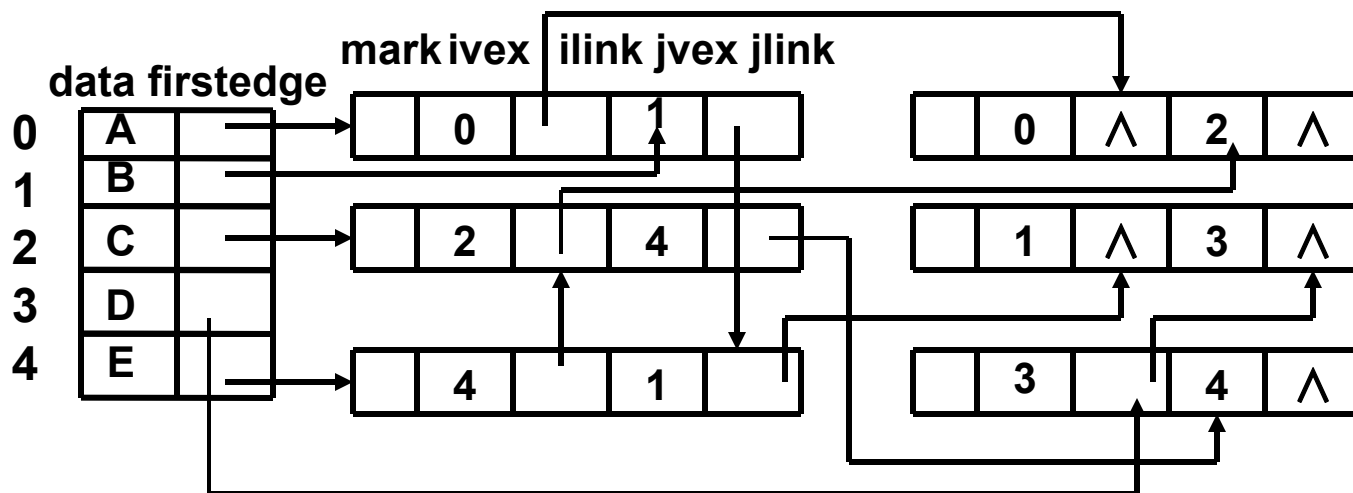
- 用途：用于有向图，查询进入结点和离开结点的边容易。



邻接多重表



- **用途**：用于**无向图**，边表中的边结点只需存放一次，节约内存。

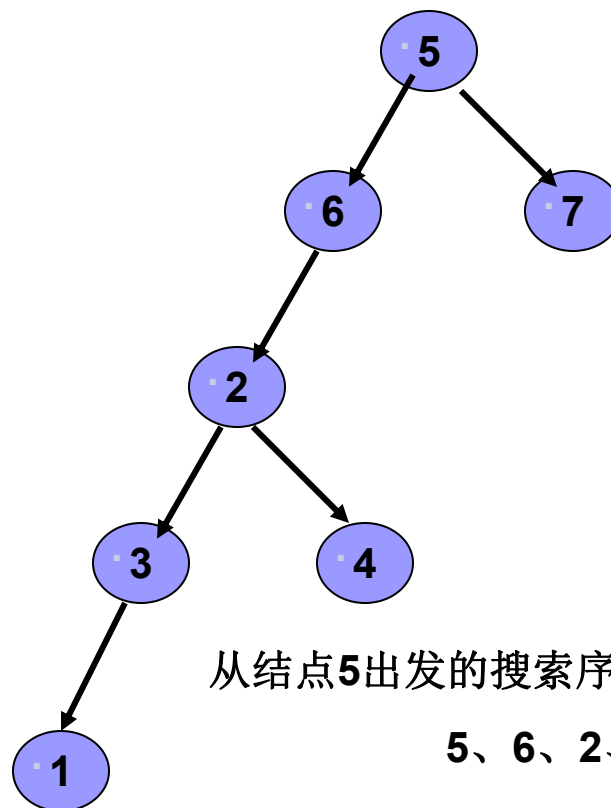


2、图的遍历

- 从图中某个顶点出发游历图，访遍图中其余顶点，并且使图中的每个顶点仅被访问一次的过程。
- 图的常见的遍历形式：
 - 深度优先搜索：类似树的先根遍历，适用栈
 - 广度优先搜索：类似树的按层次遍历，适用队列

深度优先搜索遍历图

例子：为了说明问题，邻接结点的访问次序以序号为准。序号小的先访问。
如：结点 **5** 的邻接结点有两个 **6**、**7**，则先访问结点 **6**，再访问结点 **7**。

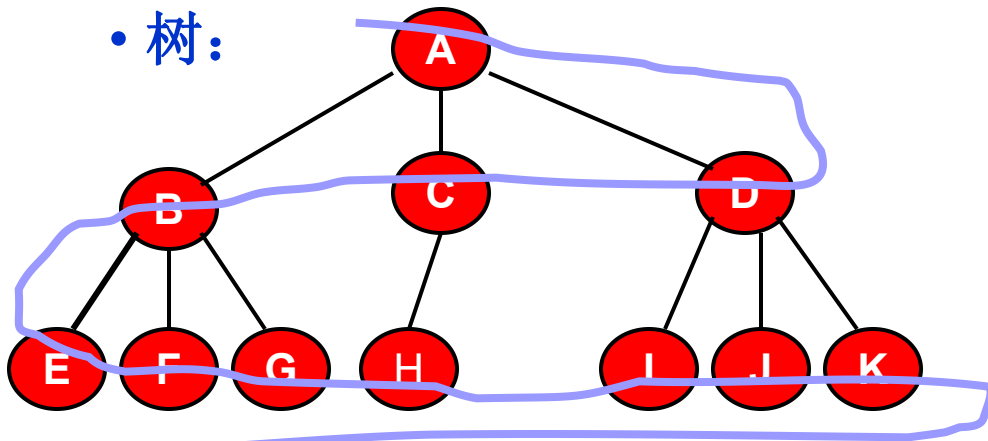


从结点**5**出发的搜索序列：

5、6、2、3、1、4、7

广度优先搜索遍历图

• 树:



树的按层次进行访问的次序:

A、B、C、D、E、F、G、H、
I、J、K、L

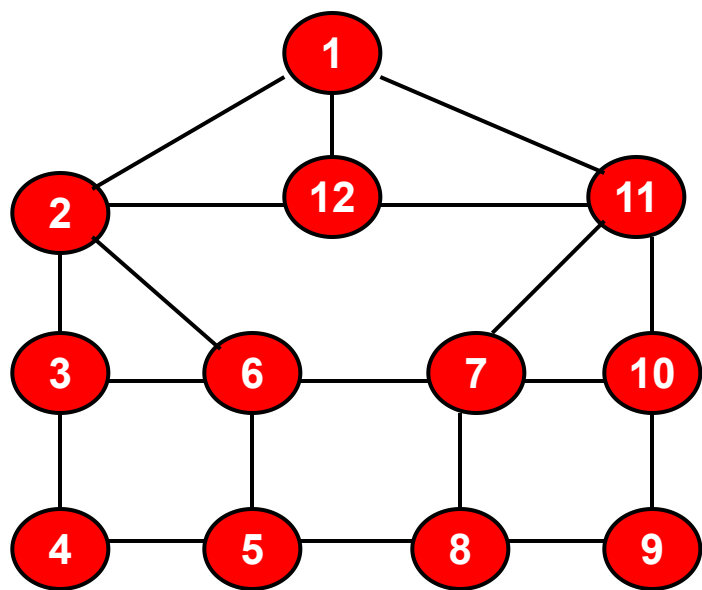
适用的数据结构: 队列

• 队列的变化:

·
·
·
·

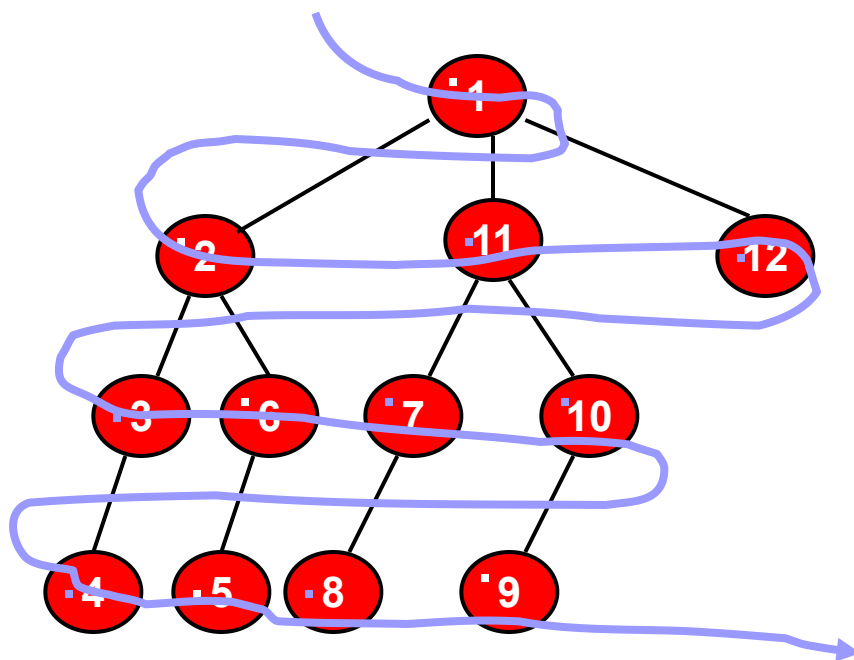
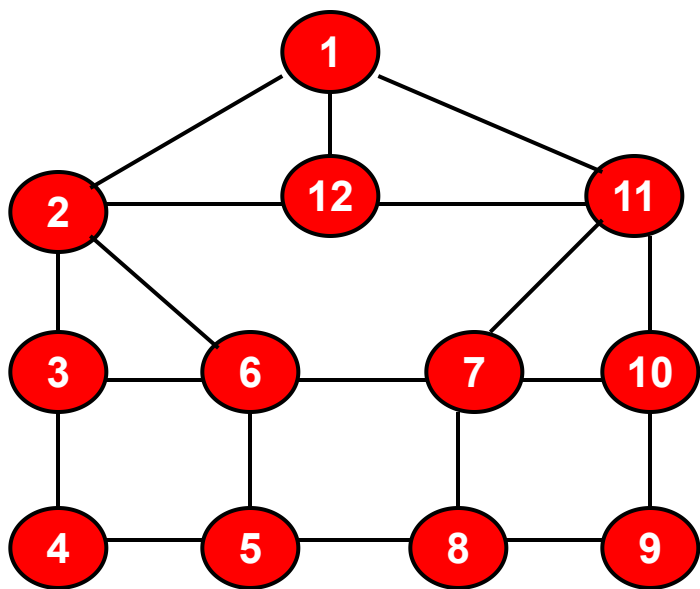
广度优先搜索遍历图

- 例子：为了说明问题，邻接结点的访问次序以序号为准。序号小的先访问。
如：结点 1 的邻接结点有三个 2、12、11，则先访问结点 2、11，再访问12。



广度优先搜索遍历图

- 例子：为了说明问题，邻接结点的访问次序以序号为准。序号小的先访问。
如：结点 1 的邻接结点有三个 2、12、11，则先访问结点 2、11，再访问12。



图的广度优先的访问次序：

1、2、11、12、3、6、7、10、4、5、8、9

适用的数据结构：队列

3、有向图的强连通分量的求法

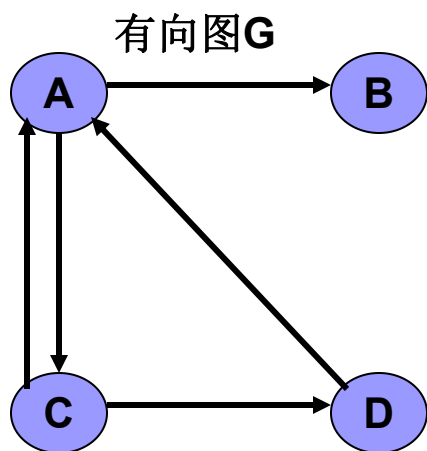
■ 强连通分量的求法：

- 1、从有向图**G**的某个顶点出发进行**深度优先搜索遍历**，按照所有邻接点的搜索都完成的顺序（即退出**DFS**函数的顺序）给结点进行编号。最先退出**DFS**函数的结点的编号为1，其它结点的编号按次序逐次增大1。
- 2、将有向图**G**的所有的有向边进行**倒置**，构造新的有向图记为**G_r**。
- 3、根据步骤1对结点进行的编号，选取**最大编号的结点**。从该结点出发，在有向图**G_r**上进行**深度优先搜索遍历**。如果没有访问到所有的结点，则从剩余的那些结点中选取编号最大的结点再进行深度为主的遍历。直至所有的结点都被访问到为止。
- 4、在最后得到的生成森林中的每一棵生成树，都是有向图**G**的强连通分量顶点集。

利用遍历求强连通分量的
时间复杂度和遍历相同。

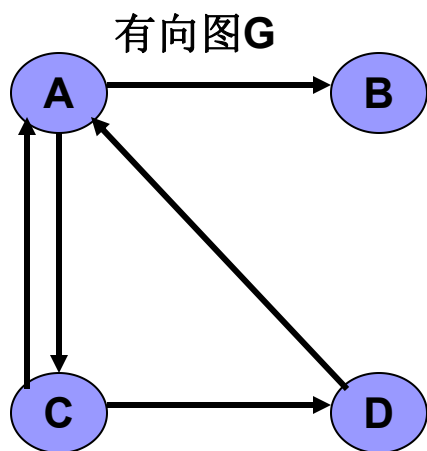
有向图的强连通分量的求法

■ 求法实例

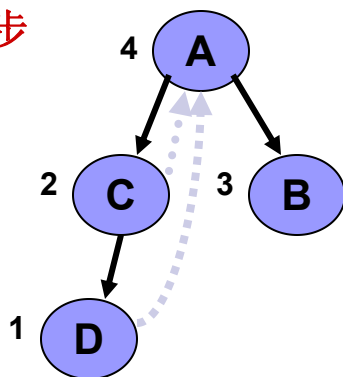


有向图的强连通分量的求法

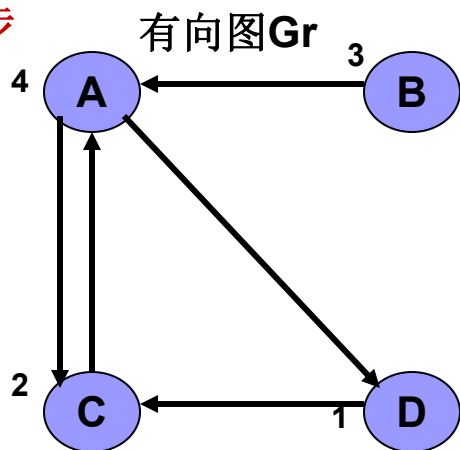
■ 求法实例



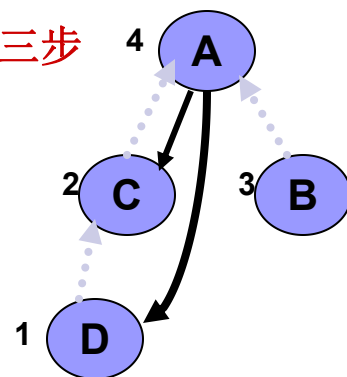
第一步



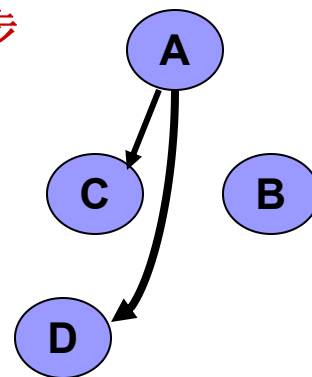
第二步



第三步



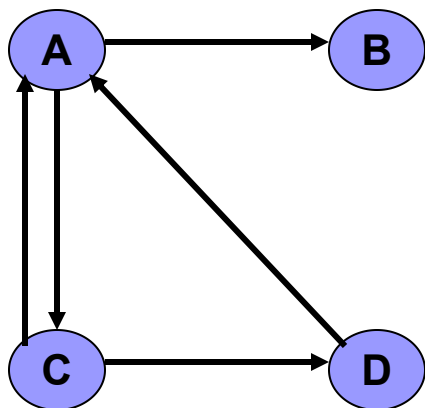
第四步



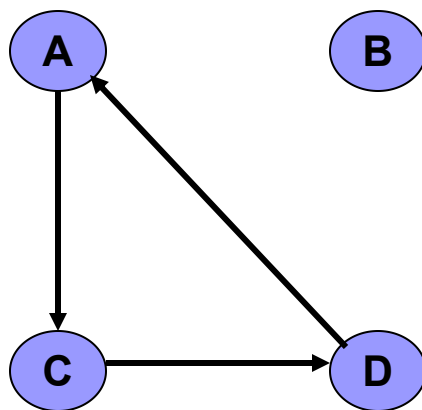
二棵生成树的结点是有向图G的两个强连通分量的顶点集：**{A、C、D}**及**{B}**

有向图的强连通分量的求法

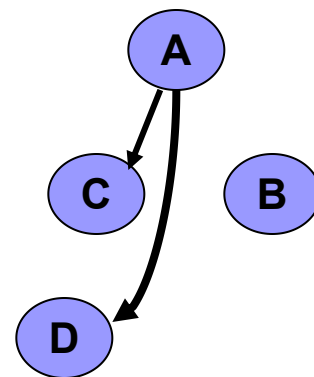
■ **断言**：有向图中的强连通分量中的顶点集和第二次在**Gr**图上深度优先搜索时得到的生成森林中的每一棵树的结点集精确对应。



有向图G



有向图G的两个强连通分量



二棵生成树的结点
是有向图G的两个
强连通分量的顶点
集：**{A、C、D}**及
{B}

3、有向图的强连通分量的求法

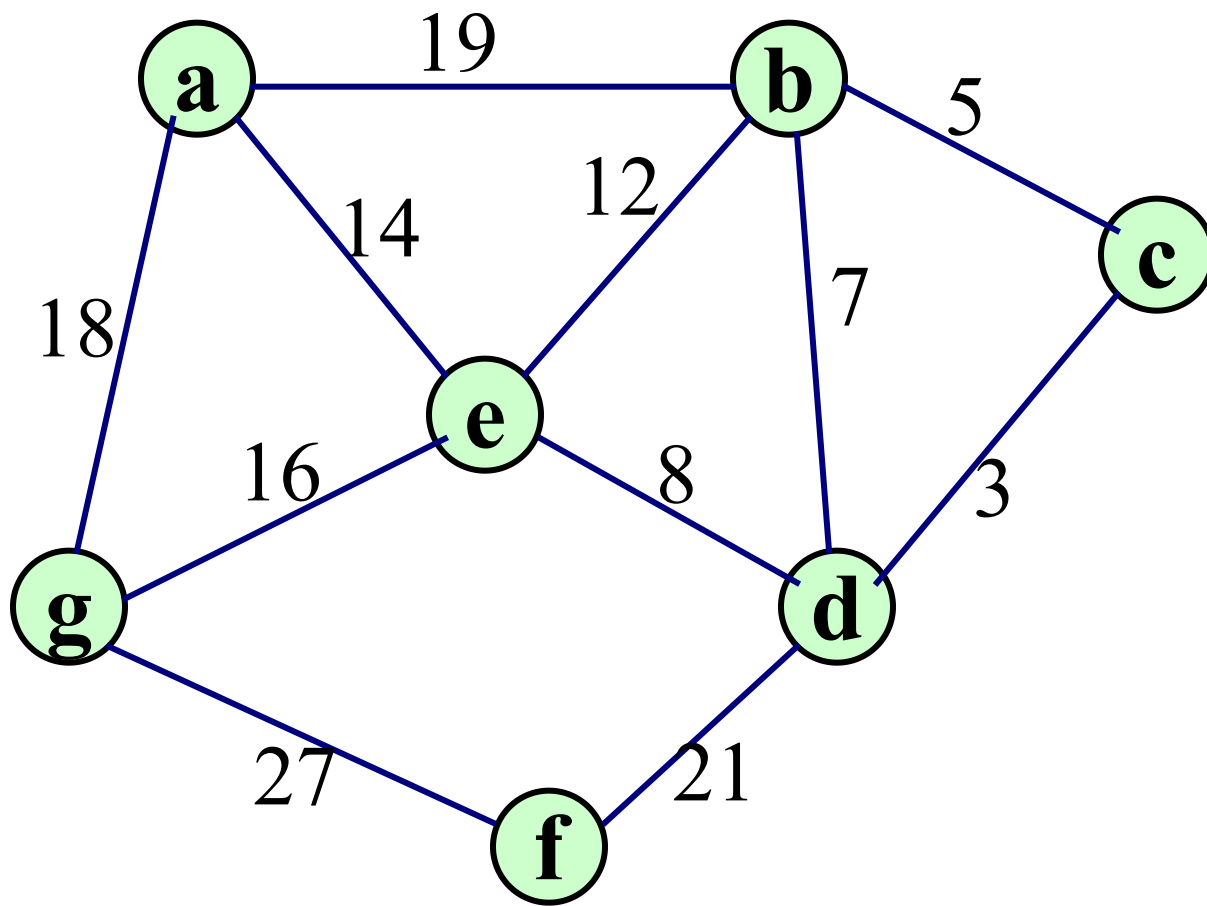
■ 强连通分量的求法：

- 1、从有向图**G**的某个顶点出发进行**深度优先搜索遍历**，按照所有邻接点的搜索都完成的顺序（即退出**DFS**函数的顺序）给结点进行编号。最先退出**DFS**函数的结点的编号为1，其它结点的编号按次序逐次增大1。
- 2、将有向图**G**的所有的有向边进行**倒置**，构造新的有向图记为**G_r**。
- 3、根据步骤1对结点进行的编号，选取**最大编号的结点**。从该结点出发，在有向图**G_r**上进行**深度优先搜索遍历**。如果没有访问到所有的结点，则从剩余的那些结点中选取编号最大的结点再进行深度为主的遍历。直至所有的结点都被访问到为止。
- 4、在最后得到的生成森林中的每一棵生成树，都是有向图**G**的强连通分量顶点集。

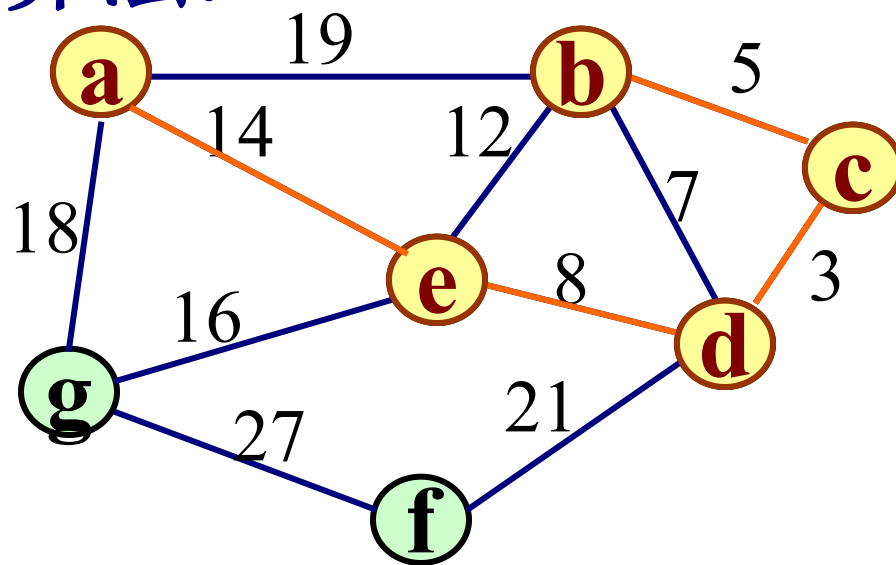
利用遍历求强连通分量的
时间复杂度和遍历相同。

4、最小生成树：

- 解决问题：假设要在 n 个城市之间建立通讯联络网，则连通 n 个城市只需要修建 $n-1$ 条线路，如何在最节省经费的前提下建立这个通讯网？
- 普里姆（Prim）算法
 - 利用MST最小生成树性质。
 - 设置一个辅助数组closedge，对当前 $V-U$ 集中的每个顶点，记录和顶点集 U 中顶点相连接的代价最小的边：
- 克鲁斯卡尔（Kruscal）算法
 - 为使生成树上边的权值之和达到最小，则应使生成树中每一条边的权值尽可能地小。
 - 先构造一个只含 n 个顶点的子图SG，然后从权值最小的边开始，若它的添加不使SG中产生回路，则在SG上加上这条边，如此重复，直至加上 $n-1$ 条边为止。

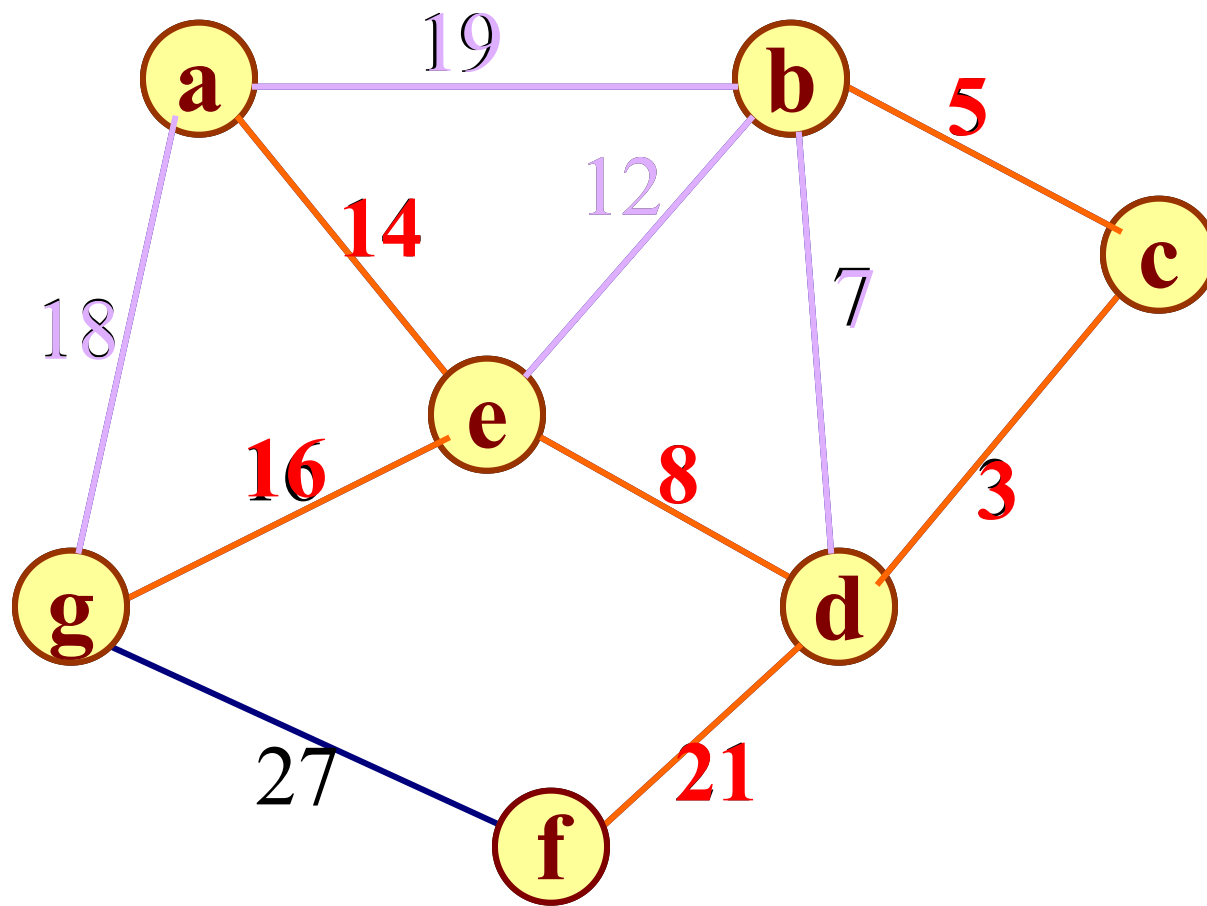


Prim算法:



closededge \ Adjvex	0	1	2	3	4	5	6
Lowcost		c	d	e	a	d	e
		5	3	8	14	21	16

Kruscal算法:



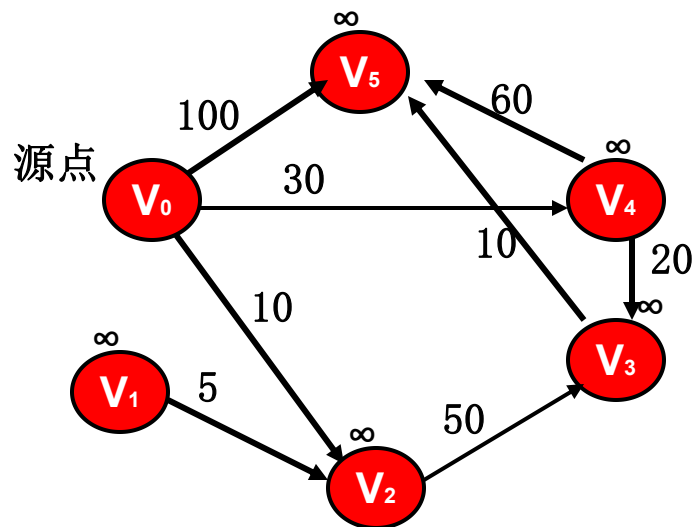
5、关键路径问题：

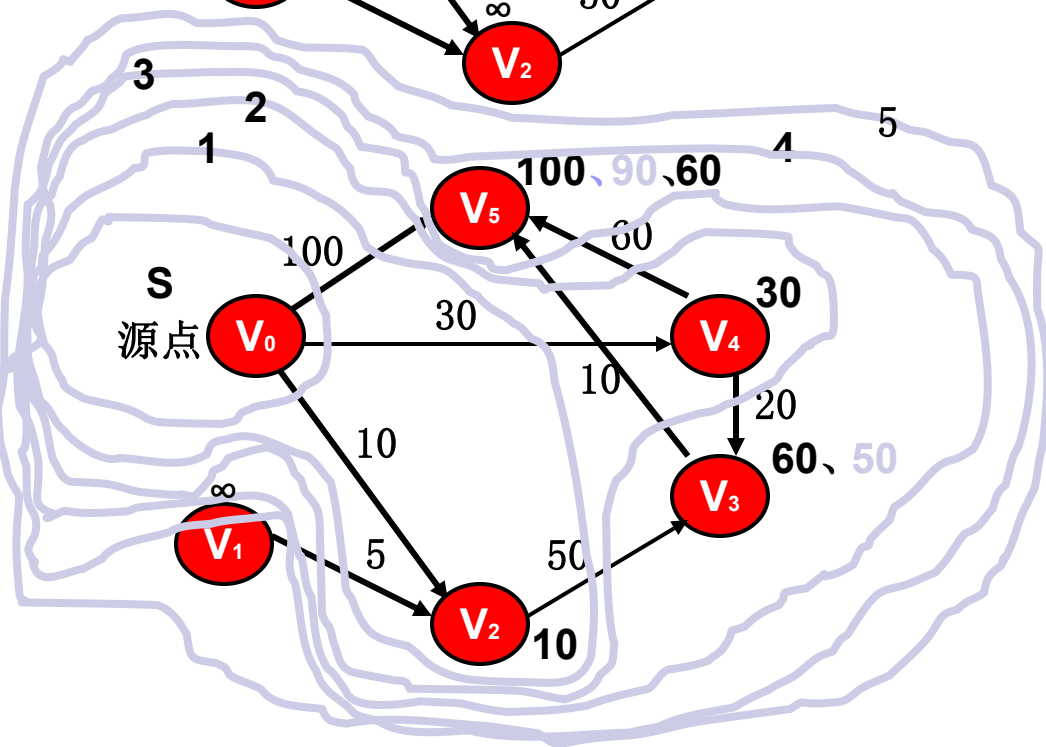
- 解决估算工程项目完成时间的问题！！！！
- 确定关键活动：
 - 求事件的最早发生时间——拓扑排序算法，邻接表
 - 求事件的最迟发生时间——逆拓扑排序算法，逆邻接表
 - 求活动的最早开始时间
 - 由事件的最早发生时间推导
 - 求活动的最迟开始时间
 - 由事件的最迟发生时间推导

6、最短路径问题:

- 求两个顶点之间的一条路径长度最短的路径: 基于广度优先搜索遍历进行, 但需要修改链队列的结点结构及其入队列和出队列的算法。
- 求从某个源点到其余各点的最短路径: 迪杰斯特拉 (Dijkstra) 算法, 依最短路径的长度递增的次序求得各条路径, 设置辅助数组 **Dist**, 其中每个分量 **Dist[k]** 表示当前所求得的从源点到其余各顶点 **k** 的最短路径。
- 每一对顶点之间的最短路径: 弗洛伊德 (Floyd) 算法, 从 v_i 到 v_j 的所有可能存在的路径中, 选出一条长度最短的路径。

Dijkstra算法



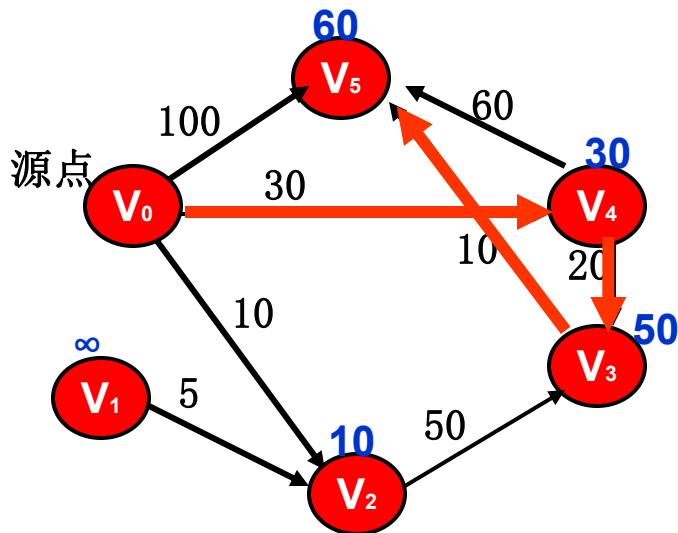


Dijkstra 算法求单源最短路径:

设 V 是该有向图的结点的集合、集合 S 是已求得最短路径的结点的集合, 求 v_0 至其余各结点的最短距离。

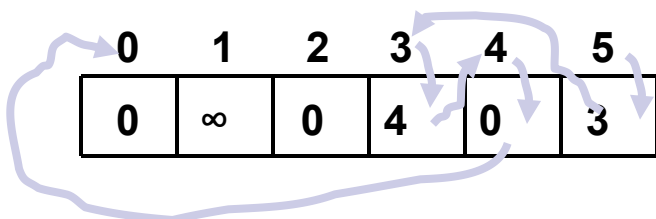
- 1、 $S = \{0\}$; // 结点 V_0 最短路径已求得
- 2、 for ($i=1; i<n; i++$)
- 3、 $D[i]=c[0, i]$; // V_0 至 V_i 的边长
- 4、 for ($i=1; i<n; i++$)
- 5、 { 在 $V-S$ 中选择一个结点 V_w ; 使得
 $D[w]$ 最小。将 W 加入集合 S
- 6、 for (每一个在 $V-S$ 中的结点 V)
- 7、 { $D[v]=\text{MIN}(D[v], D[w]+C[w,v])$;
- 8、 }
- 9、 }

求从某个源点到其余各顶点的最短路径



• Dijkstra 算法:

路径的求法: 在 8、处增加 $P[v] = w$;



• Dijkstra 算法的时间复杂性:

邻接矩阵: $O(n*n)$

Dijkstra 算法求单源最短路径:

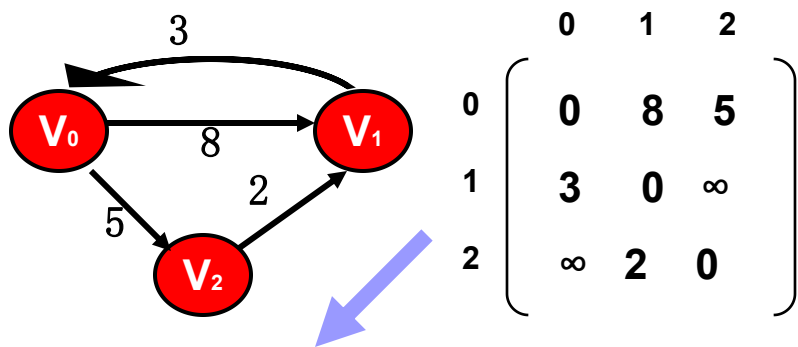
设 V 是该有向图的结点的集合、集合 S 是已求得最短路径的结点的集合, 求 V_0 至其余各结点的最短距离。

- 1、 $S = \{0\}$; // 结点 V_0 最短路径已求得
- 2、for ($i=1; i<n; i++$)
- 3、 $D[i]=c[0, i]$; // V_0 至 V_i 的边长
- 4、for ($i=1; i<n; i++$)
- 5、{ 在 $V-S$ 中选择一个结点 V_w ; 使得 $D[w]$ 最小。将 W 加入集合 S
- 6、for (每一个在 $V-S$ 中的结点 V)
- 7、 $\{ D[v]=MIN(D[v], D[w]+C[w,v])$;
- 8、如果 $D[V]$ 改变, 则 $P[v] = w$;
- 9、 }

每一对顶点之间的最短路径:

弗洛伊德 (Floyd) 算法

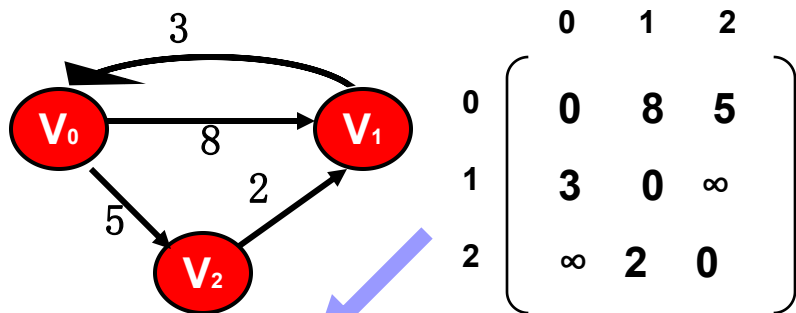
- 实例及求解过程:



每一对顶点之间的最短路径:

弗洛伊德 (Floyd) 算法

实例及求解过程:



$$\begin{bmatrix} 0 & 8 & 5 \\ 3 & 0 & \infty \\ \infty & 2 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 8 & 5 \\ 3 & 0 & 8 \\ \infty & 2 & 0 \end{bmatrix}$$

$A_{\text{初值}}$

A_0

$$\begin{bmatrix} 0 & 8 & 5 \\ 3 & 0 & 8 \\ 5 & 2 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 7 & 5 \\ 3 & 0 & 8 \\ 5 & 2 & 0 \end{bmatrix}$$

A_1

A_2

Floyd 算法的求解过程

设 C 为 n 行 n 列的代价矩阵, $c[i, j]$ 为 $i \rightarrow j$ 的权值。如果 $i=j$; 那么 $c[i, j]=0$ 。如果 i 和 j 之间无有向边; 则 $c[i, j]=\infty$

1、使用 n 行 n 列的矩阵 A 用于计算最短路径。

初始时, $A[i, j] = c[i, j]$

2、进行 n 次迭代

在进行第 k 次迭代时, 我们将使用如下的公式:

$$A_k[i, j] = \min \begin{cases} A_{k-1}[i, j] \\ A_{k-1}[i, k] + A_{k-1}[k, j] \end{cases}$$

注意: 第 k 次迭代时, 针对结点 k 进行。原 A_{k-1} 矩阵的第 k 行, 第 k 列保持不变。左上至右下的对角线元素也不变。

7.6.2 每一对顶点之间的最短路径

弗洛伊德（Floyd）算法的基本思想是：

从 V_i 到 V_j 的所有可能存在的路径中，选出一条长度最短的路径。

本章要点

1. 熟悉图的各种存储结构及其构造算法，了解实际问题的求解效率与采用何种存储结构和算法有密切联系。
2. 熟练掌握图的两​​种搜索路径的遍历：遍历的逻辑定义、深度优先搜索和广度优先搜索的算法。

在学习中应注意图的遍历算法与树的遍历算法之间的类似和差异。

3. 应用图的遍历算法求解各种简单路径问题。
4. 理解教科书中讨论的各种图的算法。
5. 熟练掌握两种最小生成树算法。
6. 熟悉有向无环图的两个重要应用：拓扑排序和关键路径。
7. 了解最短路径算法。

■ 推荐作业:

■ 7.1, 7.15, 7.16

■ 7.3, 7.4, 7.25, 7.27

■ 7.7, 7.9, 7.33

■ 7.10, 7.11, 7.13, 7.36